

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
ESCOLA DE ENGENHARIA
DEPARTAMENTO DE ENGENHARIA ELÉTRICA

Enzo Victor Zucchetti Pietta

Comparação de Desempenho de Redes Neurais em Diferentes Plataformas de Hardware

Porto Alegre

2025

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
ESCOLA DE ENGENHARIA
DEPARTAMENTO DE ENGENHARIA ELÉTRICA

Enzo Victor Zucchetti Pietta

Comparação de Desempenho de Redes Neurais em Diferentes Plataformas de Hardware

Projeto de Diplomação II, apresentado ao Departamento de Engenharia Elétrica da Escola de Engenharia da Universidade Federal do Rio Grande do Sul, como requisito parcial para a obtenção do grau de Engenheiro Eletricista

UFRGS

Orientador: Prof. Dr. José Rodrigo Furlanetto de Azambuja

Porto Alegre

2025

Enzo Victor Zucchetti Pietta

Comparação de Desempenho de Redes Neurais em Diferentes Plataformas de Hardware

Projeto de Diplomação II, apresentado ao Departamento de Engenharia Elétrica da Escola de Engenharia da Universidade Federal do Rio Grande do Sul, como requisito parcial para a obtenção do grau de Engenheiro Eletricista

BANCA EXAMINADORA

Prof. Dr. Tiago R. Balen
UFRGS

Prof. Dr. Tiago O. Weber
UFRGS

Prof. Dr. José Rodrigo F. de Azambuja
Orientador - UFRGS

Agradecimentos

A minha família pelo apoio e incentivo.

Aos meus amigos e colegas pelo companheirismo e amizade.

Ao meu orientador, José Rodrigo Furlanetto de Azambuja, pela orientação, paciência e conhecimentos transmitidos durante a realização do trabalho.

Resumo

Este trabalho investigou o desempenho de Redes Neurais Convolucionais (CNN) aplicadas à classificação de imagens em três plataformas de hardware distintas: CPU, GPU e FPGA. Para isso, foram utilizados três conjuntos de dados (MNIST, CIFAR-10 e SAT-6), cada um associado a uma rede neural específica. Inicialmente, implementaram-se modelos em precisão total, executados na CPU e GPU, e, em paralelo, desenvolveu-se uma versão quantizada dessas redes para implementação em FPGA por meio do *framework* FINN. As principais métricas de avaliação contemplaram acurácia, latência, vazão e dissipação de potência, possibilitando uma análise abrangente de desempenho. Os resultados indicaram que a GPU se sobressai em termos de vazão, sobretudo em cenários com lotes grandes, graças ao processamento maciço e paralelo. No entanto, apresenta dissipação de potência mais elevada e sofre com latências maiores para lotes unitários, em função dos custos de transferência de dados. Já a CPU obteve latências baixas para inferências individuais, mantendo um consumo energético intermediário e um desempenho adequado para aplicações que não exigem alta taxa de processamento simultâneo. A implementação em FPGA, por sua vez, demonstrou vantagens em cenários que priorizam eficiência energética e latências mais previsíveis, especialmente em lotes pequenos. Mesmo com as limitações de recursos e a necessidade de quantização, o hardware reconfigurável alcançou acurácias semelhantes aos modelos em precisão total, evidenciando o potencial do FINN para acelerar redes neurais embarcadas. Assim, a escolha da plataforma mais apropriada depende dos requisitos específicos de cada aplicação, balanceando aspectos de velocidade, consumo de energia e complexidade do projeto.

Palavras-chave: Redes neurais. FPGA. FINN. CPU. GPU. Inferência. Classificação de imagens. Quantização. Eficiência energética.

Abstract

This work investigated the performance of Convolutional Neural Networks (CNNs) applied to image classification on three distinct hardware platforms: CPU, GPU, and FPGA. To achieve this, three datasets (MNIST, CIFAR-10, and SAT-6) were used, each associated with a specific neural network. Initially, full-precision models were implemented and executed on the CPU and GPU. Simultaneously, a quantized version of these networks was developed for FPGA implementation using the FINN framework. The main evaluation metrics included accuracy, latency, throughput, and power dissipation, enabling a comprehensive performance analysis. The results indicated that the GPU excelled in terms of throughput, particularly in scenarios with large batch sizes, due to its massive parallel processing capabilities. However, it exhibited higher power dissipation and faced increased latency for single-batch inferences due to data transfer overhead. The CPU, on the other hand, achieved low latencies for individual inferences, maintained intermediate energy consumption, and delivered adequate performance for applications not requiring high simultaneous processing rates. The FPGA implementation demonstrated advantages in scenarios prioritizing energy efficiency and more predictable latencies, especially with small batch sizes. Despite resource limitations and the need for quantization, the reconfigurable hardware achieved accuracy levels comparable to full-precision models, highlighting the potential of FINN for accelerating embedded neural networks. Therefore, the choice of the most suitable platform depends on the specific requirements of each application, balancing speed, energy consumption, and project complexity considerations.

Keywords: Neural networks. FPGA. FINN. CPU. GPU. Inference. Image classification. Quantization. Energy efficiency.

Lista de Figuras

Figura 1	–	Representação gráfica do <i>Perceptron</i> .	16
Figura 2	–	Funções de ativação comuns.	18
Figura 3	–	Representação gráfica de uma camada totalmente conectada.	19
Figura 4	–	Representação gráfica das camadas de uma Rede Convolutacional.	21
Figura 5	–	Unidades básicas de uma CPU.	25
Figura 6	–	Arquitetura básica de uma GPU.	27
Figura 7	–	Visão geral da arquitetura de um FPGA.	29
Figura 8	–	Fluxo completo do FINN.	39
Figura 9	–	Exemplos de imagens do <i>dataset</i> MNIST.	44
Figura 10	–	Exemplos de imagens do <i>dataset</i> CIFAR-10.	45
Figura 11	–	Exemplos de imagens do <i>dataset</i> SAT-6.	47
Figura 12	–	GPU NVIDIA TITAN X (Pascal).	49
Figura 13	–	Placa PYNQ-Z1.	51
Figura 14	–	Fluxograma com visão geral das principais etapas da metodologia.	55
Figura 15	–	Representação gráfica da topologia para classificação do <i>dataset</i> MNIST.	56
Figura 16	–	Representação gráfica da topologia para classificação do <i>dataset</i> CIFAR-10.	56
Figura 17	–	Representação gráfica da topologia para classificação do <i>dataset</i> SAT-6.	57
Figura 18	–	Fluxograma da visão geral do fluxo FINN realizado.	60
Figura 19	–	Latência por iterações para o <i>dataset</i> MNIST.	67
Figura 20	–	Latência por iterações para o <i>dataset</i> CIFAR.	68
Figura 21	–	Latência por iterações para o <i>dataset</i> SAT-6.	69
Figura 22	–	Vazão por tamanho de lote para o <i>dataset</i> MNIST na CPU.	70
Figura 23	–	Vazão por tamanho de lote para o <i>dataset</i> CIFAR na CPU.	71
Figura 24	–	Vazão por tamanho de lote para o <i>dataset</i> SAT na CPU.	71
Figura 25	–	Dissipação de potência para o <i>dataset</i> MNIST na CPU.	73
Figura 26	–	Dissipação de potência para o <i>dataset</i> CIFAR na CPU.	73
Figura 27	–	Dissipação de potência para o <i>dataset</i> SAT na CPU.	74
Figura 28	–	Latência por iterações para o <i>dataset</i> MNIST na GPU.	76
Figura 29	–	Latência por iterações para o <i>dataset</i> CIFAR na GPU.	77
Figura 30	–	Latência por iterações para o <i>dataset</i> SAT-6 na GPU.	78
Figura 31	–	Vazão por tamanho de lote para o <i>dataset</i> MNIST na GPU.	79
Figura 32	–	Vazão por tamanho de lote para o <i>dataset</i> CIFAR-10 na GPU.	80
Figura 33	–	Vazão por tamanho de lote para o <i>dataset</i> SAT-6 na GPU.	80
Figura 34	–	Dissipação de potência para o <i>dataset</i> MNIST na GPU.	82
Figura 35	–	Dissipação de potência para o <i>dataset</i> CIFAR-10 na GPU.	82
Figura 36	–	Dissipação de potência para o <i>dataset</i> SAT-6 na GPU.	83

Figura 37 – Modelo de precisão total ONNX para o <i>dataset</i> MNIST.	97
Figura 38 – Modelo FINN para o <i>dataset</i> MNIST.	98
Figura 39 – Modelo de precisão total ONNX para o <i>dataset</i> CIFAR-10.	99
Figura 40 – Modelo FINN para o <i>dataset</i> CIFAR-10.	100
Figura 41 – Modelo de precisão total ONNX para o <i>dataset</i> SAT-6.	101
Figura 42 – Modelo FINN para o <i>dataset</i> SAT-6.	102

Lista de Tabelas

Tabela 1	– Detalhes da topologia da rede neural baseada na arquitetura LeNet-5 para classificação do <i>dataset</i> MNIST.	57
Tabela 2	– Detalhes da topologia baseada em exemplo do projeto TinyDNN para classificação do <i>dataset</i> CIFAR-10.	57
Tabela 3	– Descrição da topologia baseado no trabalho de Ely <i>et al.</i> (2024) para classificação do <i>dataset</i> SAT-6.	57
Tabela 4	– Tamanhos de entrada das redes neurais utilizadas.	58
Tabela 5	– Mapeamento de funções entre PyTorch e Brevitas.	58
Tabela 6	– Parâmetros Comuns de Treinamento.	63
Tabela 7	– Desempenho e Potência para Inferência na CPU.	66
Tabela 8	– Desempenho e Potência para Inferência na GPU.	75
Tabela 9	– Desempenho e Potência para Inferência em FPGA.	84
Tabela 10	– Utilização de Recursos no FPGA para Classificação do MNIST.	85
Tabela 11	– Utilização de Recursos no FPGA para Classificação do CIFAR-10.	85
Tabela 12	– Utilização de Recursos no FPGA para Classificação do SAT-6.	85
Tabela 13	– Acurácia (%) por plataforma e <i>dataset</i>	87
Tabela 14	– Latência (μs) por plataforma e <i>dataset</i>	87
Tabela 15	– Vazão máxima (fps) por plataforma e <i>dataset</i>	87
Tabela 16	– Potência (W) por plataforma e <i>dataset</i>	87

Lista de abreviaturas

ADC	<i>Aalog-to-Digital Converter</i>
ASIC	<i>Application-Specific Integrated Circuit</i>
CIFAR-10	<i>Canadian Institute for Advanced Research - 10 Classes</i>
CLB	<i>Configurable Logic Block</i>
CPU	<i>Central Processing Unit</i>
CNN	<i>Convolutional Neural Network</i>
DMA	<i>Direct Access Memory</i>
DSP	<i>Digital Signal Processing</i>
DWC	<i>Data Width Converter</i>
FIFO	<i>First-In-First-Out</i>
FP32	<i>Floating Point 32</i>
FP64	<i>Floating Point 64</i>
FPGA	<i>Field-Programmable Gate Array</i>
GeoTIFF	<i>Georeferenced Tagged Image File Format</i>
GPU	<i>Graphics Processing Unit</i>
HLS	<i>High-Level Synthesis</i>
INT1	<i>Integer 1-bit</i>
INT8	<i>Integer 8-bit</i>
IoT	<i>Internet of Things</i>
IR	<i>Intermediate Representation</i>
I/O	<i>Input/Output</i>
LUT	<i>Look-Up Table</i>
MAC	<i>Multiply-Accumulate</i>
MLP	<i>Multilayer Perceptron</i>
MMCM	<i>Mixed-Mode Clock Manager</i>
MNIST	<i>Modified National Institute of Standards and Technology</i>

NAIP	<i>National Agriculture Imagery Program</i>
NIR	<i>Near-Infrared</i>
NIST	<i>National Institute of Standards and Technology</i>
ONNX	<i>Open Neural Network Exchange</i>
PLL	<i>Phase-Locked Loop</i>
PTQ	<i>Post-Training Quantization</i>
PYNQ	<i>Python Productivity for Zynq</i>
QAT	<i>Quantization-Aware Training</i>
QNN	<i>Quantized Neural Network</i>
QONNX	<i>Quantized ONNX</i>
ReLU	<i>Rectified Linear Unit</i>
RNA	<i>Rede Neural Artificial</i>
RNN	<i>Recurrent Neural Network</i>
RTL	<i>Register-Transfer Level</i>
SoC	<i>System-on-Chip</i>
VHDL	<i>VHSIC Hardware Description Language</i>
VHSIC	<i>Very High-Speed Integrated Circuit</i>

Sumário

1	INTRODUÇÃO	14
2	FUNDAMENTAÇÃO TEÓRICA	16
2.1	Redes Neurais Artificiais (RNAs)	16
2.1.1	Redes Multicamadas	18
2.1.2	Redes Convolucionais	20
2.2	Métricas de Desempenho	23
2.2.1	Acurácia	24
2.2.2	Latência	24
2.2.3	Vazão	24
2.2.4	Dissipação de potência	24
2.3	Arquiteturas de Hardware	25
2.3.1	Unidade Central de Processamento	25
2.3.2	Unidade de Processamento Gráfica	26
2.3.3	Circuitos Integrados para Aplicação Específica	28
2.3.4	Matriz de Portas Programável em Campo	29
2.4	Quantização	31
2.4.1	Quantização Pós-Treinamento	32
2.4.2	Quantização Consciente de Treinamento	33
3	FERRAMENTAS E FRAMEWORKS	34
3.1	Kaggle	34
3.2	PyTorch	35
3.3	Open Neural Network Exchange - ONNX	35
3.4	Netron	36
3.5	Brevitas	36
3.6	ONNX Runtime	37
3.7	Vivado	37
3.8	FINN	38
3.8.1	Exportação do modelo FINN-ONNX pelo Brevitas	40
3.8.2	Preparação da Rede	40
3.8.3	Implementação em Hardware	41
4	DATASETS	43
4.1	Instituto Nacional de Padrões e Tecnologia Modificado	43
4.1.0.1	Estrutura dos dados	43

4.1.0.2	Características e Aplicações	43
4.2	Instituto Canadense de Pesquisa Avançada - 10 classes	44
4.2.0.1	Estrutura dos dados	44
4.2.0.2	Características e Aplicações	45
4.3	DeepSat SAT-6	46
4.3.0.1	Estrutura dos dados	46
4.3.0.2	Características e Aplicações	46
5	PLATAFORMAS UTILIZADAS	48
5.1	Intel Core i5	48
5.2	NVIDIA TITAN X (Pascal)	49
5.3	Pynq Board Z1	51
6	METODOLOGIA	54
6.1	Aquisição dos Dados	55
6.2	Topologia com Precisão Total	56
6.3	Topologia Quantizada	58
6.4	Fluxo FINN	59
6.4.1	Conversão para Formato FINN	60
6.4.2	Pré e Pós Processamento	60
6.4.3	Transformações na Rede	61
6.4.4	Parâmetros de Compilação	62
6.4.5	Produtos Finais	62
6.5	Treinamento	62
6.6	Avaliação	64
6.6.1	Avaliação na CPU e GPU	64
6.6.2	Avaliação no FPGA	65
7	RESULTADOS E DISCUSSÕES	66
7.1	Desempenho na CPU	66
7.1.1	Latência	66
7.1.2	Vazão	70
7.1.3	Dissipação de Potência	72
7.2	Desempenho na GPU	75
7.2.1	Latência	75
7.2.2	Vazão	79
7.2.3	Dissipação de Potência	81
7.3	Desempenho no FPGA	84
7.3.1	Latência	84
7.3.2	Vazão	84

7.3.3	Dissipação de potência	85
7.3.4	Utilização de Recursos	85
7.3.5	Modelos ONNX	86
7.4	Comparação Final entre as Plataformas	87
8	CONCLUSÕES	89
8.1	Trabalhos Futuros	90
8.2	Considerações Finais	90
	REFERÊNCIAS BIBLIOGRÁFICAS	92
	 APÊNDICES	 96
	APÊNDICE A – MODELOS ONNX	97
A.1	Modelos para o <i>dataset</i> MNIST	97
A.2	Modelos para o <i>dataset</i> CIFAR-10	99
A.3	Modelos para o <i>dataset</i> SAT-6	101

1 Introdução

O avanço exponencial da Inteligência Artificial (IA) e do Aprendizado de Máquina (AM) tem impactado profundamente diversas áreas do conhecimento e setores industriais. Desde reconhecimento de padrões e classificação de imagens até análise de texto e detecção de doenças, as redes neurais têm desempenhado um papel central, contribuindo significativamente para resolver desafios complexos de maneira eficiente (CHEN *et al.*, 2020). Essa transformação é impulsionada pela disponibilidade de grandes volumes de dados e pela evolução das capacidades computacionais, que viabilizam a implementação de algoritmos cada vez mais sofisticados (MOHAIDAT; KHALIL, 2024).

Entre as arquiteturas de aprendizado profundo, Redes Neurais Convolucionais (CNNs) destacam-se pela habilidade de capturar padrões complexos em dados visuais, sendo amplamente aplicadas em tarefas como reconhecimento facial, diagnóstico médico e sistemas autônomos (MITTAL, 2020). Entretanto, o aumento contínuo na complexidade das redes e no volume de dados processados impõe desafios significativos às plataformas de hardware convencionais, como a Unidade Central de Processamento (CPU) e a Unidade de Processamento Gráfica (GPU), devido às limitações em desempenho, eficiência energética e escalabilidade (BAEK; KWON; KIM, 2020).

FPGAs (Field-Programmable Gate Arrays) têm emergido como uma alternativa promissora para lidar com essas questões, oferecendo alta eficiência energética, flexibilidade de reconfiguração e a capacidade de realizar operações otimizadas diretamente no hardware (MITTAL, 2020). Essas características tornam os FPGAs ideais para aplicações em dispositivos embarcados e Internet das Coisas (IoT), onde a redução do consumo energético e a execução eficiente de redes neurais são essenciais. Além disso, a capacidade de realizar inferências diretamente na borda reduz significativamente a dependência de transferências intensivas de dados para servidores centrais, minimizando atrasos e o consumo energético associado à comunicação (BAEK; KWON; KIM, 2020).

Ferramentas modernas, como o FINN, desenvolvido pela Xilinx, têm facilitado a implementação de redes neurais quantizadas em FPGAs. Esse fluxo de trabalho especializado integra etapas de quantização, otimização e mapeamento para hardware, permitindo a execução eficiente de redes em dispositivos de borda. A abordagem do FINN é particularmente interessante em cenários onde previsibilidade de latência e eficiência energética são fatores críticos, como sistemas embarcados e aplicações em tempo real (BLOTT *et al.*, 2018). Além disso, o uso de redes quantizadas em FPGAs permite não apenas economizar recursos computacionais, mas também maximizar o desempenho dentro das limitações de hardware (MITTAL, 2020).

Apesar das vantagens, o uso de FPGAs na aceleração de redes neurais também envolve desafios. Entre eles, destacam-se a necessidade de adequações específicas para contornar as limitações de recursos desses dispositivos e o impacto na acurácia resultante da quantização do modelo (MITTAL, 2020; CHEN *et al.*, 2020).

Neste trabalho, busca-se explorar o potencial dos FPGAs na aceleração de redes neurais quantizadas, analisando e comparando seu desempenho com CPUs e GPUs em termos de métricas como acurácia, latência, vazão e consumo energético. Para isso, foram utilizadas três redes neurais distintas, aplicadas, respectivamente, aos conjuntos de dados MNIST, CIFAR-10 e SAT-6, e avaliadas em versões de precisão total e quantizadas.

Espera-se que os resultados obtidos contribuam para a identificação de direções promissoras para pesquisas futuras, bem como para a compreensão das vantagens e limitações das diferentes plataformas no contexto da aceleração de redes neurais e abordagens práticas no uso de FPGAs em sistemas embarcados e dispositivos de borda. Dessa forma, o trabalho se insere em um cenário crescente de demanda por soluções de inteligência artificial que combinem alta performance com eficiência energética, alinhando-se às necessidades de sistemas modernos e sustentáveis.

O trabalho está organizado da seguinte forma: no Capítulo 2 (Fundamentação Teórica), são apresentados os principais conceitos teóricos que fundamentam o desenvolvimento do projeto; na Seção 3 (Ferramentas e *Frameworks*), são descritas as principais ferramentas, bibliotecas e *frameworks* utilizados ao longo do trabalho; na Seção 4 (*Datasets*), é fornecida uma visão geral sobre os conjuntos de dados empregados; na Seção 5 (Plataformas Utilizadas), são detalhadas as plataformas de hardware específicas usadas na inferência das redes neurais; no Capítulo 6 (Metodologia), é descrita a metodologia adotada e o desenvolvimento do trabalho; no Capítulo 7 (Resultados e Discussões), são apresentados os resultados obtidos, juntamente com suas análises e discussões; e, finalmente, no Capítulo 8 (Conclusões), são expostas as considerações finais, além de sugestões para trabalhos futuros.

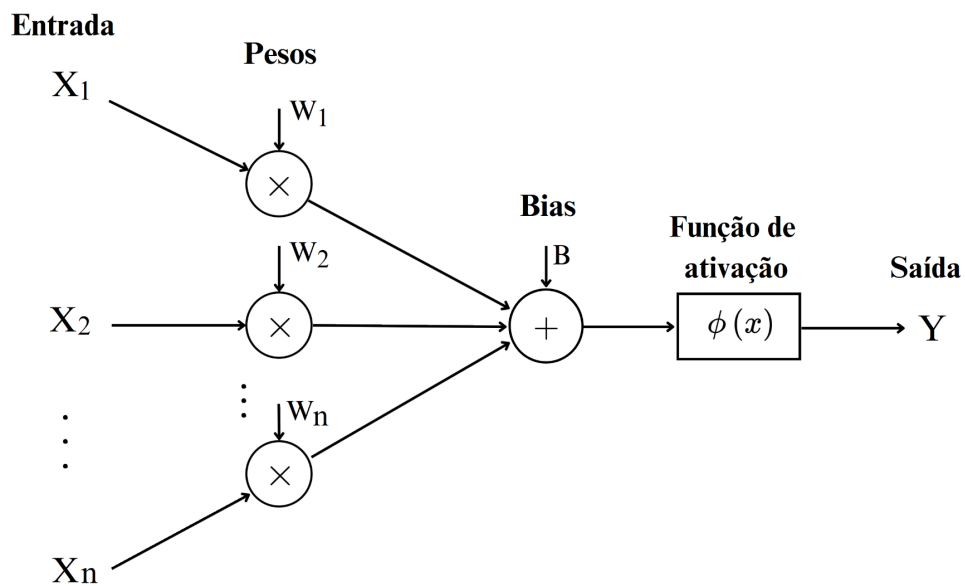
2 Fundamentação Teórica

2.1 Redes Neurais Artificiais (RNAs)

As redes neurais artificiais (RNAs) são modelos computacionais inspirados no funcionamento do cérebro humano. Compostas por “neurônios artificiais” organizados em camadas e conectados por “sinapses”, esses modelos ajustam seus pesos para aprender padrões em dados e resolver problemas complexos, como classificação, previsão e reconhecimento de padrões.

O conceito de redes neurais foi introduzido inicialmente por McCulloch e Pitts (1943), ao modelarem os neurônios como unidades matemáticas. Essa abordagem evoluiu com o desenvolvimento do *Perceptron*, proposto por Rosenblatt (1958), que é considerado o precursor das redes neurais modernas.

Figura 1 – Representação gráfica do *Perceptron*.



Fonte: o Autor, 2024.

Cada *Perceptron* ou neurônio, realiza uma combinação linear de suas entradas, ponderadas por pesos, e somadas por um *bias*, nessa combinação é aplicada, então, uma função de ativação, a qual introduz não linearidades ao sistema e permite a identificação de padrões complexos. Exemplos de funções de ativação comuns são as funções Unidade Linear Retificada (*Rectified Linear Unit* - ReLU), *Sigmoid* e *Tanh*, as quais podem ser observadas na Figura 2. A descrição matemática da saída e componentes do neurônio é apresentada na Equação 1.

$$y = \phi \left(\sum_{i=1}^n w_i x_i + b \right) \quad (1)$$

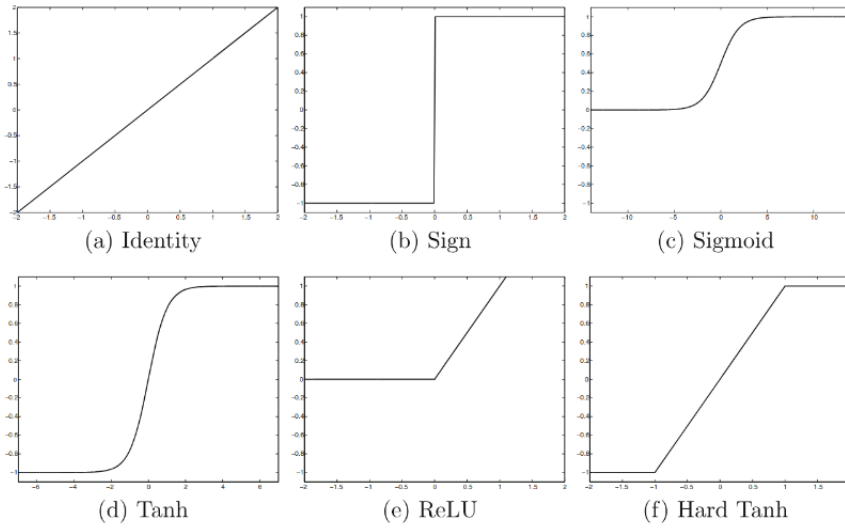
Onde:

- y é a saída do *Perceptron*;
- ϕ é a função de ativação;
- n é o número de entradas;
- w_i são os pesos associados às entradas;
- x_i são os valores das entradas;
- b é o termo de viés (*bias*).

O *Perceptron* é a forma mais básica de uma rede neural. Por outro lado, as redes multicamadas (*Multilayer Perceptron* - MLP) apresentam diversas camadas ocultas, sendo amplamente versáteis para diferentes tipos de tarefas. As redes convolucionais (*Convolutional Neural Networks* - CNNs) são especializadas no processamento de dados com estrutura espacial, como imagens, enquanto as redes recorrentes (*Recurrent Neural Networks* - RNNs) são voltadas para o processamento de sequências temporais, como texto ou séries temporais. Modelos mais recentes, como os *Transformers*, têm se destacado em tarefas de processamento de linguagem natural e tradução automática.

Como este trabalho foca nas CNNs, nas subseções seguintes serão apresentados inicialmente os conceitos das redes MLPs, que são fundamentais para compreender as CNNs e, em muitos casos, fazem parte de sua estrutura. Em seguida, serão introduzidos os conceitos específicos das CNNs.

Figura 2 – Funções de ativação comuns.



Fonte: o Autor, 2024.

2.1.1 Redes Multicamadas

A rede neural do tipo multicamadas (*Multilayer Perceptron* - MLP), introduzida formalmente no artigo de Rumelhart, Hinton e Williams (1986), é uma das arquiteturas mais fundamentais em aprendizado de máquina, amplamente reconhecida por sua simplicidade e eficácia em problemas supervisionados, como classificação e regressão. A MLP é composta por múltiplas camadas de neurônios interconectados, organizadas em camada de entrada, camadas ocultas e camada de saída. Para que uma rede seja classificada como MLP, ela deve conter pelo menos duas camadas ocultas, o que possibilita a modelagem de relações não lineares complexas no espaço de entrada.

A camada de entrada corresponde diretamente aos dados fornecidos ao modelo, onde cada neurônio recebe uma característica ou variável de entrada. As camadas ocultas são compostas por neurônios totalmente conectados (*fully connected*), o que significa que cada saída de uma camada é conectada a todos os neurônios da camada seguinte.

A camada de saída é composta pelos neurônios que produzem os resultados da rede, sendo que esses neurônios podem ou não incluir uma função de ativação. Em problemas de classificação, é comum que a camada de saída utilize a função de ativação *Softmax*, que transforma os valores de saída de forma que representem probabilidades associadas a cada classe. A descrição matemática da função *Softmax* é apresentada na Equação 2.

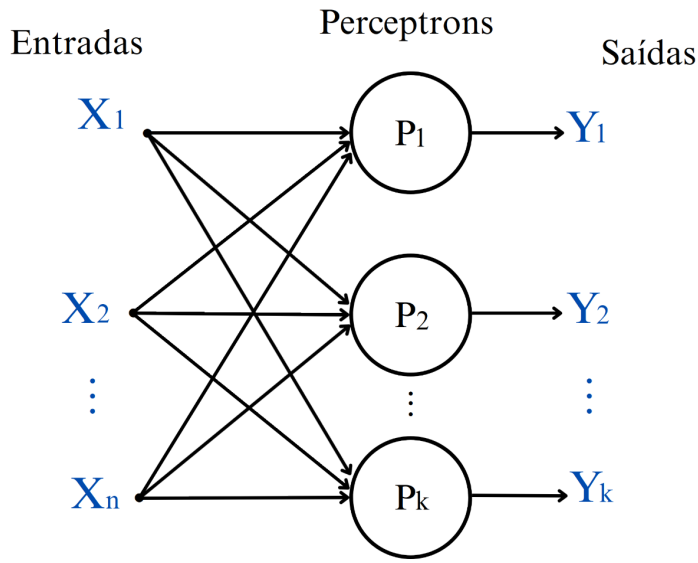
$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}, \quad \text{para } i = 1, \dots, k \quad (2)$$

Onde:

- z_i : valor de entrada para o neurônio i na camada de saída;
- k : número total de classes (ou neurônios na camada de saída);
- $\sigma(z_i)$: probabilidade associada à classe i , tal que $\sum_{i=1}^k \sigma(z_i) = 1$.

Essa função é amplamente utilizada em problemas de classificação multi-classes, garantindo que as saídas da rede sejam interpretáveis como probabilidades normalizadas. A classe com maior probabilidade é geralmente escolhida como a predição final do modelo.

Figura 3 – Representação gráfica de uma camada totalmente conectada.



Fonte: o Autor, 2024.

A representação matemática de uma camada oculta em uma MLP é apresentada na Equação 3. Essa equação é uma generalização da Equação 1, usada no modelo de *Perceptron*, com a diferença de considerar múltiplos neurônios interligados, gerando um vetor de saída em vez de um único valor. Vale notar que para computarmos toda a rede é necessário computar todas as camadas sequencialmente utilizando as saídas da camada anterior como as entradas da próxima camada.

$$\mathbf{y} = \phi(\mathbf{W}\mathbf{x} + \mathbf{b}) \quad (3)$$

onde:

- $\mathbf{x}$: vetor de entrada para a camada;
- $\mathbf{W}$: matriz de pesos que conecta os neurônios de entrada aos neurônios da camada;
- $\mathbf{b}$: vetor de vieses (*bias*) dos neurônios;

- $\phi(\cdot)$: função de ativação não linear aplicada elemento a elemento;
- $\mathbf{y}$: vetor de saída da camada após a ativação.

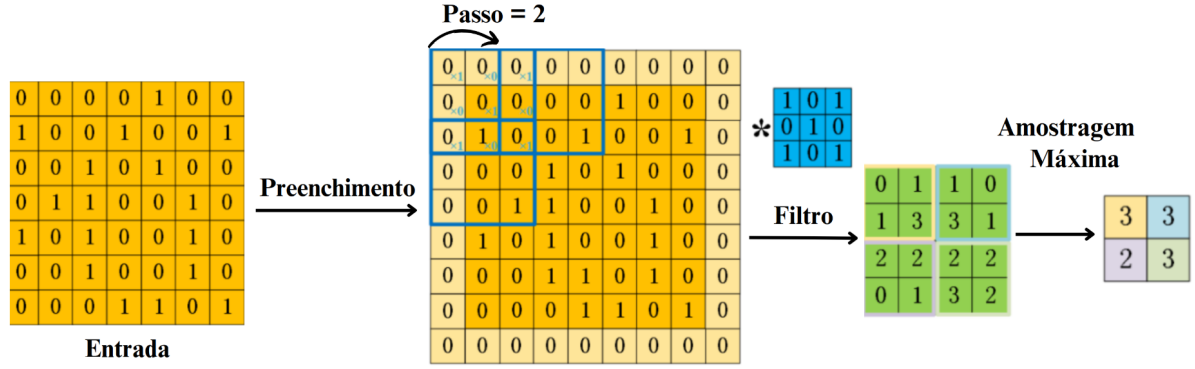
Além disso, a retropropagação, também introduzida por Rumelhart, Hinton e Williams (1986), é usada para ajustar os pesos das conexões durante o treinamento da MLP. O algoritmo de retropropagação permitiu que redes com mais de uma camada oculta possam ser treinadas fazendo com que seus erros entre as previsões e os valores reais sejam minimizados. Essa característica, aliada à flexibilidade estrutural da MLP, a torna uma ferramenta essencial no aprendizado de máquina, capaz de resolver problemas variados. Além disso, essa rede é bastante utilizada em conjunto com outras topologias, como as Redes Convolucionais apresentadas na Seção 2.1.2.

2.1.2 Redes Convolucionais

Redes Neurais Convolucionais (CNNs, do inglês *Convolutional Neural Networks*) são um tipo de rede neural projetada especificamente para o processamento de dados com uma estrutura de grade, como imagens, inspiradas na organização do córtex visual de animais, que responde a diferentes estímulos visuais em regiões espaço-específicas (LECUN *et al.*, 1998). Diferentemente de redes neurais tradicionais, as CNNs utilizam a operação matemática da convolução em substituição à multiplicação matricial geral em pelo menos uma de suas camadas. Essa abordagem tem sido amplamente bem-sucedida em aplicações práticas, tornando as CNNs especialmente adequadas para tarefas que envolvem padrões estruturados em grades, como o processamento de imagens (GOODFELLOW; BENGIO; COURVILLE, 2016, p. 330).

Os principais componentes que distinguem as CNNs de outras redes neurais são suas camadas convolucionais e camadas de Amostragem (do inglês *Pooling*). Além dessas, as CNNs também possuem funções de ativação e camadas totalmente conectadas, como nas redes MLP. Um exemplo das camadas básicas de uma rede convolucional pode ser observado na Figura 4. Cada uma dessas operações será detalhada a seguir.

Figura 4 – Representação gráfica das camadas de uma Rede Convolutional.



Fonte: Adaptado de Li *et al.* (2022).

O principal objetivo de uma Camada Convolutiva é extrair características relevantes dos dados de entrada. Ela realiza isso por meio da aplicação de uma convolução entre o sinal de entrada e um conjunto de filtros (também chamados de núcleos). Cada filtro é treinado para reconhecer um padrão específico, seja uma aresta vertical, horizontal ou mesmo texturas mais complexas. A saída de cada filtro constitui um mapa de características individual, o qual representa a presença e a intensidade desses padrões na entrada, servindo de base para as etapas subsequentes de análise e classificação da rede neural. Dessa forma, a profundidade do mapa de características equivale ao número de filtros da camada.

Nas primeiras camadas da rede, os filtros tipicamente aprendem padrões mais simples e locais. À medida que se avançam para camadas mais profundas, eles passam a detectar estruturas cada vez mais complexas e abstratas, chegando a identificar partes de objetos ou até mesmo objetos inteiros. A descrição matemática da convolução encontra-se na Equação 4.

$$y[m, n] = \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} h[i, j] x[m - i, n - j] \quad (4)$$

Onde:

- $y[m, n]$ é o valor da saída da operação de convolução na posição (m, n) ;
- $x[m, n]$ é a entrada (por exemplo, uma imagem);
- $h[i, j]$ é o filtro da convolução, que tem tamanho $M \times N$;
- M e N são as dimensões do filtro;
- m e n são os índices da posição atual da saída;

- i e j são os índices das posições do filtro.

Nesta equação, os produtos dos elementos do filtro h e os elementos correspondentes da entrada x são somados, enquanto o filtro desliza sobre a entrada. Contudo, dois parâmetros adicionais também são considerados, são eles o passo (*stride*) e o preenchimento (*padding*).

O passo indica quantos pixels os filtros serão deslocados sobre a imagem, ou seja, com um passo igual à 1, temos que o filtro deslizará sobre a imagem 1 pixel por vez, ao mesmo tempo que se o passo for, por exemplo, 5, o filtro se deslocará 5 pixels por vez. Além disso, vale notar que a Equação 4 possui um passo de 1. Caso fosse inserido o passo, seria necessário multiplicar os valores de m e n pelo valor do passo.

Já o parâmetro de preenchimento é utilizado quando são adicionadas bordas na imagem, dessa forma, o valor de preenchimento representa quantos pixels são adicionais nas dimensões da imagem. Geralmente, os pixels adicionados no preenchimento possuem valores iguais à zero, porém, outros valores também podem ser utilizados em outros casos. O preenchimento é utilizado principalmente para controlar o tamanho de saída de uma camada convolucional, já que o tamanho da saída dessa camada depende tanto do tamanho da entrada e do filtro, como dos valores de passo e de preenchimento e é dada pela Equação 5. Além disso, enquanto a equação de convolução define como cada valor da saída é calculado, o tamanho da entrada, passo e preenchimento afetam, também, quantas operações de convolução serão realizadas.

$$H_{\text{out}} = \frac{H - M + 2P}{S} + 1, \quad W_{\text{out}} = \frac{W - N + 2P}{S} + 1, \quad D_{\text{out}} = K \quad (5)$$

Onde:

- H e W são as dimensões da entrada (altura e largura).
- D é a profundidade da entrada (número de canais).
- H_{out} e W_{out} são, respectivamente, a altura e largura da saída.
- D_{out} é a profundidade da saída, que corresponde ao número de filtros aplicados, K .
- M e N são as dimensões do filtro (ou núcleo).
- P é o preenchimento aplicado em cada lado da entrada.
- S é o passo, que indica o passo de deslocamento do filtro.
- K é o número de filtros aplicados na camada convolucional.

Como vários filtros são normalmente utilizados em uma camada de convolução, ela frequentemente gera saídas com dimensões grandes. Para lidar com isso, as redes convolucionais geralmente incluem uma camada de Amostragem (*Polling*) logo após a camada convolucional. O objetivo dessa camada é reduzir a dimensionalidade dos mapas de características gerados, diminuindo a quantidade de parâmetros e, conseqüentemente, o custo computacional da rede.

A camada de Amostragem opera em pequenas regiões dos mapas de características, resumindo as informações de cada região em um único valor. Os dois tipos mais comuns de Amostragem são a **Amostragem Máxima** (*Max Pooling*) e a **Amostragem Média** (*Average Pooling*):

- **Amostragem Máxima:** Seleciona o valor máximo dentro de uma pequena região do mapa de características. Essa abordagem destaca as características mais relevantes e significativas, preservando os valores mais altos, que indicam maior intensidade de detecção de padrões.
- **Amostragem Média:** Calcula a média dos valores dentro de uma região do mapa de características. Esse método suaviza as características e pode ser útil em cenários onde há presença de ruído, fornecendo uma representação mais equilibrada.

Assim, a camada de Amostragem desempenha um papel importante na simplificação dos dados processados pela rede, ao mesmo tempo em que preserva informações importantes para a identificação de padrões.

Além das camadas convolucionais e de Amostragem, a rede também possui funções de ativação, como mencionado anteriormente e em muitos casos, ela também inclui uma camada final totalmente conectada para realizar a classificação final. Essa camada é semelhante à descrita na Seção 2.1.1.

2.2 Métricas de Desempenho

Avaliar o desempenho de redes neurais é crucial para entender sua eficácia e viabilidade em aplicações práticas. As métricas comumente utilizadas, como acurácia, latência, vazão e dissipação de potência, fornecem diferentes perspectivas sobre como a rede neural opera e se adapta às exigências específicas do sistema. Essas métricas serão apresentadas com maior detalhe nas subseções seguintes.

2.2.1 Acurácia

A acurácia mede a proporção de previsões corretas feitas por um modelo em relação ao total de amostras avaliadas. Ela é amplamente usada para avaliar o desempenho em problemas de classificação, sendo definida como apresentada na Equação 6.

$$\text{Acurácia} = \frac{\text{Número de previsões corretas}}{\text{Número total de previsões}} \quad (6)$$

A acurácia é uma métrica essencial para determinar se o modelo consegue resolver o problema para o qual foi treinado. Ainda mais, ela garante que as otimizações em hardware não comprometam a qualidade das previsões. Ela serve como ponto de referência para comparar diferentes implementações, auxilia na tomada de decisões de design, como a escolha da arquitetura e das técnicas de aceleração, e permite avaliar o impacto de técnicas como a quantização.

2.2.2 Latência

A latência refere-se ao tempo necessário para que o modelo processe uma única entrada e produza uma saída. Ela pode ser medida como o tempo entre o recebimento de uma solicitação e a entrega do resultado. Ela é fundamental para aplicações em tempo real, como sistemas de visão computacional em veículos autônomos, e para garantir uma experiência fluida em aplicações interativas, sendo crucial para garantir respostas rápidas e seguras (JOUPPI *et al.*, 2017).

2.2.3 Vazão

Vazão é a taxa de transferência de dados. No contexto de redes neurais, a vazão representa o número de inferências (ou predições) que um sistema pode realizar por unidade de tempo. Esse conceito é importante para avaliar o desempenho de sistemas de aprendizado de máquina, especialmente em aplicações de inferência em tempo real, onde é necessário processar grandes quantidades de dados rapidamente (SZE *et al.*, 2017).

No caso de imagens, é possível representar essa métrica pelo número de imagens de segundo ou *frames* por segundo (FPS) classificadas.

A vazão é particularmente relevante em cenários de alto volume de dados, como processamento de imagens em larga escala ou inferência em fluxos contínuos de dados, como em aplicações em Internet das Coisas (IoT) e em tempo real.

2.2.4 Dissipação de potência

A dissipação de potência refere-se à quantidade de energia por unidade de tempo gasta por um sistema. Em dispositivos de borda, como sensores Internet das Coisas

(IoT), drones e smartphones, a eficiência energética é um fator crítico, pois esses sistemas geralmente operam com recursos limitados, como baterias ou fontes de energia com capacidade restrita (BORDIGNON, 2023).

Essa característica é especialmente relevante em cenários como redes de sensores remotos, onde a substituição ou recarga de baterias pode ser inviável. Além disso, a eficiência energética reduz a geração de calor, o que possibilita maior densidade de hardware sem necessidade de sistemas complexos de dissipação térmica (RIOS, 2022).

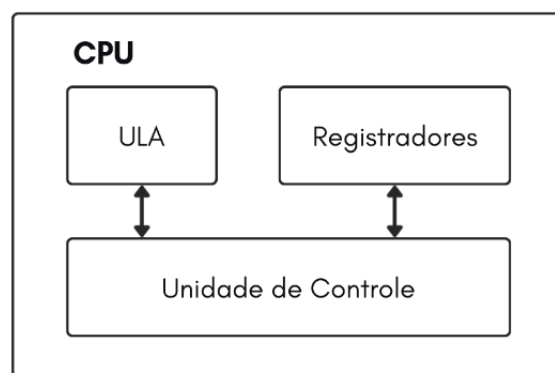
Por fim, a dissipação de potência está intimamente ligado ao desempenho geral do sistema. Reduzir a potência, sem comprometer a precisão ou a latência, é um desafio essencial em aplicações de inteligência artificial em borda, motivando o desenvolvimento de técnicas para a implementação de algoritmos em hardware especializado.

2.3 Arquiteturas de Hardware

2.3.1 Unidade Central de Processamento

A Unidade Central de Processamento (*Central Processing Unit* - CPU) é o componente principal de um computador, responsável por interpretar e executar instruções. Sua estrutura é composta por três partes principais: a Unidade de Controle (UC), que coordena a execução das instruções, buscando-as na memória, decodificando e gerenciando sua execução; a Unidade Lógica e Aritmética (ULA), responsável por realizar cálculos matemáticos e operações lógicas; e os registradores, pequenas áreas de memória que armazenam temporariamente dados necessários para o processamento. A relação simplificada entre essas três partes pode ser observada na Figura 5.

Figura 5 – Unidades básicas de uma CPU.



Fonte: o Autor, 2024.

O funcionamento da CPU segue o chamado Ciclo de Máquina, que consiste em buscar, decodificar e executar instruções de maneira sequencial (STALLINGS, 2016, p.

28). Com a evolução das arquiteturas computacionais, as CPUs modernas passaram a incorporar recursos como execução paralela, pipelines e hierarquia de cache para aumentar o desempenho e reduzir a latência. Na execução de redes neurais, a arquitetura da CPU desempenha um papel essencial, especialmente em aplicações de inferência e, em menor escala, no treinamento. Isso ocorre devido à flexibilidade e à capacidade de processar tarefas diversas com eficiência.

A CPU se beneficia também na manipulação de pesos e cálculos intermediários devido à sua hierarquia de cache, a qual reduz a latência ao armazenar temporariamente dados frequentemente acessados, como os pesos de uma camada. A hierarquia existe devido à diferença significativa de velocidade entre a CPU e a memória principal (RAM). Enquanto a CPU pode operar em ciclos extremamente rápidos, acessar dados diretamente na RAM pode ser milhares de vezes mais lento. Com base nesses princípios, a CPU armazena os dados mais usados e acessados recentemente no cache L1, dados menos frequentes no L2, e assim por diante. Nessas aplicações, os pesos e dados intermediários podem ser mantidos nos níveis superiores da cache, reduzindo a latência de processamento.

Contudo, as CPUs possuem limitações significativas que podem comprometer seu desempenho em cenários mais complexos. Sua capacidade de paralelismo é limitada, tornando-as menos eficientes em operações massivamente paralelas, como as comuns em redes neurais profundas. Além disso, seu vazão é inferior ao de GPUs por exemplo, especialmente ao processar grandes lotes de dados (*batch processing*), dificultando sua escalabilidade em aplicações de maior demanda. Outro ponto negativo é o consumo de energia, que tende a ser elevado para tarefas intensivas, resultando em menor eficiência energética.

Embora outras arquiteturas, como GPUs, sejam frequentemente preferidas para treinamento de redes neurais devido às suas altas capacidades de paralelismo, as CPUs continuam sendo amplamente usadas para inferência em tempo real. Segundo Liu *et al.* (2019), otimizações específicas permitem que CPUs alcancem desempenho competitivo na inferência de modelos de redes neurais convolucionais, especialmente em cenários que exigem baixa latência. Assim, para aplicações de baixa escala, sistemas embarcados e tarefas que exigem baixa latência mas não são intensivas em cálculos, as CPUs podem se sobressair em relação a outras arquiteturas.

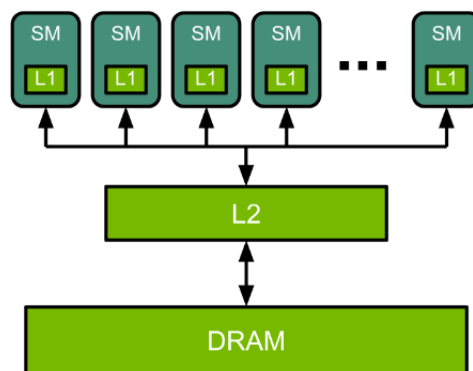
2.3.2 Unidade de Processamento Gráfica

A Unidade de Processamento de Gráficos (*Graphics Processing Unit* - GPU) é um componente essencial na computação moderna, especialmente projetado para realizar cálculos paralelos de forma eficiente. Sua arquitetura é otimizada para operações massivamente paralelas, com milhares de núcleos capazes de executar tarefas simultâneas, diferentemente das CPUs, que são projetadas para desempenho sequencial. Segundo NVIDIA Corporation

(2023), as GPUs maximizam a vazão ao dividir grandes problemas computacionais em milhares de encadeamentos (*threads*), que podem ser executadas paralelamente. Isso torna a GPU especialmente útil em aplicações como processamento gráfico, aprendizado de máquina e simulações científicas.

A arquitetura das GPUs combina núcleos de processamento, memória compartilhada e controladores de memória que trabalham em conjunto para maximizar o desempenho. Os núcleos realizam cálculos matemáticos enquanto a hierarquia de memória, composta por memória global, caches e memória compartilhada, reduz a latência ao armazenar dados frequentemente acessados (STALLINGS, 2016, p. 698). Uma versão simplificada da arquitetura básica de uma GPU pode ser observada na Figura 6.

Figura 6 – Arquitetura básica de uma GPU.



Fonte: (NVIDIA CORPORATION, 2023).

Assim como a CPU, a GPU também utiliza memórias cache para otimizar o desempenho. No entanto, ao contrário da CPU, que possui uma Unidade Lógica e Aritmética (ULA) principal, a GPU conta com múltiplos multiprocessadores de *streaming*, cada um equipado com diversos núcleos de processamento. Esses núcleos, porém, realizam operações mais simples em comparação aos núcleos da CPU, sendo projetados para processamento em massa Goodfellow, Bengio e Courville (2016, p. 447).

A maior parte das operações realizadas por uma GPU são do tipo multiplicar e acumular (*Multiply-Accumulate*, ou MAC), essenciais para cálculos envolvendo multiplicações de matrizes, entre outros. A arquitetura das GPUs foi inicialmente desenvolvida para aplicações gráficas, como renderização de objetos tridimensionais em sistemas de jogos. No entanto, sua alta capacidade de paralelismo e grande largura de banda de memória as tornam ideais para aplicações como aprendizado de máquina, especialmente no treinamento de redes neurais profundas.

Em aprendizado de máquina, as GPUs desempenham um papel central, particularmente no treinamento de redes neurais profundas. A capacidade de realizar milhões de

operações matriciais simultâneas acelera o ajuste de pesos e o cálculo de gradientes em grandes volumes de dados.

No entanto, as GPUs apresentam limitações em aplicações que exigem baixa latência ou processamento sequencial, áreas onde as CPUs ainda mantêm vantagem. Além disso, o processo de transferência de dados entre a GPU e a memória principal pode introduzir atrasos significativos, reduzindo a eficiência em determinadas aplicações.

Apesar dessas limitações, as GPUs continuam a evoluir, tornando-se mais integradas em arquiteturas híbridas em que se combinam com CPUs e outras arquiteturas especializadas. Seu impacto no avanço da inteligência artificial e na computação de alto desempenho é inegável, consolidando-as como ferramentas essenciais nessa área.

2.3.3 Circuitos Integrados para Aplicação Específica

Os Circuitos Integrados para Aplicação Específica (*Application-Specific Integrated Circuits* - ASICs) são circuitos integrados projetados especificamente para executar tarefas muito particulares com extrema eficiência. Diferentemente de CPUs e GPUs, que são de uso geral, os ASICs são criados sob medida para uma aplicação específica. Sua especialização permite um desempenho superior em termos de velocidade, consumo energético e custo por operação (WANG; LUO, 2022).

A arquitetura dos ASICs é completamente dedicada ao objetivo para o qual foram projetados. Em redes neurais, ASICs são projetadas para operações matriciais intensivas, como multiplicação e soma acumulativa específicas e especializadas para determinado modelo de rede neural. Essa arquitetura oferece vantagens significativas em aplicações onde altos volumes de dados precisam ser processados com baixa latência e alta eficiência energética. Sua estrutura especializada elimina redundâncias típicas de arquiteturas de uso geral, reduzindo a necessidade de operações de controle e memória não essenciais. Isso é particularmente valioso em sistemas embarcados e dispositivos de IoT, onde a eficiência energética é crucial.

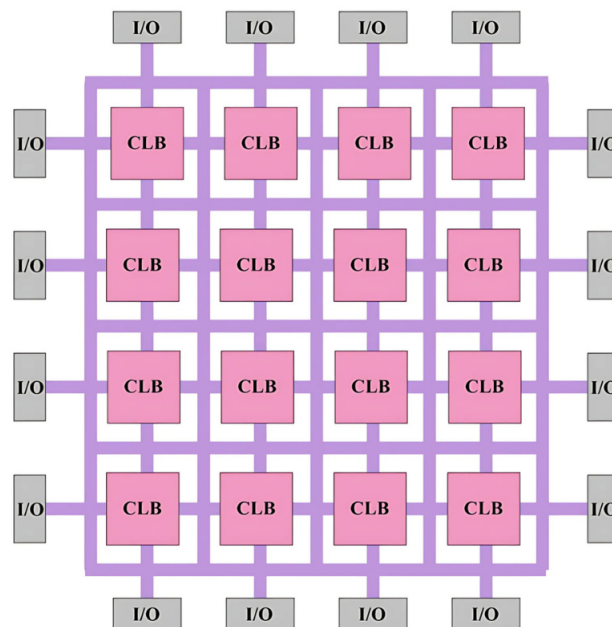
Apesar de maior desempenho, os ASICs possuem limitações significativas. Seu desenvolvimento é caro e demorado, o que os torna inviáveis para aplicações de curto prazo ou que exigem flexibilidade. Além disso, a falta de adaptabilidade significa que, se os requisitos da aplicação mudarem, o ASIC se torna obsoleto (WANG; LUO, 2022).

Com a crescente demanda por soluções computacionais especializadas, os ASICs têm se tornado cada vez mais comuns em áreas como inteligência artificial, comunicações 5G e análise em tempo real. Sua capacidade de realizar cálculos com precisão e eficiência os coloca na vanguarda da computação de alto desempenho, sendo essenciais para atender às demandas das tecnologias emergentes.

2.3.4 Matriz de Portas Programável em Campo

As Matrizes de Portas Programável em Campo, mais comumente conhecida como FPGAs (do inglês Field-Programmable Gate Array) são dispositivos semicondutores programáveis capazes de realizar diversas funções lógicas digitais. Ao contrário de CPUs e GPUs, que possuem arquiteturas fixas, os FPGAs oferecem a flexibilidade de reconfiguração diretamente no nível de hardware. Essa característica permite que os usuários projetem circuitos personalizados para atender a tarefas específicas de maneira eficiente (FAROOQ; MARRAKCHI; MEHREZ, 2012, p. 2). Sua adaptabilidade possibilita implementações como processadores dedicados, *pipelines* paralelos ou arranjos especializados, otimizando o desempenho e o consumo energético em aplicações como inteligência artificial, processamento de sinais e controle de sistemas embarcados.

Figura 7 – Visão geral da arquitetura de um FPGA.



Fonte: (FAROOQ; MARRAKCHI; MEHREZ, 2012, p. 9).

A Figura 7 apresenta a visão geral da arquitetura de um FPGA genérico, composto por três elementos principais: (i) Blocos Lógicos Configuráveis (*Configurable Logic Blocks* - CLBs), consistem em células lógicas programáveis que podem executar funções lógicas básicas, como AND, OR e XOR, além de operações aritméticas mais complexas, como somadores ou multiplicadores; (ii) uma matriz de interconexão, que possibilita a comunicação entre os blocos; (iii) e entradas e saídas configuráveis (*Input/Output* - I/O), responsáveis por conectar o FPGA a outros dispositivos ou sensores como apresentado na Figura 7. Além desses principais blocos, a maioria dos FPGA modernos também inclui blocos de memória (Block RAM) e blocos de Processamento de Sinais Digitais (*Digital Signal Processing* - DSP).

Os CLBs normalmente contêm os seguintes componentes:

- **Tabelas Verdade (*Look Up Tables* - LUTs):** Estruturas que implementam funções lógicas por meio de tabelas de verdade.
- **Flip-Flops:** Elementos de armazenamento que permitem a sincronização e a criação de circuitos sequenciais.
- **Multiplexadores:** Componentes usados para rotear sinais dentro do bloco.

O funcionamento do FPGA envolve a programação de hardware por meio de linguagens de Nível de Transferência entre Registradores (*Register-Transfer Level* - RTL), como VHDL ou Verilog, permitindo que desenvolvedores descrevam a lógica do circuito desejado. O processo consiste em projetar o circuito, sintetizar e mapear a lógica para os recursos internos do FPGA, e carregar o projeto no dispositivo, tornando-o funcional para executar a lógica definida diretamente em hardware.

Apesar das linguagens de RTL como VHDL e Verilog serem linguagens bastante consolidadas no desenvolvimento de FPGA, o seu desenvolvimento tem evoluído para incluir linguagens de alto nível, como as empregadas na Síntese de Alto nível. O HLS é uma abordagem que permite aos desenvolvedores projetar circuitos digitais utilizando linguagens mais acessíveis, como C, C++ ou Python, reduzindo a barreira técnica e acelerando o desenvolvimento (AMD, 2025).

Com HLS, o processo de programação é abstraído para um nível superior, permitindo que o desenvolvedor descreva comportamentos algorítmicos em vez de detalhar a lógica do hardware em si. Ferramentas de HLS, como o Vivado HLS da Xilinx, convertem automaticamente essas descrições em hardware configurável para o FPGA, gerando código em VHDL ou Verilog que pode ser implementado diretamente.

Considerando sua arquitetura altamente paralela e flexível, os FPGAs são uma ótima opção para acelerar redes neurais (NECHI *et al.*, 2023). A capacidade de configurar os blocos lógicos e interconexões do FPGA para executar cálculos matemáticos complexos, como as operações matriciais presentes nas RNAs, torna-os uma alternativa poderosa para acelerar essas redes, especialmente em aplicações de inteligência artificial em tempo real. As redes neurais artificiais exigem um grande volume de operações matemáticas, como multiplicações e somas em alta velocidade, geralmente envolvendo vetores e matrizes. Em arquiteturas tradicionais, como CPUs e GPUs, essas operações são realizadas de forma sequencial ou parcialmente paralela, dependendo do hardware. Por outro lado, os FPGAs, devido à sua natureza reconfigurável, permitem a implementação em hardware dedicado, otimizando tanto a performance quanto o consumo de energia (HAMANAKA *et al.*, 2023).

As arquiteturas de FPGAs podem ser ajustadas para atender às necessidades específicas das redes neurais:

- **Pipeline de Dados:** Os FPGAs podem ser configurados para criar *pipelines* otimizados, onde cada estágio do *pipeline* processa uma parte da rede neural. Isso reduz significativamente o tempo de processamento por amostra (WANG; LUO, 2022).
- **Quantização e Compactação:** FPGAs permitem a implementação de redes neurais com precisão reduzida (por exemplo, usando 8 ou até 4 bits em vez de 32 bits de ponto flutuante). Essa abordagem, reduz o consumo de recursos e aumenta a eficiência e geralmente sem comprometer drasticamente a precisão da rede (NECHI *et al.*, 2023).
- **Memória On-Chip:** A arquitetura de FPGA oferece memória distribuída próxima aos blocos lógicos, reduzindo a necessidade de acessar memórias externas, o que melhora o desempenho e economiza energia em aplicações intensivas de RNAs (GUO *et al.*, 2018).

Dessa forma, as principais vantagens na utilização de FPGA são sua baixa latência devido às otimizações de sua arquitetura, eficiência energética por se tratar de um dispositivo que consome menos energia comparado à outros dispositivos como CPU e GPU e sua reconfigurabilidade, a qual permite a otimizações para redes específicas e a reutilização de seu hardware.

No entanto, apesar das vantagens, a utilização de FPGAs para RNAs ainda apresenta desafios, como a maior complexidade no desenvolvimento em comparação com GPUs e CPUs, além do tempo necessário para programar hardware em linguagens como VHDL ou Verilog. No entanto, linguagens de alto nível e frameworks como o FINN (o qual será apresentado nas próximas seções) se apresentam com o objetivo de facilitar esse processo, tornando os FPGAs mais acessíveis para aplicações em aprendizado de máquina.

2.4 Quantização

A quantização, refere-se à discretização de valores contínuos em um conjunto finito de níveis, permitindo que informações analógicas, como som e imagem, sejam representadas em formato digital. Esse conceito é amplamente utilizado em áreas como processamento de sinais, compressão de dados, redes neurais e outras aplicações que requerem a digitalização ou otimização de dados.

Durante a quantização, um sinal contínuo é amostrado em intervalos regulares (amostragem) e os valores amostrados são arredondados ou aproximados para os níveis mais próximos em uma grade finita pré-determinada. O resultado é um conjunto de valores discretos que podem ser armazenados ou transmitidos eficientemente.

A precisão da quantização depende diretamente do número de níveis disponíveis. Quanto mais níveis forem usados, maior será a fidelidade ao sinal original, mas também maior será a quantidade de dados gerados. Em contrapartida, uma redução no número de níveis leva a uma compactação mais eficiente, mas com perda de informação.

Na área de redes neurais, a quantização é uma técnica que reduz a precisão dos pesos e ativações, geralmente representados por números em ponto flutuante de 32 ou 64 bits (FP32 e FP64), para formatos de menor precisão, como inteiros de 8 bits (INT8) ou até mesmo de 1 bit (INT1). O ponto flutuante é um formato de representação numérica que permite expressar números reais com ampla variação de magnitude, sendo eficiente para representar tanto valores extremamente grandes quanto extremamente pequenos. Entretanto, a quantização diminui os requisitos de memória e acelera cálculos, especialmente em hardware especializados. A quantização em redes neurais permite uma otimização significativa, mantendo níveis competitivos de precisão para inferência (NAVEEN; KOUNTE, 2024).

A quantização em redes neurais podem ocorrer de formas diferentes. As principais são a Quantização Pós-Treinamento (*Post-Training Quantization*, PTQ) e a Quantização Consciente de Treinamento (*Quantization-Aware Training*, QAT).

2.4.1 Quantização Pós-Treinamento

A Quantização Pós-Treinamento (*Post-Training Quantization* - PTQ) é uma técnica que reduz a precisão numérica de modelos de aprendizado de máquina treinados, convertendo pesos e ativações de alta precisão, como FP32, para formatos de menor precisão, como INT8. Esse processo é realizado após o treinamento do modelo, sem a necessidade de re-treinamento, tornando-o mais eficiente em termos de memória e computação.

O processo envolve a calibração de um pequeno conjunto de dados para determinar os intervalos de pesos e ativações, seguido do cálculo de fatores de escala que permitem mapear os valores de ponto flutuante para inteiros. Após a quantização, o modelo opera diretamente no formato quantizado durante a inferência. Isso resulta em uma redução significativa do tamanho do modelo e no aumento da velocidade de execução, já que a aritmética com números inteiros é menos custosa. Segundo Wei *et al.* (2023), A PTQ exige muito menos computação para obter um modelo quantizado, pois não requer treinamento de ponta a ponta, por isso, tem recebido muita atenção recentemente devido ao seu baixo custo e características de fácil utilização na prática.

A PTQ se destaca por sua simplicidade de implementação, no entanto, ela apresenta limitações, como possíveis perdas de precisão, especialmente em tarefas sensíveis como visão computacional, e dependência de dados de calibração de alta qualidade. Apesar dessas

limitações, a PTQ é utilizada adotada devido à sua capacidade de equilibrar eficiência e desempenho.

Assim, a PTQ apresenta limitações, especialmente ao buscar quantizações de baixa precisão nas ativações, como 4 bits ou menos. Conforme apontado por Nagel *et al.* (2021), técnicas de pós-treinamento podem não ser suficientes para mitigar os grandes erros de quantização introduzidos por representações de baixa precisão. Nesses casos, torna-se necessário recorrer à Quantização Consciente de Treinamento (*Quantization-Aware Training* - QAT).

2.4.2 Quantização Consciente de Treinamento

A Quantização Consciente de Treinamento (*Quantization-Aware Training* - QAT) é uma técnica que incorpora a quantização durante o processo de treinamento de redes neurais. Diferentemente da quantização de pós-treinamento (PTQ), onde a quantização é aplicada após o treinamento, a QAT simula os efeitos da menor precisão (como INT8) durante o próprio treinamento. Isso permite que o modelo ajuste seus pesos e ativações para minimizar a perda de precisão causada pela quantização, resultando em modelos mais robustos e precisos.

Durante a QAT, as operações de quantização são simuladas, mas o treinamento continua utilizando pesos em alta precisão, como FP32. Essa abordagem ajuda a modelar os erros introduzidos pela quantização e a ajustar os parâmetros da rede de forma a compensá-los. Como resultado, modelos treinados com QAT alcançam precisão superior em comparação com aqueles quantizados apenas após o treinamento, especialmente em tarefas sensíveis, como visão computacional e processamento de linguagem natural. A QAT permite manter a precisão do modelo em aplicações críticas, mesmo com formatos de menor precisão Sakr *et al.* (2022).

A principal vantagem da QAT é sua alta precisão, tornando-a ideal para aplicações que exigem desempenho superior. No entanto, ela tem algumas limitações: exige mais recursos computacionais e tempo, pois o treinamento completo do modelo deve ser refeito considerando os efeitos da quantização. Além disso, requer acesso ao conjunto de dados de treinamento original, o que nem sempre é viável, especialmente em cenários onde os dados são sensíveis ou restritos.

3 Ferramentas e *Frameworks*

Este capítulo apresenta uma visão geral sobre as principais ferramentas e *frameworks* utilizados no desenvolvimento e implementação desse trabalho. Cada uma dessas tecnologias desempenha um papel fundamental em diferentes etapas do trabalho, desde a aquisição dos dados, treinamento e quantização de modelos até a sua exportação, visualização e implementação.

O Kaggle, por exemplo, foi uma fonte importante para a obtenção de conjuntos de dados. O PyTorch oferece uma base flexível para o desenvolvimento e treinamento de redes neurais. Já o ONNX e o ONNX Runtime garantem a interoperabilidade entre diferentes *frameworks* e dispositivos, enquanto o Netron auxilia na validação visual da estrutura dos modelos. Para aplicações de quantização, o Brevitas permite a criação de redes otimizadas para hardware, alinhando-se com os fluxos do FINN, que tem como objetivo facilitar a implementação dessas redes em FPGAs, o qual também faz uso de ferramentas como o Vivado para a síntese e a implementação em hardware. Ao longo deste capítulo, essas ferramentas são apresentadas em maiores detalhes.

3.1 Kaggle

O Kaggle (KAGGLE, 2025) é uma plataforma online que oferece um ambiente colaborativo para cientistas de dados e pesquisadores. Ele é amplamente reconhecido por fornecer acesso a uma grande variedade de conjuntos de dados, competições de aprendizado de máquina, tutoriais, e ferramentas de análise de dados. No contexto deste trabalho, o Kaggle foi uma fonte fundamental para a obtenção de conjuntos de dados.

Uma das principais vantagens do Kaggle é a disponibilidade de conjuntos de dados bem organizados e diversificados, abrangendo desde problemas simples, como classificação de imagens, até desafios complexos, como previsão de séries temporais e processamento de linguagem natural.

Outra funcionalidade que vale destacar é a existência de um repositório de códigos e tutoriais compartilhados por outros usuários. Esses recursos servem como ponto de referência para soluções pré-existentes, permitindo a utilização de códigos otimizados ou a adaptação de abordagens para novos contextos.

3.2 PyTorch

O PyTorch é uma biblioteca de código aberto amplamente utilizada para aprendizado de máquina e inteligência artificial, desenvolvida pelo Facebook AI Research (FAIR). O PyTorch é uma biblioteca de aprendizado profundo que fornece grande flexibilidade e velocidade ao construir e treinar redes neurais. Escrito em Python e C++, ele se destaca por combinar facilidade de uso com alta performance, sendo uma ferramenta essencial para criação, treinamento e implementação de modelos de aprendizado de máquina.

Sua principal função é permitir a construção de modelos personalizados, o treinamento de redes neurais e sua utilização para realizar inferências. Além disso, se destaca por sua integração com Python, que facilita o uso de outras bibliotecas do ecossistema e reduz a barreira de entrada para novos usuários.

Nesse trabalho, o PyTorch foi empregado na implementação, treinamento e conversões dos modelos de redes neurais utilizados.

3.3 Open Neural Network Exchange - ONNX

O Open Neural Network Exchange (ONNX) ou Intercâmbio Aberto de Redes Neurais é um formato aberto que facilita a interoperabilidade entre diferentes *frameworks* de aprendizado de máquina. Inicialmente desenvolvido por empresas como Facebook e Microsoft, o ONNX define um padrão unificado para a representação e exportação de modelos de aprendizado profundo, promovendo a colaboração e a reutilização entre ferramentas e ecossistemas distintos (JAJAL *et al.*, 2024). Ele define um conjunto comum de operadores para os blocos de construção de modelos de aprendizado de máquina e aprendizado profundo, além de um formato de arquivo padrão, permitindo que desenvolvedores utilizem esses modelos com uma combinação de *frameworks*, ferramentas e compiladores (NAGATA *et al.*, 2024).

O principal propósito do ONNX é possibilitar a transferência de modelos treinados em um *framework*, como PyTorch ou TensorFlow, para outros ambientes que utilizam ferramentas especializadas em inferência ou produção, como ONNX Runtime ou *frameworks* embarcados. Essa interoperabilidade é crucial para otimizar o desempenho de modelos em diferentes plataformas de hardware, incluindo CPUs, GPUs e dispositivos especializados, como FPGAs.

Além disso, o ONNX promove a adoção de padrões abertos, reduzindo a dependência de *frameworks* proprietários e permitindo que desenvolvedores e organizações utilizem a melhor ferramenta para cada etapa do desenvolvimento. Dessa forma, o ONNX é uma solução robusta e estratégica para garantir a flexibilidade e a eficiência em projetos de aprendizado de máquina modernos.

Apesar do ONNX ser um formato bastante útil, é necessário atentar-se ao realizar as conversões de modelos. Dependendo do modelo, erros podem ocorrer durante esse processo, tornando essencial a verificação da integridade do modelo após as conversões (JAJAL *et al.*, 2024).

No contexto desse trabalho, o formato ONNX foi importante na inferência na CPU e GPU realizada pelo ONNX Runtime. Além disso, é parte fundamental do FINN.

3.4 Netron

Netron (ROEDER, 2025) é um visualizador de modelos de aprendizado de máquina que suporta uma ampla variedade de formatos, incluindo ONNX, PyTorch, entre outros. Ele permite uma análise visual das redes neurais, exibindo a topologia completa do modelo, as dimensões das entradas e saídas, bem como os parâmetros de cada camada.

A ferramenta é particularmente útil para verificar a consistência de modelos durante as conversões entre diferentes formatos e *frameworks*, garantindo que a estrutura e os parâmetros da rede permaneçam intactos. No contexto deste trabalho, o Netron foi utilizado para inspecionar os modelos nos formatos ONNX, identificando eventuais discrepâncias ou incompatibilidades antes de avançar para as etapas subsequentes, como a quantização e o fluxo FINN.

Além disso, o Netron oferece uma interface amigável e interativa, permitindo uma navegação detalhada pela arquitetura da rede. Isso é especialmente relevante em casos de redes complexas, onde a compreensão visual da topologia facilita a identificação de erros e ajustes necessários.

O principal papel do Netron nesse trabalho foi a verificação dos modelos intermediários gerados pelas transformações realizadas no fluxo do FINN.

3.5 Brevitas

O Brevitas (PAPPALARDO, 2024) é uma biblioteca de código aberto para aprendizado de máquina, focada na criação e treinamento de modelos quantizados. Desenvolvido para ser compatível com *frameworks* como PyTorch, ele oferece suporte avançado para redes neurais otimizadas para hardware, especialmente em aplicações de inferência em dispositivos embarcados e aceleradores, como FPGAs.

O principal objetivo do Brevitas é fornecer ferramentas para desenvolver redes neurais compactas e eficientes, reduzindo o consumo de memória e o custo computacional sem comprometer significativamente a precisão. Isso é alcançado por meio da quantização

de pesos, ativação e gradientes, o que torna os modelos mais adequados para execução em hardware especializado.

Além disso, seu diferencial em comparação à outras bibliotecas de quantização é a quantização personalizada da rede onde é possível utilizar diferentes tipos de quantizações para cada parte da rede individualmente. Isso oferece maior flexibilidade na quantização da rede, no entanto, necessita de conhecimentos avançados para a definição dos parâmetros de quantização. O foco do Brevitas é na realização de QAT, no entanto, ele também fornece suporte para PTQ.

Nesse trabalho, o Brevitas foi utilizado na quantização dos modelos para a inferência das redes neurais no FPGA.

3.6 ONNX Runtime

O ONNX Runtime é uma biblioteca de código aberto que tem como objetivo otimizar e acelerar a execução de modelos de aprendizado de máquina definidos no formato ONNX (Open Neural Network Exchange). Desenvolvido pela Microsoft, ele é projetado para oferecer uma inferência de alto desempenho e baixa latência em uma ampla variedade de dispositivos e plataformas, incluindo CPUs e GPUs.

O principal objetivo do ONNX Runtime é fornecer um ambiente de execução padronizado para modelos ONNX, permitindo a interoperabilidade entre *frameworks* de treinamento, como PyTorch e TensorFlow (Outra biblioteca Python focada em algoritmos de aprendizado de máquina), e dispositivos de inferência otimizados. Ele suporta otimizações automáticas no gráfico computacional, o que reduz o tempo de execução e melhora o uso de recursos.

Um dos destaques do ONNX Runtime é sua compatibilidade com diversos *backends*, como CUDA, uma plataforma da NVIDIA que permite aproveitar o processamento paralelo massivo das GPUs para acelerar cálculos, e TensorRT, um kit de ferramentas da NVIDIA especializado na otimização e execução de inferência em redes neurais, reduzindo a latência e aumentando a vazão.

O ONNX Runtime foi utilizado para realizar a inferência na CPU e GPU. Além disso, possibilitou a verificação de modelos intermediários no fluxo FINN.

3.7 Vivado

O Vivado é um ambiente de desenvolvimento integrado (IDE) desenvolvido pela Xilinx, amplamente utilizado para o projeto e implementação de sistemas baseados em FPGAs. Ele oferece uma abordagem otimizada para a criação de circuitos digitais, combi-

nando funcionalidades avançadas para síntese, simulação, implementação e depuração de projetos.

Uma de suas principais características é o suporte à entrada de projetos em diferentes níveis de abstração, incluindo linguagens de descrição de hardware (HDLs) como VHDL e Verilog, além de blocos IP pré-configurados e reutilizáveis. O Vivado também integra o *Vivado IP Integrator*, que facilita a construção de sistemas complexos ao permitir a conexão visual de blocos IP com interfaces intuitivas.

Além disso, a ferramenta inclui capacidades para análise detalhada de desempenho, permitindo otimizações em áreas como uso de recursos, latência, consumo de energia e frequência de operação. O Vivado também suporta a geração de *bitstreams*, arquivos binários que configuram o FPGA de acordo com o projeto desenvolvido.

O ambiente de projeto oferece ainda funcionalidades específicas para depuração, como inserção de sondas lógicas e análise do comportamento em tempo de execução. Para projetos mais avançados, o Vivado possibilita a integração com o Vitis, ampliando o suporte ao desenvolvimento de software para plataformas heterogêneas que combinam processadores e FPGAs.

Nesse trabalho, o Vivado foi utilizado como parte do FINN, apresentado na Seção 3.8, para síntese, extração de relatórios e simulações.

3.8 FINN

O FINN é uma estrutura de código aberto desenvolvida pela Xilinx Research Labs para facilitar a implementação eficiente de RNAs em hardware especializado, como os FPGAs. Este *framework* é particularmente voltado para redes neurais que utilizam quantização binária ou de baixa precisão, como as Redes Neurais Quantizadas (*Quantized Neural Networks* - QNNs) e as Redes Neurais Binárias (*Binary Neural Networks* - BNNs). Ao juntar técnicas de quantização com otimizações para hardware, o FINN possibilita a criação de sistemas de inferência rápidos e energeticamente eficientes, especialmente úteis para aplicações embarcadas com restrições de energia (BLOTT *et al.*, 2018).

Uma das principais vantagens do FINN é sua facilidade de uso, mesmo para desenvolvedores que não possuem experiência aprofundada em design de hardware. O *framework* oferece fluxos intuitivos para criar e implementar redes neurais. Além disso, o FINN permite adaptação e escalabilidade, possibilitando que redes neurais sejam ajustadas conforme as especificações de diferentes dispositivos FPGA.

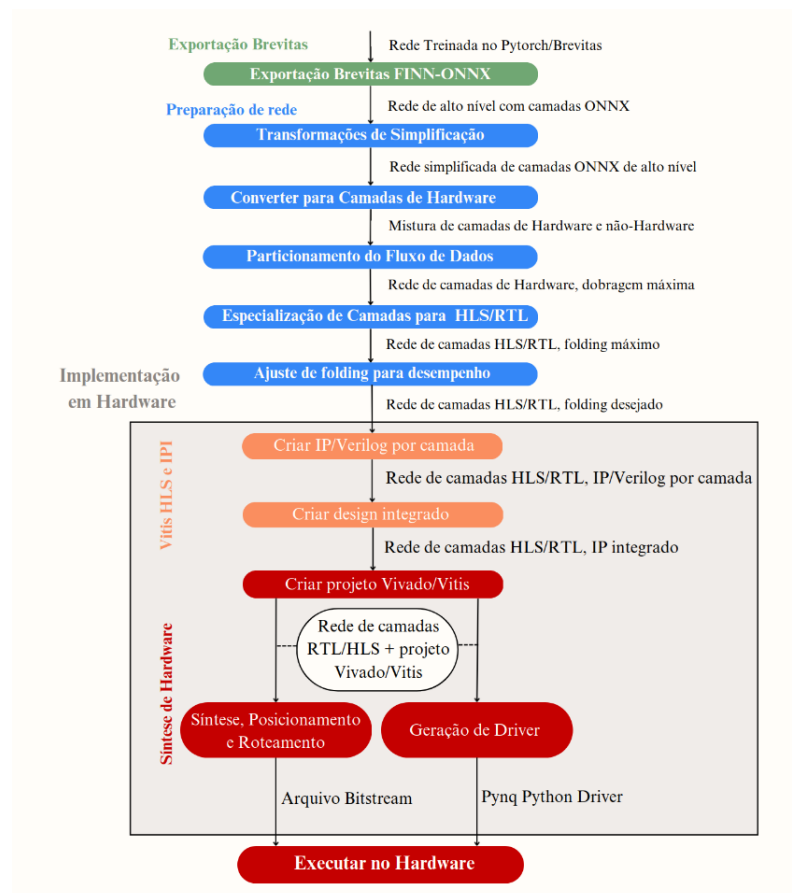
Embora ofereça fluxos simplificados, alcançar resultados otimizados e de alta qualidade exige maior conhecimento técnico, tanto em redes neurais quanto em configurações de hardware FPGA. Desenvolvedores precisam ter familiaridade com ferramentas da Xilinx,

como o Vivado, além de compreender o funcionamento de redes quantizadas, limitações do hardware, assim como dos mecanismos internos do FINN. Essa curva de aprendizado mais acentuada pode dificultar sua adoção, especialmente por equipes com menor experiência (PLAGWITZ *et al.*, 2021).

Outro ponto a considerar é que o FINN é mais adequado para redes quantizadas, como BNNs e QNNs, o que limita sua aplicabilidade a projetos que exigem alta precisão ou utilizam redes muito grandes. Ainda assim, ele é amplamente utilizado em aplicações específicas, como sistemas de visão computacional em tempo real e inferência de redes neurais em dispositivos embarcados (UMUROGLU *et al.*, 2017).

O FINN possui um fluxo bem definido e pode ser observado na Figura 8. O fluxo consiste em três etapas principais. (i) A primeira é o treinamento da rede neural utilizando o Brevitas, (ii) a segunda é a preparação de rede neural e a (iii) terceira é a implementação em hardware. Cada uma dessas etapas será explicada com maior detalhe nas sub-seções seguintes.

Figura 8 – Fluxo completo do FINN.



Fonte: Adaptado de (XILINX, 2025).

3.8.1 Exportação do modelo FINN-ONNX pelo Brevitas

O FINN apenas aceita como entrada modelos no formato ONNX, sendo que redes quantizadas podem ser exportadas no formato QONNX (*Quantized ONNX*) por meio do Brevitas. O QONNX é uma versão adaptada do ONNX, projetada especificamente para redes quantizadas, e é totalmente compatível com o FINN.

No formato QONNX, a quantização é representada por nós específicos. Para que o modelo possa ser processado pelo FINN, ele deve ser convertido de QONNX para o formato FINN-ONNX, que é a representação intermediária (IR, ou do inglês *Intermediate Representation*) utilizada internamente pelo *framework*. Essa transformação adapta a estrutura do modelo para que ele possa ser manipulado de forma eficiente no fluxo do FINN.

Nesse estágio inicial, é possível realizar uma verificação funcional do modelo, simulando seu funcionamento diretamente em Python para garantir que ele opere conforme o esperado.

Após as etapas de carregamento e verificação, o modelo avança para o fluxo completo do FINN, iniciando pela fase de preparação da rede, que marca o próximo passo no processo de otimização e implementação.

3.8.2 Preparação da Rede

A preparação da rede que o FINN realiza consiste em várias transformações que possuem como objetivo realizar simplificações no modelo com a eliminação de redundâncias e transformar o modelo para que o hardware possa entendê-lo diretamente. Entre essas transformações as principais são: **i) Transformações de Simplificação; ii) Conversão para Camadas de Hardware; iii) Especialiação de Camadas; iv) *Folding*.**

As **Transformações de Simplificação** visam eliminar operações de ponto flutuante, movendo-as no gráfico e as colapsando de forma a simplificar a rede, tornando-a mais eficiente. Essas simplificações são baseadas no trabalho de Umuroglu e Jahre (2018). Ao fazer isso, as transformações de otimização melhoram significativamente o desempenho e a utilização de recursos da rede, tornando-a mais rápida, eficiente e apta para implementação em dispositivos como FPGAs. Além disso, em alguns casos, essas transformações podem até melhorar a precisão da rede, reduzindo erros de quantização.

A **Conversão para Camadas de Hardware** é um processo que mapeia e transforma as camadas tradicionais de uma rede neural em representações abstratas, otimizadas para serem mapeadas diretamente para hardware. Em outras palavras, blocos específicos da rede são transformados em blocos que o FINN "entende" como um bloco de hardware e permite que ele seja posteriormente mapeado para uma estrutura pre-definida de hardware. Ao abstrair as operações da rede em camadas de hardware, o FINN permite

aplicar otimizações específicas para o hardware, como a utilização de operações de ponto fixo e o paralelismo, resultando em um desempenho superior e menor consumo de recursos. Além disso, essa representação facilita a portabilidade da rede para diferentes plataformas de hardware.

A etapa de **Especialização de Camadas** no FINN é responsável por transformar as representações abstratas das camadas de hardware em implementações concretas e otimizadas para um determinado hardware. Ao selecionar algoritmos e configurações específicas para cada camada, essa etapa permite adaptar o modelo às características únicas do hardware alvo, como a quantidade de memória, a velocidade de processamento e a arquitetura interna. A especialização resulta na geração de código de hardware, como Verilog ou VHDL, que pode ser compilado e executado diretamente no dispositivo.

É ainda na etapa de especialização que é realizado o **Folding**, o qual define o nível de paralelismo a ser utilizado na rede, ajustando a quantidade de recursos de hardware utilizados e distribuindo as operações em ciclos de clock para maximizar a eficiência. O *folding* é particularmente importante na implementação de redes neurais em FPGAs, pois permite que um hardware limitado execute operações complexas por meio da reutilização de componentes, ou, ainda, aumentar o paralelismo da rede conforme recursos estejam disponíveis para aumentar o desempenho.

O grau de *folding* influencia diretamente o balanceamento entre o desempenho (tempo de execução) e o uso eficiente dos recursos. Assim, as configurações de *folding* são definidas com base nos requisitos do modelo e nas restrições do hardware. O FINN permite, também ajustar o grau de *folding* automaticamente, permitindo que explore diferentes configurações para alcançar o melhor equilíbrio entre desempenho e economia de hardware.

3.8.3 Implementação em Hardware

Após a preparação da rede, as camadas, agora representadas em HLS (High-Level Synthesis) ou RTL (Register Transfer Level), são processadas pelo compilador FINN. Nesse estágio, é possível gerar um Vivado IP Integrator (bloco IP, ou seja, um componente de hardware pré-configurado e reutilizável), que pode ser integrado em qualquer sistema de hardware projetado, facilitando a composição de sistemas mais complexos.

Além disso, o FINN permite a geração de um bitstream file (arquivo binário contendo a configuração necessária para programar o FPGA) e drivers (software que atua como interface para o hardware). Contudo, atualmente, apenas as placas Zynq e Alveo são suportadas para a geração desses drivers.

Os principais passos dessa etapa incluem: i) geração de drivers; ii) inserção de blocos de Acesso Direto à Memória (*Direct Access Memory* - DMA) e Conversor de Largura

de Dados (*Data Width Converter* - DWC); iii) particionamento para *floorplanning*; iv) inserção de memórias FIFO (First-In-First-Out ou "Primeiro a Entrar, Primeiro a Sair") e geração de IPs; v) geração e síntese dos projetos utilizando o Vivado e o Vitis.

Os *drivers* têm como objetivo controlar o hardware gerado e podem ser testados utilizando linguagem Python pela plataforma PYNQ, em que é possível controlar tanto a parte de software (processador) quanto a parte de hardware (FPGA) desses dispositivos. A plataforma PYNQ e os dispositivos Xilinx Zynq são apresentados com maiores detalhes na Seção 5.3.

Nessa etapa também são inseridos blocos de DMA e DWC. O DMA é responsável por transferir dados entre a memória principal do FPGA e o acelerador neural de forma eficiente. Já o DWC garante que os dados sejam transferidos com a largura correta, adaptando-se às diferentes interfaces dos componentes do sistema.

O particionamento para *floorplanning* divide a rede neural em pedaços menores, chamados de partições, para otimizar a sua implementação em um FPGA. Dessa forma, é possível alocar cada parte em uma região específica do FPGA de forma mais eficiente, minimizando os atrasos e o consumo de recursos.

As memórias FIFOs, atuam como *buffers*, armazenando temporariamente os dados entre diferentes componentes da rede neural. Isso garante um fluxo de dados mais suave e sincronizado, evitando gargalos e atrasos. Após a inserção dessas filas, são então gerados os blocos de IP. Por fim, com o IP gerado é possível gerar um arquivo de *bitstream* para a execução na placa escolhida.

4 Datasets

Este capítulo apresenta e descreve os principais conjuntos de dados (*datasets*) utilizados para o desenvolvimento e avaliação das redes neurais exploradas neste trabalho. A escolha desses *datasets* foi baseada em sua relevância acadêmica e prática. Três *datasets* distintos foram selecionados, cada um com características específicas.

Cada *dataset* é apresentado nas próximas seções, abordando sua estrutura, características e principais aplicações.

4.1 Instituto Nacional de Padrões e Tecnologia Modificado

O *dataset* Instituto Nacional de Padrões e Tecnologia Modificado (*Modified National Institute of Standards and Technology* - MNIST) é um conjunto de dados amplamente utilizado para o treinamento e avaliação de algoritmos de aprendizado de máquina, particularmente em tarefas de classificação de imagens. Ele foi introduzido por Lecun *et al.* (1998) como uma versão modificada do conjunto original de dígitos manuscritos do Instituto Nacional de Padrões e Tecnologia (*National Institute of Standards and Technology* - NIST). O objetivo principal do MNIST é oferecer um *benchmark* padronizado para avaliar modelos de reconhecimento de padrões.

4.1.0.1 Estrutura dos dados

O MNIST contém 70.000 imagens de dígitos manuscritos, divididas em:

- 60.000 imagens para treinamento.
- 10.000 imagens para teste.

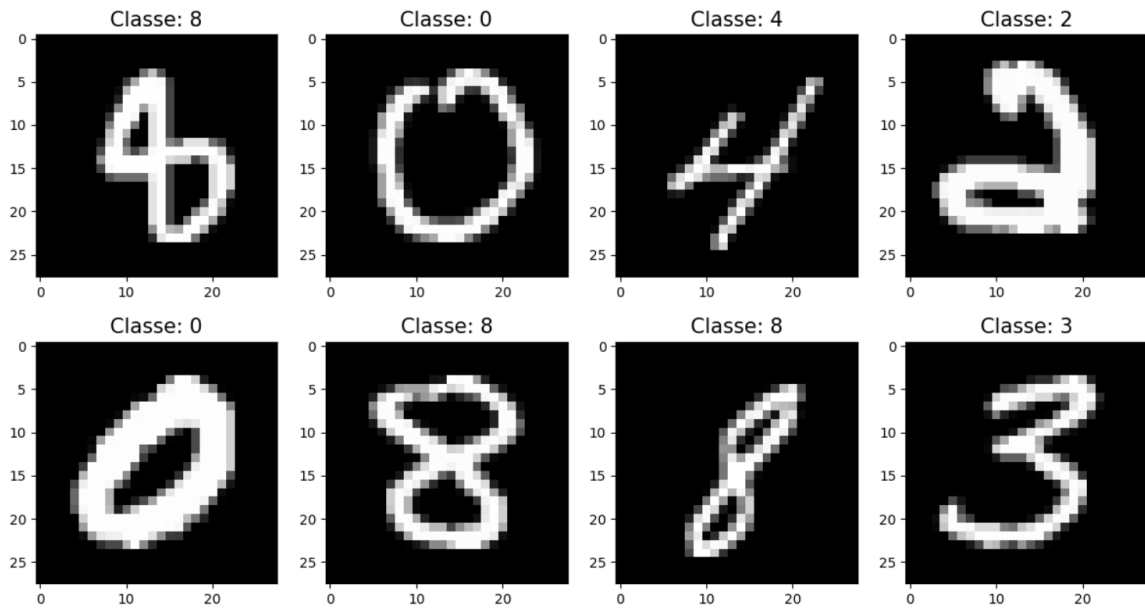
Cada imagem é monocromática, com dimensão de 28x28 pixels, representada em tons de cinza com valores normalizados entre 0 e 255. As imagens são categorizadas em 10 classes, correspondentes aos dígitos de 0 a 9. O *dataset* é balanceado, ou seja, possui uma distribuição aproximadamente uniforme de exemplos entre as classes, garantindo representatividade de todos os dígitos.

4.1.0.2 Características e Aplicações

O MNIST é considerado um padrão de referência para estudos introdutórios em visão computacional e aprendizado de máquina devido à sua simplicidade e acessibilidade. Ele apresenta características importantes, como seu tamanho compacto, que permite o

processamento eficiente mesmo em hardware limitado; a diversidade de estilos de escrita, abrangendo uma ampla variedade de formas de dígitos manuscritos; e a necessidade de pré-processamento mínimo, já que as imagens são centralizadas e normalizadas. Por essas razões, o MNIST é amplamente utilizado para avaliar novas arquiteturas de redes neurais, testar algoritmos de classificação e reconhecimento de padrões, e realizar estudos iniciais em transferência de aprendizado e otimização de hiperparâmetros. Alguns dos dígitos do *dataset* podem ser observados na Figura 9.

Figura 9 – Exemplos de imagens do *dataset* MNIST.



Fonte: o Autor, 2024.

4.2 Instituto Canadense de Pesquisa Avançada - 10 classes

O *dataset* do Instituto Canadense de Pesquisa Avançada - 10 classes (*Canadian Institute for Advanced Research - 10 classes* - CIFAR10) é um conjunto de dados amplamente utilizado em tarefas de aprendizado de máquina e visão computacional, particularmente no treinamento e avaliação de modelos de classificação de imagens. Foi introduzido por Krizhevsky (2009) com o objetivo de fornecer um *benchmark* desafiador para modelos modernos.

4.2.0.1 Estrutura dos dados

O CIFAR-10 contém 60.000 imagens coloridas, organizadas em:

- 50.000 imagens para treinamento.

- 10.000 imagens para teste.

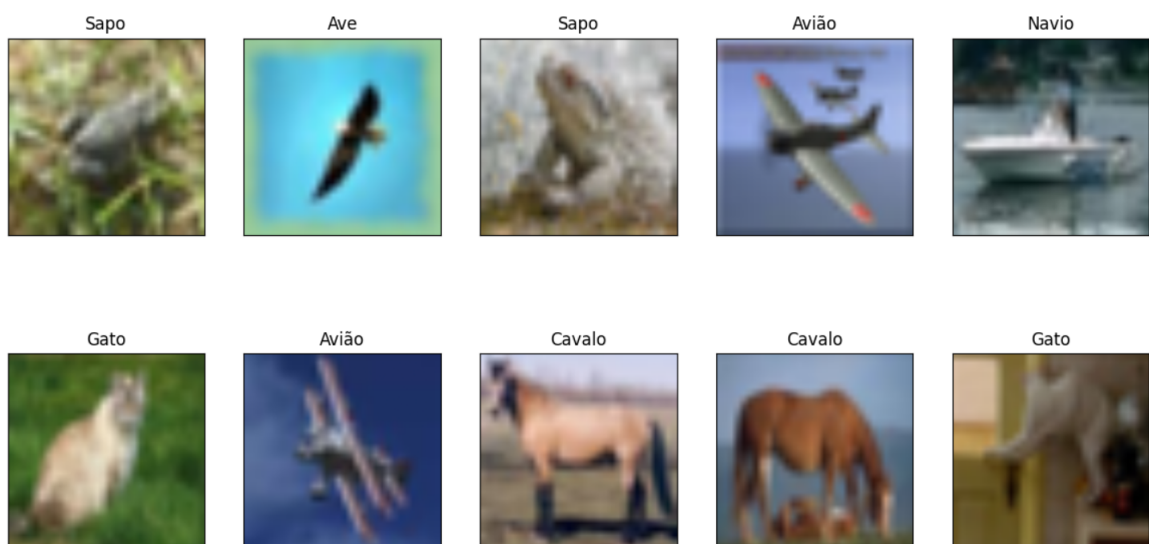
Cada imagem possui dimensão de 32x32 pixels e é representada em cores vermelho, verde, azul (*Red, Green, Blue* - RGB), totalizando 3072 atributos por imagem (1024 pixels para cada canal de cor). O conjunto é dividido em 10 classes distintas, que representam categorias amplas de objetos do mundo real, são elas: avião, automóvel, pássaro, gato, veado, cachorro, sapo, cavalo, navio e caminhão.

O CIFAR-10 é balanceado, o que significa que cada classe contém a mesma quantidade de imagens, garantindo que nenhuma classe seja super ou sub-representada.

4.2.0.2 Características e Aplicações

O CIFAR-10 é amplamente reconhecido como um *benchmark* fundamental para o desenvolvimento e avaliação de modelos de classificação de imagens. Suas principais características incluem a diversidade de classes, representando uma ampla gama de objetos do mundo real, e o tamanho compacto das imagens, que permite experimentação rápida e eficiente mesmo em configurações de hardware limitadas. Além disso, o conjunto é notoriamente desafiador devido à baixa resolução das imagens e à presença de variações significativas nas condições de iluminação, posicionamento e ângulos dos objetos. Algumas das imagens do *dataset* são apresentadas na Figura 10.

Figura 10 – Exemplos de imagens do *dataset* CIFAR-10.



Fonte: o Autor, 2024.

4.3 DeepSat SAT-6

O *DeepSat SAT-6* é um conjunto de dados derivado de imagens aéreas coletadas do Programa Nacional de Imagens Agrícolas (*National Agriculture Imagery Program - NAIP*), projetado para tarefas de aprendizado de máquina em sensoriamento remoto e classificação de cobertura terrestre.

O SAT-6 foi introduzido por Basu et al. (BASU *et al.*, 2015), sendo gerado a partir de amostras de 1500 cenas que abrangem diferentes tipos de paisagem no estado da Califórnia, incluindo áreas rurais, urbanas, florestas densas, terrenos montanhosos, corpos d'água e regiões agrícolas.

4.3.0.1 Estrutura dos dados

O SAT-6 contém um total de 405.000 imagens aéreas, organizadas em:

- 324.000 imagens para treinamento;
- 81.000 imagens para teste.

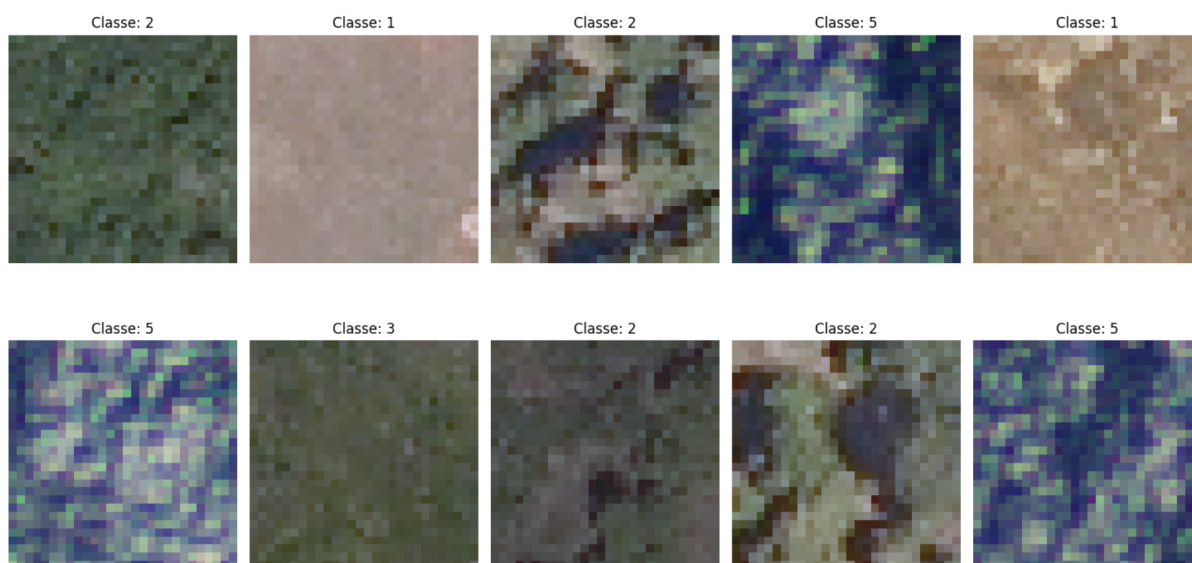
As imagens do SAT-6 foram extraídas do conjunto de dados NAIP, que cobre aproximadamente 330.000 cenas em todo o território continental dos Estados Unidos. O NAIP utiliza imagens GeoTIFF (formato padrão para imagens georreferenciadas que armazenam informações de localização geográfica, como sistema de coordenadas e projeção) com resolução de 1 metro por pixel, garantindo uma precisão horizontal de até seis metros em relação a pontos de controle terrestres identificáveis.

No total, o NAIP compreende cerca de 65 terabytes de dados, com cada imagem possuindo, em média, dimensões de 6000x7000 pixels e um tamanho aproximado de 200 megabytes. Para padronizar o conjunto, blocos de 28x28 pixels não sobrepostos foram extraídos de regiões uniformes de imagens maiores, evitando sobreposição entre classes. Cada imagem no SAT-6 contém 4 bandas espectrais: vermelho (R), verde (G), azul (B) e infravermelho próximo (NIR).

O SAT-6 apresenta seis classes principais de cobertura terrestre, são elas: Solo árido, Árvores, Pastagens, Estradas, Edificações, Corpos d'água.

4.3.0.2 Características e Aplicações

As principais características do SAT-6 incluem a alta resolução espectral e espacial, abrangendo múltiplas bandas (vermelho, verde, azul e infravermelho próximo), e a diversidade de classes que representam diferentes tipos de cobertura terrestre, como áreas urbanas, rurais e corpos d'água. Algumas fotos do *dataset* podem ser observadas na Figura 11.

Figura 11 – Exemplos de imagens do *dataset* SAT-6.

Fonte: o Autor, 2024.

5 Plataformas Utilizadas

As plataformas de hardwares utilizados nesse trabalho foram utilizados um CPU da Intel, uma GPU da NVIDIA e um FPGA da Xilinx. Cada uma dessas plataformas serão detalhados nas próximas sub-seções.

5.1 Intel Core i5

O CPU utilizado foi o **Intel Core i5-1335U**, da 13^a geração de processadores Intel Core, processador de baixo consumo de energia (série U), projetado para notebooks ultrafinos e dispositivos móveis. Detalhes sobre suas características podem ser extraídos da sua nomenclatura como apresentado a seguir:

- **13^a geração:** Conhecida como *Raptor Lake*, processadores desta geração apresentam melhorias na arquitetura, ganhos de desempenho, maior eficiência energética e melhor suporte a novas tecnologias em comparação às gerações anteriores.
- **Core:** Indica a família de processadores da Intel. Dentro desta família, existem subdivisões (*i3*, *i5*, *i7*, *i9*) que diferenciam níveis de desempenho e custo-benefício.
- **i5:** Trata-se de um modelo intermediário. Enquanto a série *i3* é mais básica, a série *i5* atende bem a demandas de produtividade, multitarefa, multimídia e aplicações comuns. Já as séries *i7* e *i9* visam aplicações de alta performance e cargas de trabalho mais exigentes.
- **1335:** O número específico do modelo identifica o processador dentro da gama de produtos da 13^a geração. Estes números indicam variações no número de núcleos, frequências base e turbo, quantidades de cache e outros parâmetros específicos do silício.
- **U:** A letra ao final do nome indica a categoria de consumo energético e a aplicação do processador. Processadores com sufixo "U" são projetados para baixo consumo, sendo geralmente utilizados em ultrabooks, notebooks finos e leves, priorizando eficiência energética e autonomia de bateria. Isto contrasta com outras classificações, como "H" (alto desempenho para laptops), "P" (equilíbrio entre desempenho e mobilidade) ou "K" (desbloqueados para *overclock* em desktops).

O Intel Core i5-1335U emprega uma arquitetura híbrida, combinando *P-cores* (núcleos de desempenho) e *E-cores* (núcleos de eficiência). Esta abordagem, introduzida

pela Intel em gerações mais recentes, permite que tarefas mais exigentes sejam direcionadas aos núcleos de alto desempenho, enquanto tarefas em segundo plano ou menos intensivas são encaminhadas aos núcleos de alta eficiência, otimizando a relação entre desempenho e consumo energético.

No caso do i5-1335U, a configuração típica apresenta 2 núcleos de alto desempenho (*P-cores*) e 8 núcleos de eficiência (*E-cores*), totalizando 10 núcleos. Graças ao *Hyper-Threading* (tecnologia que permite a um único núcleo físico executar duas *threads* simultaneamente) nos P-cores, o processador consegue lidar com até 12 *threads* simultaneamente, o que amplia consideravelmente a capacidade de multitarefa (Intel Corporation, 2025).

Com uma Potência de Projeto Térmico (*Thermal Design Power* - TDP), que representa a quantidade máxima de calor gerada por um componente de hardware, base de cerca de 15 Watts, o i5-1335U é otimizado para baixa dissipação de calor, o que permite projetos de notebooks mais compactos, bem como maior duração da bateria. Em cenários específicos, o TDP pode chegar a até 55 Watts em modo Turbo.

5.2 NVIDIA TITAN X (Pascal)

A GPU utilizada neste projeto foi a **NVIDIA TITAN X (Pascal) GP102 (rev a1) 12 GB RAM**, uma placa gráfica de alto desempenho desenvolvida pela NVIDIA, líder no mercado de GPUs para aplicações que vão desde jogos a computação científica, inteligência artificial e aprendizado de máquina. Essa placa pode ser observada na Figura 12.

Figura 12 – GPU NVIDIA TITAN X (Pascal).



Fonte: (PETER, 2016).

A seguir, é apresentado o detalhamento das principais partes do nome e características da placa:

- **TITAN X:** Representa a linha da GPU, na qual a TITAN X é voltada ao segmento entusiasta e profissional, situando-se acima das linhas tradicionais de consumo (como GeForce GTX no período da série Pascal).
- **Pascal:** Refere-se à arquitetura da GPU, denominada Pascal. A arquitetura Pascal sucedeu a Maxwell e antecedeu Volta, oferecendo avanços significativos em eficiência energética, desempenho em precisão simples e dupla, além de melhorias em recursos específicos para aprendizado profundo, como o uso eficiente de tensores e instruções otimizadas (NVIDIA Corporation, 2025).
- **GP102:** É o nome do chip gráfico (*GPU die*) utilizado na TITAN X (Pascal). O GP102 é uma das variantes de alto desempenho dentro da família Pascal.
- **rev a1:** Esta é a revisão específica do silício GP102 presente na placa. Revisões podem indicar pequenas correções, ajustes ou otimizações na fabricação do chip.
- **12 GB RAM:** A TITAN X (Pascal) possui 12 GB de memória *GDDR5X*, um tipo de memória gráfica de alta largura de banda utilizada para acelerar a transferência de dados entre a GPU e a memória. Tal capacidade é importante para lidar com grandes conjuntos de dados, reduzindo a necessidade de troca constante de dados com a memória principal do sistema.

A arquitetura Pascal da TITAN X (GP102) apresenta uma estrutura interna composta por múltiplos multiprocessadores de *streaming* (SMs), cada um deles contendo centenas de núcleos capazes de executar operações em paralelo. Além disso, há aprimoramentos no gerenciamento de energia e escalabilidade da arquitetura, permitindo maior desempenho com menor consumo energético em comparação às gerações anteriores.

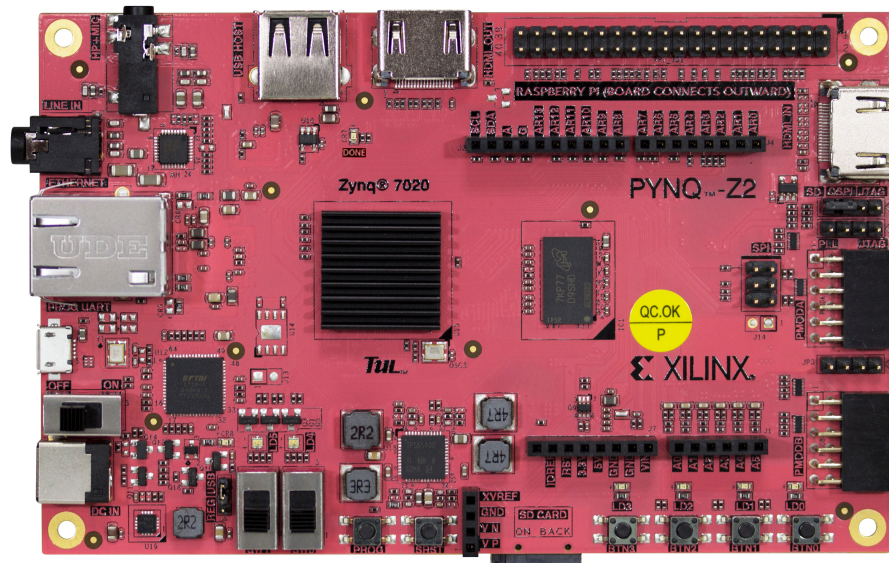
Embora a **NVIDIA TITAN X (Pascal)** tenha sido uma das GPUs mais avançadas em seu lançamento em 2016, atualmente existem placas mais recentes no mercado, como as baseadas nas arquiteturas Ampere e Ada Lovelace, respectivamente. Essas novas GPUs oferecem avanços significativos em desempenho, eficiência energética e recursos especializados, como o suporte a núcleos de tensores mais avançados voltados para algoritmos utilizados na área de IA.

Ainda assim, a TITAN X (Pascal) continua sendo uma GPU poderosa, com capacidade suficiente para lidar com os requisitos computacionais de diversos projetos, especialmente em tarefas de aprendizado de máquina e computação científica que não demandem os recursos mais recentes.

5.3 Pynq Board Z1

A placa de FPGA utilizada foi a **Pynq Board Z1**. Essa é uma placa de desenvolvimento projetada para explorar o **SoC (System-on-Chip) Xilinx Zynq-7000**. Tanto a Pynq Board como o Zynq-7000, trazem características que serão abordadas a seguir. A Pynq Board Z1 pode ser observada na Figura 13.

Figura 13 – Placa PYNQ-Z1.



Fonte: (DIGILENT, 2025).

As **Pynq Boards** são placas que suportam o projeto PYNQ. O projeto PYNQ (Python Productivity for Zynq) é um ecossistema de desenvolvimento de software e hardware aberto, lançado pela Xilinx, com o objetivo de facilitar o uso de sistemas baseados no SoC Zynq-7000. Ao invés de exigir conhecimentos profundos em linguagens de descrição de hardware (HDL) e ferramentas de síntese, o PYNQ permite que o desenvolvedor interaja com o hardware reconfigurável através da linguagem Python, ampliando a acessibilidade e a rapidez na criação de protótipos.

O **SoC Zynq-7000** é uma linha de dispositivos produzidos pela Xilinx que integra, em um único chip, um núcleo de Sistema de Processamento (*Processing System* - PS) e um núcleo de Lógica Programável (*Programmable Logic* - PL). De forma geral, é um circuito integrado que combina o processamento baseado em software (CPU) e de hardware reconfigurável (FPGA). O PS conta com um núcleo de processamento ARM Cortex-A9, já o PL possui um FPGA equivalente ao que seria um FPGA da família Artix-7.

O ARM Cortex-A9 é um processador baseado na arquitetura ARMv7-A, que segue o modelo RISC (*Reduced Instruction Set Computer*), conhecido por eficiência energética e alta performance. Ele é projetados para oferecer alto desempenho em sistemas de propósito

geral, com design escalável que permite múltiplos núcleos (*dual-core* ou *quad-core*), sendo bastante utilizado em dispositivos móveis e sistemas embarcados.

O Artix-7 é uma família de FPGAs da Xilinx, otimizada para desempenho e integração de sistemas utilizando tecnologia de 28 nm. Focado em eficiência energética, ele combina alto desempenho por watt com recursos avançados de DSP e largura de banda de I/Os. É amplamente utilizado em aplicações como redes de alta velocidade (10G a 100G), prototipagem de ASICs e sistemas embarcados. Além disso, pertence à série 7 de FPGAs, que compartilham uma arquitetura escalável e otimizada, permitindo migração rápida de designs e reutilização de IPs entre gerações.

Semelhante ao que foi feito nas seções anteriores, a nomenclatura referente ao SoC Zynq-7000 é detalhada a seguir:

- **SoC:** Indica que se trata de um circuito integrado que combina diversos componentes de um sistema completo em um único chip. Esses componentes geralmente incluem um processador central (CPU), memória, interfaces de entrada/saída e, em alguns casos, unidades especializadas como GPUs, DSPs ou, como nesse caso, FPGAs.
- **Zynq:** Refere-se à linha de dispositivos híbridos desenvolvidos pela Xilinx. Essa linha possui uma arquitetura heterogênea que combina subsistemas multicore ARM otimizados para uma ampla gama de requisitos de processamento com a lógica programável de FPGAs. Além disso, integra blocos de hardware especializados para aceleração de aplicações completas, oferecendo uma solução flexível e escalável que varia desde dispositivos otimizados para custo até opções de alto desempenho e específicas para aplicações embarcadas.
- **7000:** Identifica a série específica dentro da linha Zynq. A série 7000 abrange dispositivos com diferentes capacidades de hardware, variando no número de blocos lógicos, blocos DSP e memória RAM em bloco. A combinação desses recursos atende tanto aplicações de baixo consumo quanto aquelas que exigem maior desempenho computacional.

O FPGA presente no Zynq-7000 possui os seguintes recursos:

- 13.300 blocos lógicos, cada um com quatro LUTs de 6 entradas e 8 flip-flops;
- 630 KB de memória RAM em bloco rápida (Block RAM);
- 4 blocos de gerenciamento de clock, cada um com um loop de fase travada (*Phase-Locked Loops* - PLL) e um gerenciador de clock em modo misto (*Mixed-Mode Clock Manager* - MMCM);

- 220 blocos DSPs (processamento digital de sinais);
- Conversor analógico-digital (*Analog-to-Digital Converter* - ADC) integrado no chip.

Além disso, a placa Pynq Z1 ainda disponibiliza diversos recursos periféricos e interfaces de expansão os quais tornam o desenvolvimento de projetos complexos (como aplicações de visão computacional, controle, IoT, entre outras) muito mais simples, ao oferecer um ecossistema de hardware pronto e bem documentado.

6 Metodologia

A metodologia deste trabalho foi desenvolvida em várias etapas, contemplando diferentes *datasets*, topologias e plataformas de hardware. Foram implementadas três redes neurais, uma para cada *dataset* (MNIST, CIFAR-10 e SAT-6), avaliadas em três arquiteturas distintas (CPU, GPU e FPGA).

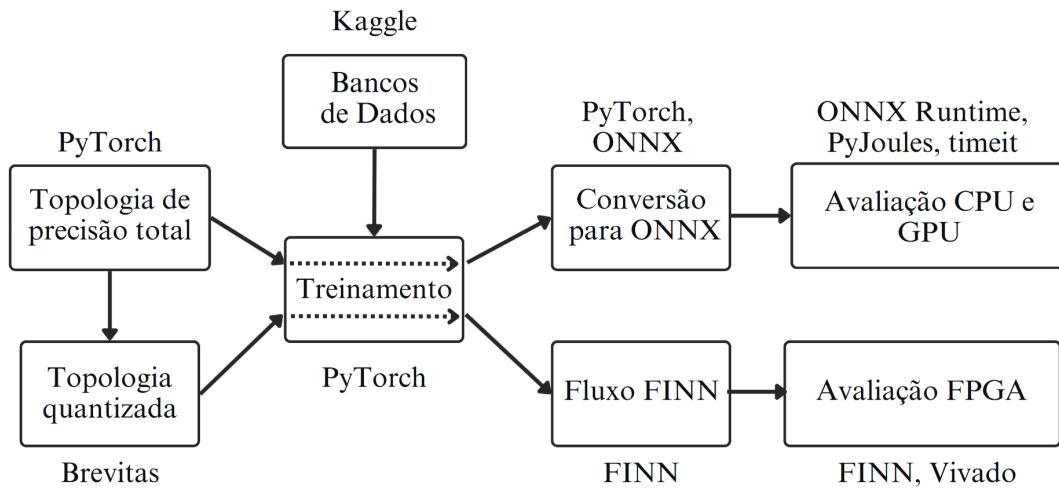
As etapas realizadas foram a aquisição do banco de dados, definição de modelos em precisão total, definição de modelos quantizados, treinamento dos modelos, conversão dos modelos para o formato ONNX, fluxo FINN, avaliação na CPU, GPU e FPGA.

Para organizar o trabalho, as etapas podem ser agrupadas em dois fluxos principais: i) a implementação em CPU e GPU e ii) a implementação em FPGA. Essa divisão se justifica porque a implementação das redes na CPU e GPU, foi praticamente idêntica, porém, difere significativamente da implementação no FPGA. Enquanto na CPU e GPU utilizou-se um modelo em precisão total, a implementação na FPGA exigiu a quantização do modelo, uma etapa essencial no fluxo do FINN.

Além disso, a semelhança entre as implementações em CPU e GPU também foi favorecida pela utilização de bibliotecas Python como o ONNX Runtime, que permite a execução do mesmo modelo em diferentes hardwares sem a necessidade de ajustes adicionais, e o PyJoules, responsável por encapsular a coleta de dados sobre consumo de energia de diferentes componentes do sistema.

Na Figura 14 é apresentado o fluxograma com a visão geral de como as etapas do trabalho estão relacionadas. Nas próximas seções as principais etapas serão apresentadas em maiores detalhes.

Figura 14 – Fluxograma com visão geral das principais etapas da metodologia.



Fonte: o Autor, 2024.

6.1 Aquisição dos Dados

Os dados dos *datasets* MNIST, CIFAR-10 e SAT-6 foram adquiridos pela plataforma Kaggle (KAGGLE, 2025). Primeiramente, foi realizado o *download* dos arquivos diretamente para um diretório local, mantendo os formatos originais de cada conjunto. A partir dos dados nos formatos originais e com base nos códigos disponíveis no Kaggle, foi possível transformar esses dados para o formato NCHW (Canal, Altura, Largura - *Channel, Height, Width*), formato de dados de imagens padrão usado pelo PyTorch.

Após essa primeira transformação, os dados foram separados entre conjunto de treinamento e conjunto de teste e organizados para serem lidos em tamanhos de lotes arbitrários através de um *Dataloader*. Um *Dataloader* é um componente essencial em muitos *frameworks* de aprendizado de máquina, que facilita o carregamento e a manipulação de dados durante o treinamento e a avaliação de modelos. Ele ajuda a organizar, carregar e processar os dados de maneira eficiente. Basicamente, ele cria um iterador que permite acessar os dados de forma sequencial ou aleatória além de poder carregar os dados em paralelo usando vários processos, aumentando a eficiência ao evitar que o treinamento fique esperando pelo carregamento dos dados. Além disso, é eficiente ao trabalhar com conjuntos de dados grandes que não cabem na memória RAM, uma vez que carrega partes dos dados conforme necessário.

Com esse procedimento, foi possível realizar a leitura dos *datasets* nas etapas subsequentes de maneira mais simples e praticamente idêntica para os três conjuntos de dados. Isso permitiu manter uma estrutura consistente e facilmente reproduzível, facilitando a troca entre os *datasets* durante o desenvolvimento do trabalho, promovendo maior uniformidade e eficiência no processo.

6.2 Topologia com Precisão Total

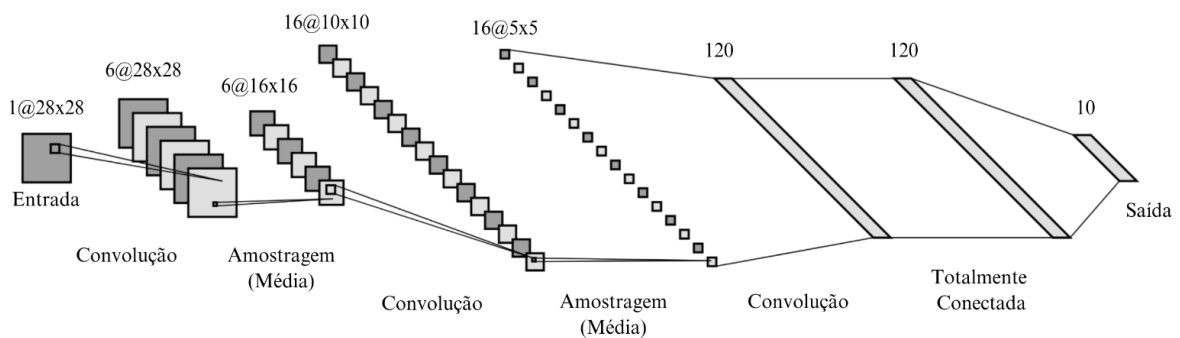
Nessa seção serão apresentadas as topologias de precisão total utilizada para cada um dos três *datasets*.

A topologia escolhida para o *dataset* MNIST foi a LeNet-5, proposta por Lecun *et al.* (1998). Trata-se de uma Rede Neural Convolucional (CNN) projetada especificamente para tarefas de reconhecimento de caracteres, como os dígitos manuscritos presentes no MNIST (LECUN *et al.*, 1998). A arquitetura implementada é detalhada na Tabela 1 e ilustrada na Figura 15.

A segunda topologia, utilizada a classificação do *dataset* CIFAR-10 foi baseada em uma adaptação de um modelo disponibilizado como exemplo no projeto *tiny-dnn* (NOUMI; CONTRIBUTORS, 2016), voltado para a classificação deste conjunto de dados. A configuração final implementada é apresentada na Tabela 2 e ilustrada na Figura 16.

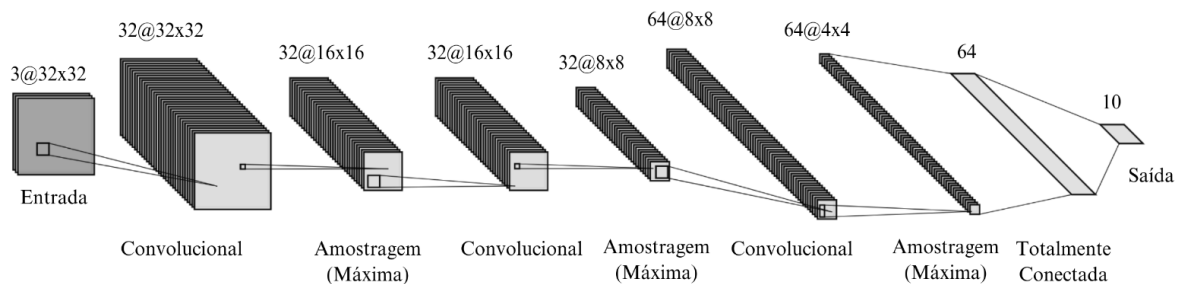
A terceira e última topologia, a qual foi utilizada para classificar o *dataset* SAT-6, é a mesma topologia utilizada por Ely *et al.* (2024) para a classificação deste mesmo *dataset*. Essa topologia é apresentada na Tabela 3 e pode ser observada na figura 17.

Figura 15 – Representação gráfica da topologia para classificação do *dataset* MNIST.

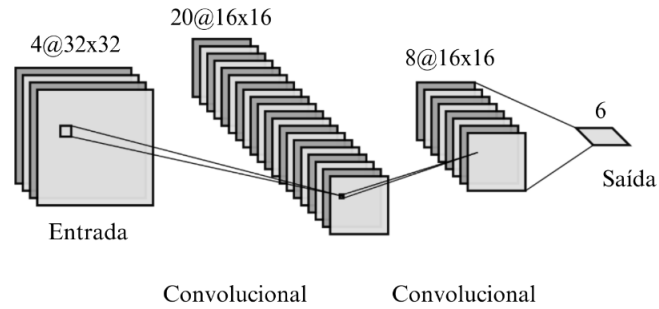


Fonte: o Autor, 2024.

Figura 16 – Representação gráfica da topologia para classificação do *dataset* CIFAR-10.



Fonte: o Autor, 2024.

Figura 17 – Representação gráfica da topologia para classificação do *dataset* SAT-6.

Fonte: o Autor, 2024.

Tabela 1 – Detalhes da topologia da rede neural baseada na arquitetura LeNet-5 para classificação do *dataset* MNIST.

Topologia da Rede Neural para o MNIST					
Tipo de Camada	Saída	Filtro	Passo	Preenchimento	Ativação
Convolutacional	$6 \times 28 \times 28$	5×5	1	2	Tanh
Amostragem (Média)	$6 \times 14 \times 14$	2×2	2	0	-
Convolutacional	$16 \times 10 \times 10$	5×5	1	0	Tanh
Amostragem (Média)	$16 \times 5 \times 5$	2×2	2	0	-
Convolutacional	$120 \times 1 \times 1$	5×5	1	0	Tanh
Totalmente Conectada	10	-	-	-	-

Tabela 2 – Detalhes da topologia baseada em exemplo do projeto TinyDNN para classificação do *dataset* CIFAR-10.

Topologia da Rede Neural para o CIFAR-10					
Tipo de Camada	Saída	Filtro	Passo	Preenchimento	Ativação
Convolutacional	$32 \times 32 \times 32$	5×5	1	2	ReLU
Amostragem (Máxima)	$32 \times 16 \times 16$	2×2	2	0	-
Convolutacional	$32 \times 16 \times 16$	5×5	1	2	ReLU
Amostragem (Máxima)	$32 \times 8 \times 8$	2×2	2	0	-
Convolutacional	$64 \times 8 \times 8$	5×5	1	2	ReLU
Amostragem (Máxima)	$64 \times 4 \times 4$	2×2	2	0	-
Totalmente Conectada	64	-	-	-	ReLU
Totalmente Conectada	10	-	-	-	-

Tabela 3 – Descrição da topologia baseado no trabalho de Ely *et al.* (2024) para classificação do *dataset* SAT-6.

Topologia da Rede Neural para o SAT-6					
Tipo de Camada	Saída	Filtro	Passo	Preenchimento	Ativação
Convolutacional	$20 \times 16 \times 16$	3×3	2	1	ReLU
Convolutacional	$8 \times 16 \times 16$	1×1	1	0	ReLU
Totalmente Conectada	6	-	-	-	-

Nas Tabelas 1, 2 e 3 estão apresentadas as camadas das topologias em sequência de execução, o tamanho das saídas, tamanho dos filtros utilizados, o número dos passos e a função de ativação utilizada ao final de cada camada. O tamanho das saídas estão no formato Canal, Altura e Largura (*Channels, Height e Width* - CHW).

Além disso, como já apresentado na Seção 4, o tamanho dos dados de entrada de cada rede pode ser observado na Tabela 4. Vale notar que o tamanho de entrada para o Sat-6 é 32x32 ao invés de 28x28 apresentado anteriormente pois na aquisição dos dados foi realizado o preenchimento de 4 pixels para ser compatível com o modelo.

Tabela 4 – Tamanhos de entrada das redes neurais utilizadas.

Dataset	Tamanho de Entrada
MNIST	$1 \times 28 \times 28$
CIFAR-10	$3 \times 32 \times 32$
Sat-6	$4 \times 32 \times 32$

Ainda mais, vale notar que a última camada dessas topologias não possuem função de ativação pois: (i) durante o treinamento, a função de perda Entropia Cruzada, a qual será explicada em maiores detalhes na Seção 6.5 já aplica a função de ativação *Softmax*, e (ii) durante a inferência, por se tratar de uma tarefa de classificação, apenas é de interesse o índice de maior valor de saída da rede e em geral as funções de ativação não alteram esse índice.

Todas as topologias foram implementadas na linguagem de programação Python com o auxílio da biblioteca PyTorch.

6.3 Topologia Quantizada

As topologias quantizadas foram implementadas utilizando a biblioteca Brevitas com base nas topologias de precisão total. As modificações incluíram alteração das camadas implementadas pelo PyTorch por sua respectiva camada quantizada disponível no Brevitas e definição da número de bits utilizados.

O mapeamento das funções do PyTorch para as funções em Brevitas podem ser observado na Tabela 5.

Tabela 5 – Mapeamento de funções entre PyTorch e Brevitas.

Tipo da Camada	PyTorch	Brevitas
Convolutacional	Conv2d	QuantConv2d
Totalmente Conectada	Linear	QuantLinear
Ativação	Tanh	QuantHardTanh
Amostragem média	AvgPool2d	TruncAvgPool2d
Ativação	ReLU	QuantReLU

Para as camadas convolucionais, totalmente conectadas e a função de ativação ReLU, a única diferença está na quantização. Por outro lado, para as camadas de amostragem e demais funções de ativação, foram necessárias considerações específicas.

Nas camadas de amostragem, a versão quantizada para a amostragem média incluiu um truncamento pois a divisão é realizada através de um *shift* nos bits. Já a camada de amostragem máxima não precisou de uma versão quantizada pois a operação de obter o número máximo independe de quantização.

Quanto às funções de ativação, a função *Tanh* foi substituída pela versão quantizada *HardTanh* com valores máximos e mínimos de 1 e -1, respectivamente. Essa função pode ser observada na Figura 2. Essa alteração foi necessária pois o FINN não suporta a versão quantizada da *Tanh* diretamente.

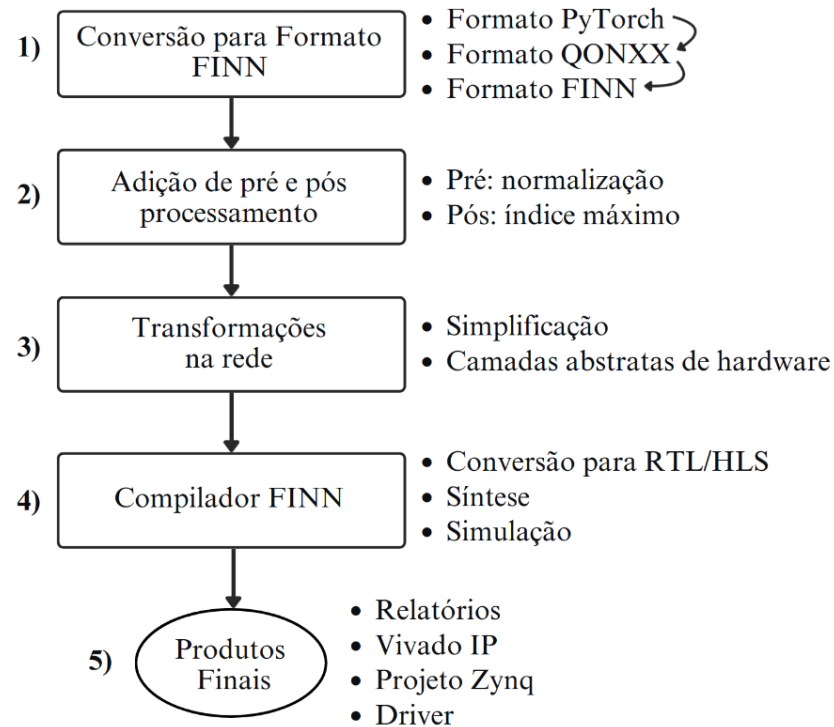
Ainda mais, quanto ao nível de quantização, o Brevitas permite utilizar diferentes números de bits entre camadas, para a função de ativação e para os pesos. Para fins de simplificação, foram utilizados a mesma quantidade de bits em todas as quantizações.

Para a rede do MNIST, a quantização foi de 8 bits, para o CIFAR-10 foi de 4 bits e para o SAT-6 a quantização foi de 8 bits. A quantidade de bits foi definida como sendo aqueles que resultaram em uma precisão do modelo quantizado equivalente ou maior do que os modelos de precisão total. De forma mais precisa, o método consistiu em realizar o treinamento da rede partindo da menor quantização (1 bit) e dobrando o número de bits até atingir uma acurácia semelhante à acurácia do modelo com precisão total. Dessa forma, com acurácias semelhantes a comparação entre o desempenho dos modelos se torna mais equitativa.

6.4 Fluxo FINN

O fluxo completo do FINN é apresentado na Seção 3.8. No entanto, a presente seção tem como objetivo apresentar esse fluxo, porém, através de uma abordagem mais prática e específica de cada rede. Uma visão geral desse fluxo é apresentado na Figura 18.

Figura 18 – Fluxograma da visão geral do fluxo FINN realizado.



Fonte: o Autor, 2024.

6.4.1 Conversão para Formato FINN

O fluxo no FINN tem início convertendo o modelo quantizados no Brevitas para o modelo *QONNX* e em seguida convertendo para o modelo FINN. Todas essas conversões são realizadas através de funções disponíveis nas bibliotecas do FINN.

6.4.2 Pré e Pós Processamento

Com o modelo no formato correto, é possível adicionar um pré-processamento e um pós-processamento dos dados na própria topologia da rede. Isso é possível e incentivado nas documentações pois como o FINN realiza várias transformações na rede, o pré-processamento e o pós-processamento geralmente são "fundidos" com outros blocos, sem custos adicionais.

O pré-processamento inserido na rede foi a divisão dos valores de entrada por 255, ou seja, a normalização dos dados. Já o pós-processamento consistiu na inserção de um bloco final que retorna o índice do maior valor de saída da rede.

6.4.3 Transformações na Rede

Após o pré-processamento e pós-processamento inseridos na rede, foi realizada a etapa de simplificação e conversão para camadas de hardware. O processo realizado nessas duas etapas, de um ponto de vista prático, consiste em, com as funções disponibilizadas no FINN, reordenar e transformar os blocos da rede em blocos mais simples (como por exemplo, absorver blocos de multiplicação junto à blocos de multiplicação de matrizes) e em blocos que possam ser mapeados para hardware posteriormente. Além disso, é necessário mover os blocos de forma que as operações que modificam o layout dos dados fiquem posicionados na entrada ou saída da rede e não reste nenhum bloco que não seja um bloco mapeável para hardware, caso contrário, serão gerados erros nas etapas seguintes. Para a verificação dos modelos intermediários foi utilizada a ferramenta Netron.

O compilador do FINN conta com um algoritmo projetado para realizar essas duas etapas automaticamente. No entanto, dependendo da topologia da rede, são necessárias transformações específicas. Essas transformações são bastante específicas do FINN, da forma como a topologia foi implementada, e geralmente envolvem tentativa e erro, já que frequentemente não há mensagens claras indicando o motivo da falha de certas transformações.

No caso da topologia para o MNIST e SAT-6, o algoritmo padrão do FINN foi praticamente suficiente para realizar a simplificação necessária, com a exceção de poucas transformações adicionais. Para a topologia do CIFAR-10 foi necessário realizar uma maior quantidade de transformações adicionais.

Além disso, ao final da etapa de simplificação, realizou-se uma verificação de integridade do modelo. Nessa verificação, valores de entrada do conjunto de teste foram fornecidos tanto ao modelo quantizado quanto ao modelo simplificado, permitindo comparar as saídas e garantir sua consistência. Essa etapa é fundamental para assegurar que, após as transformações aplicadas na rede, o modelo mantenha o comportamento esperado antes de iniciar a compilação final, a qual pode exigir um tempo considerável. Assim, caso surjam inconsistências, é possível economizar tempo ao corrigir eventuais problemas antes de concluir todo o processo.

Vale notar, também, que a etapa de verificação do modelo em Python após a simplificação, pode ser realizada pela compilação do FINN. No entanto, para ter um controle maior nessa etapa de verificação, foi optado por realizar essa verificação manualmente.

Após essas duas etapas (simplificação de conversão para camadas de hardware), é esperado que o modelo possua apenas blocos mapeados para camadas de hardware. Nesse ponto do processo é conveniente passar o modelo para o compilador do FINN que realiza o restante das etapas do fluxo do FINN detalhadas na Seção 3.8 e gera os produtos finais.

6.4.4 Parâmetros de Compilação

A compilação final possui diversos parâmetros que podem ser ajustados de acordo com a necessidade e objetivo do projeto. Entre os principais parâmetros utilizados foram: i) a placa utilizada; ii) o número de imagens por segundo alvo; e iii) a frequência de clock. Vale notar que a o número de imagens por segundo alvo é utilizada para uma geração automática da configuração de *defolding*, que por sua vez, define o quanto a rede é paralelizada.

A placa alvo foi a PYNQ Board Z1, a qual é apresentada na Seção 5.3. A frequência do clock foi definida em 100MHz em todas as redes, frequência razoável para ter obter um bom desempenho e não ter problemas de temporização.

Para a rede do MNIST e do SAT-6, o número de imagens por segundo foi definido para um número alto como 1.000.000 tendo como alvo o melhor desempenho da rede, já que na compilação, mesmo que o número de imagens por segundo alvo não consiga ser atingindo devido à limitações de hardware, o compilador do FINN busca o melhor desempenho dentro do possível.

Já para a rede do CIFAR-10, o número de imagens por segundo foi definido para 5.000, pois mesmo que o compilador do FINN adapte o *folding* para possibilitar a implementação na placa alvo, números grandes de imagem por segundo alvo, não permitiam a conclusão da síntese na placa FPGA alvo devido à falta de recursos. Assim, foi necessário diminuir o valor desse parâmetro de 1.000.000 para 5.000.

Além disso, outro parâmetro da compilação importante passado para o compilador foi um arquivo com valores de entrada e saída esperados pela rede, os quais são usados em uma simulação que o FINN realiza junto ao Vivado para medir a acurácia após a síntese durante a compilação.

6.4.5 Produtos Finais

Os produtos finais consistem em um relatório de estimativas, úteis para estimar a utilização de recursos e desempenho da rede antes de realizar toda a compilação, a qual leva tempo para concluir. Consiste, também, na síntese do hardware, geração de um Vivado IP, simulação para medir a performance e acurácia, um *bitstream* file, *driver* PYNQ e um pacote de implementação para a placa alvo como mencionado na Seção 3.8.

6.5 Treinamento

O treinamento das redes foi realizado em todas as topologias (todos os *datasets*, precisão total e quantizada), sendo eles praticamente idênticos entre si apenas diferindo pelo número de épocas e tamanho do lote. Alguns parâmetros importantes no treinamento

foram: i) Número de épocas; ii) Tamanho dos lotes; iii) Taxa de aprendizado; iv) Função de perda; v) Otimizador. Os parâmetros comuns entre os treinamentos pode ser observado na Tabela 6.

Tabela 6 – Parâmetros Comuns de Treinamento.

Parâmetro	Valor
Tamanho do lote	256
Taxa de aprendizado	0.001
Função de perda	Entropia Cruzada
Otimizador	Adam

O otimizador Adam foi escolhido pois dentre suas vantagens estão a adaptação automática da taxa de aprendizado, rápida convergência e baixo custo computacional. Além disso, é uma boa escolha inicial devido à sua robustez em relação a ajustes de hiperparâmetros. Diferentemente de outros otimizadores, que podem exigir ajustes finos na taxa de aprendizado para obter bons resultados, o otimizador Adam tende a funcionar bem com seus valores padrão de hiperparâmetros. Isso reduz a necessidade de experimentação extensiva e permite um início rápido do treinamento com resultados eficazes (GOODFELLOW; BENGIO; COURVILLE, 2016, p. 309).

Por se tratar de um problema de classificação, foi escolhida a função custo de Entropia Cruzada. Essa função de custo é interessante para problemas de classificação pois mede a discrepância entre a distribuição real das classes e a distribuição prevista pelo modelo. Além disso, a Entropia Cruzada tende a facilitar uma convergência mais rápida durante o treinamento (BISHOP, 2006, p. 235). Essa função também permite que as saídas do modelo sejam interpretadas diretamente como probabilidades reais das classes, o que é especialmente útil em aplicações onde a interpretação probabilística é importante.

A taxa de aprendizado de 0.001 foi escolhida pois forneceu um bom ponto de partida com o otimizador Adam com base em testes. Além disso, proporciona um equilíbrio entre velocidade de convergência e estabilidade, minimizando o risco de oscilações no processo de otimização. Além disso, taxas de aprendizado muito altas podem fazer com que o modelo não convirja adequadamente, enquanto taxas muito baixas podem resultar em tempos de treinamento excessivamente longos.

Optou-se por utilizar um tamanho de lote igual a 256, pois ele oferece um equilíbrio adequado entre eficiência computacional e estabilidade durante o treinamento. Lotes maiores tendem a proporcionar atualizações mais precisas nos parâmetros do modelo, reduzindo a variação entre as iterações e tornando o aprendizado mais consistente. Contudo, tamanhos excessivamente grandes podem demandar uma quantidade significativa de memória, o que limita a viabilidade em máquinas com recursos mais modestos. Assim, o tamanho de lote 256 revelou-se uma escolha eficiente, garantindo um treinamento estável sem comprometer a qualidade do modelo.

O número de épocas para o treinamento das topologias de precisão total foi ajustado levando em conta o tempo para treinamento não ser muito grande mas ainda obter uma precisão aceitável. Já para as topologias quantizadas o número de épocas foi ligeiramente reduzido devido ao seu maior custo computacional. Os valores foram de 20 e 15 épocas, respectivamente, para as topologias de precisão total e quantizadas.

6.6 Avaliação

A avaliação dos modelos foram realizadas nas três plataformas: CPU, GPU e FPGA, sendo que a avaliação na CPU e GPU foram quase idênticas e diferentes da avaliação no FPGA. As duas formas de avaliação utilizadas são apresentadas nas Seções 6.6.1 e 6.6.2.

As métricas de desempenho utilizadas foram: acurácia, latência, vazão e dissipação de potência, as quais foram apresentadas na Seção 2.2.

6.6.1 Avaliação na CPU e GPU

A avaliação das redes em CPU e GPU foi realizada durante a inferência das redes no formato ONNX utilizando a biblioteca ONNX Runtime. O primeiro passo dessa etapa foi a conversão dos modelos de precisão total desenvolvidos em PyTorch para o formato ONNX, processo realizado com o suporte de ambas bibliotecas. Com os modelos no formato ONNX, foi possível executar a inferência nas duas plataformas, CPU e GPU, utilizando o mesmo modelo. Isso foi viabilizado pela flexibilidade do ONNX Runtime, que permite alternar entre diferentes plataformas de execução modificando apenas um argumento na função de inicialização da inferência do modelo. No entanto, vale notar que para a execução das rede na GPU, o *backend* escolhido para ser executado pelo ONNX Runtime foi a plataforma CUDA. Dessa forma, para a inferência na GPU também foi necessário a instalação dessa plataforma.

A primeira métrica avaliada foi a acurácia, utilizando os dados de teste aplicados ao modelo de precisão total desenvolvido em PyTorch. Em seguida, essa mesma métrica foi analisada no modelo convertido para o formato ONNX. Essa etapa foi importante para garantir que o processo de conversão não alterou o comportamento do modelo, assegurando a consistência entre as versões PyTorch e ONNX.

A segunda avaliação realizada foi a da latência. Para isso, o modelo foi submetido a 1.000 inferências consecutivas com valores de entrada gerados aleatoriamente, enquanto o tempo de execução de cada iteração foi registrado. A medição foi feita utilizando o módulo Python `timeit`, projetado especificamente para medir o tempo de execução de trechos curtos de código Python, garantindo precisão na análise de desempenho. A partir da medida de todas as execuções foi possível plotar um gráfico com a latência em cada

execução para cada *dataset*. Além disso foi possível extrair indicadores como a latência média e desvio padrão.

A terceira avaliação foi feita de forma a medir tanto a vazão como a potência, ao mesmo tempo, para diferentes tamanhos de lote. A vazão foi obtida através da medida do tempo de execução para cada lote da mesma forma que na medida da latência, porém, com um número menor de repetições em cada lote devido ao tempo de execução ser longo para tamanhos grande de lote. A vazão pôde ser calculada dividindo o tamanho do lote pelo tempo de execução.

A avaliação da potência foi realizada utilizando a biblioteca PyJoules, a qual permitiu medir a energia consumida durante a execução de blocos específicos de código Python na CPU ou GPU, de forma individual, em sistemas operacionais baseados em Linux. Com o tempo de execução e energia gasta foi possível estimar a potência em watts de cada plataforma dividindo a energia gasta pelo tempo de execução.

Além disso, é relevante destacar que a avaliação dos modelos na CPU e GPU incluiu o tempo de processamento de normalização e a extração do índice correspondente ao maior valor na saída das redes. Embora essas operações não causem um impacto significativo nos resultados, há uma leve influência que merece ser considerada, especialmente porque a implementação no FPGA também incorpora essas operações de forma integrada.

6.6.2 Avaliação no FPGA

A avaliação das métricas de desempenho no FPGA foram realizadas através de simulações e extração dos relatórios gerados pelo FINN e pelo Vivado.

As métricas de desempenho de acurácia, latência e vazão foram obtidas através da simulação no Vivado que o compilador do FINN realiza automaticamente caso habilitado. Como mencionado na Seção 6.4, para essa simulação, é necessário gerar arquivos com valores de entrada e valores esperados de saída para serem passados como argumento para o compilador FINN. Já a métrica de desempenho de dissipação de potência foi estimada através do relatório de consumo energético, desempenho térmico e operacional do sistema.

Como uma métrica adicional pra avaliação das redes no FPGA, também foi avaliada a utilização de recursos do FPGA. Como essa métrica se aplica apenas ao FPGA não será usada para fins de comparação com outros hardwares, mas pode ser utilizada para avaliar a eficiência de implementação na placa alvo. A utilização de recursos também foi extraída dos relatórios gerados pelo Vivado no projeto final gerado pelo FINN.

Vale notar ainda, que os resultados obtidos para a plataforma do FPGA, está associada à execução de uma entrada por vez.

7 Resultados e Discussões

Neste capítulo, são apresentados os resultados do desempenho das redes neurais nas diferentes plataformas utilizadas no estudo: CPU, GPU e FPGA. A análise contempla métricas fundamentais, como acurácia, latência, vazão e dissipação de potência, fornecendo uma visão geral das capacidades e limitações de cada plataforma em diferentes cenários de aplicação. Ao final, é feita uma comparação entre as plataformas, destacando suas principais vantagens e desvantagens.

7.1 Desempenho na CPU

A Tabela 7 sintetiza os resultados de desempenho obtidos na CPU, englobando aspectos de acurácia, latência, vazão e dissipação de potência. As topologias executadas nessa plataforma foram as topologias de precisão total.

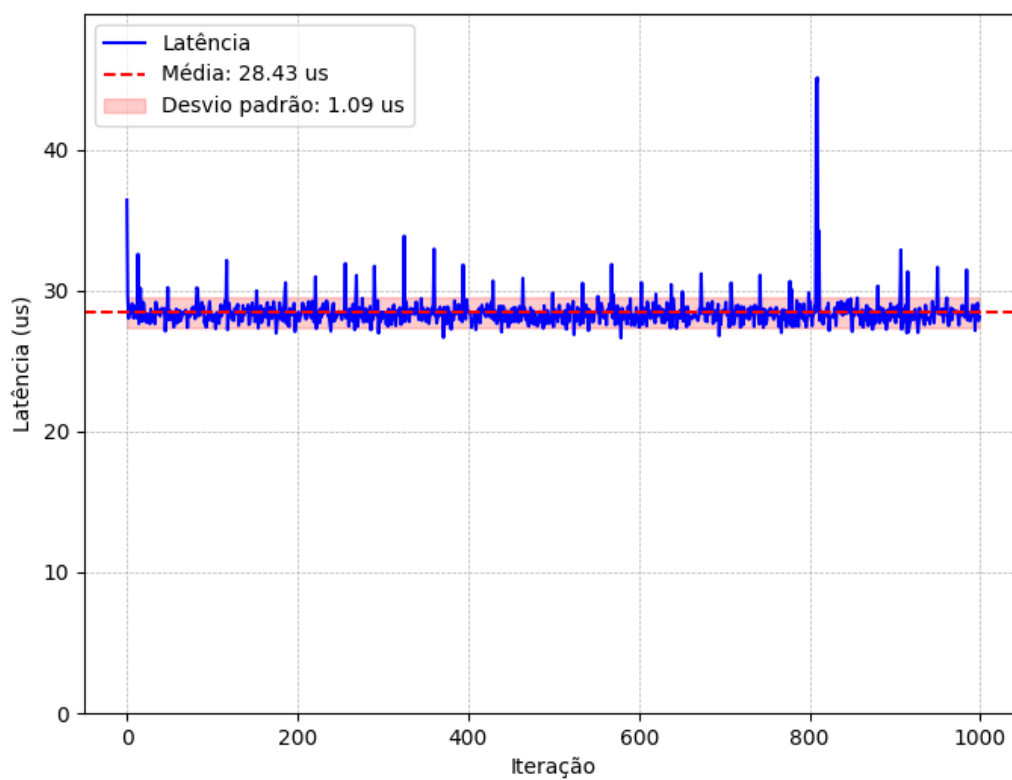
Tabela 7 – Desempenho e Potência para Inferência na CPU.

<i>Dataset</i>	Acurácia (%)	Latência (μs)	Vazão (fps)	Potência (W)
MNIST	98.64	28,43	244.000	30
CIFAR-10	70.32	93,14	20.510	36
SAT-6	97.69	17,35	475.000	57

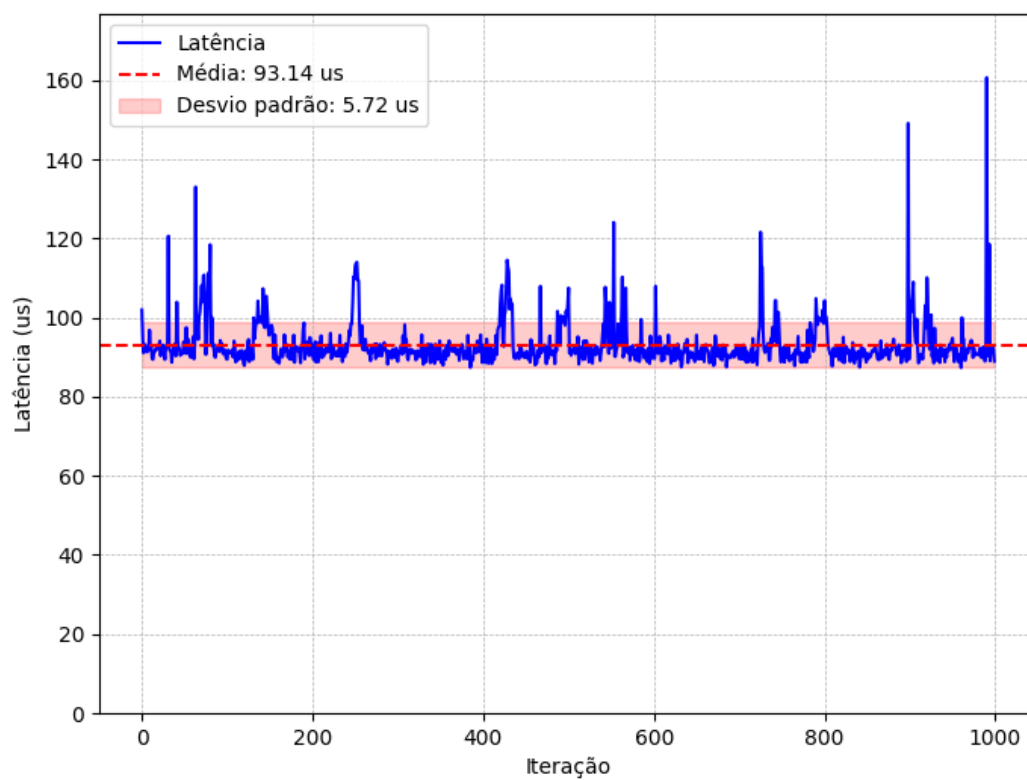
Ao longo das subseções que seguem, são apresentados e discutidos os principais dados obtidos, destacando as limitações e peculiaridades inerentes ao ambiente de execução em CPU, como a influência do sistema operacional na variação de latência, bem como aspectos relativos à dissipação de potência em diferentes tamanhos de lote.

7.1.1 Latência

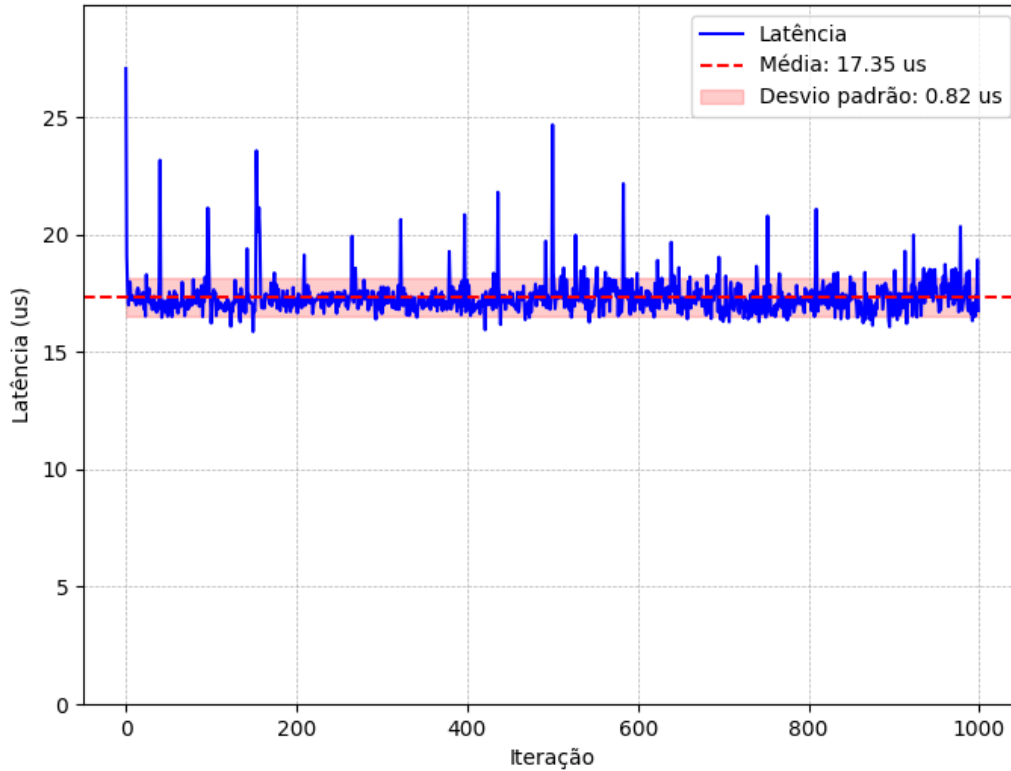
A latência obtida nas inferências realizadas na CPU pode ser observada nas Figuras 19, 20 e 21, nas quais foram obtidas as médias de 28,43 μ s, 93,14 μ s e 17,35 μ s para os *datasets*, MNIST, CIFAR-10 e SAT-6, respectivamente.

Figura 19 – Latência por iterações para o *dataset* MNIST.

Fonte: o Autor, 2024.

Figura 20 – Latência por iterações para o *dataset* CIFAR.

Fonte: o Autor, 2024.

Figura 21 – Latência por iterações para o *dataset* SAT-6.

Fonte: o Autor, 2024.

Pelos gráficos é possível observar que a latência obtida durante as inferências apresenta variações significativas em determinados momentos. Essas flutuações são relacionadas com o comportamento do sistema operacional, especificamente com as interrupções que podem ocorrer a qualquer instante durante a execução do processo. Tais interrupções incluem a execução de tarefas em segundo plano, como gerenciamento de dispositivos, operações de I/O ou alocação de recursos para outros processos em execução.

Esses eventos imprevisíveis impactam diretamente o tempo necessário para completar uma inferência, causando picos de latência observados nos gráficos. Esse fenômeno é comum em sistemas de propósito geral, como CPUs, que não possuem mecanismos dedicados exclusivamente para aplicações em tempo real. Em contrapartida, plataformas especializadas, como o FPGA, apresentam maior previsibilidade na latência devido à sua arquitetura orientada a tarefas específicas e menor interferência do sistema operacional.

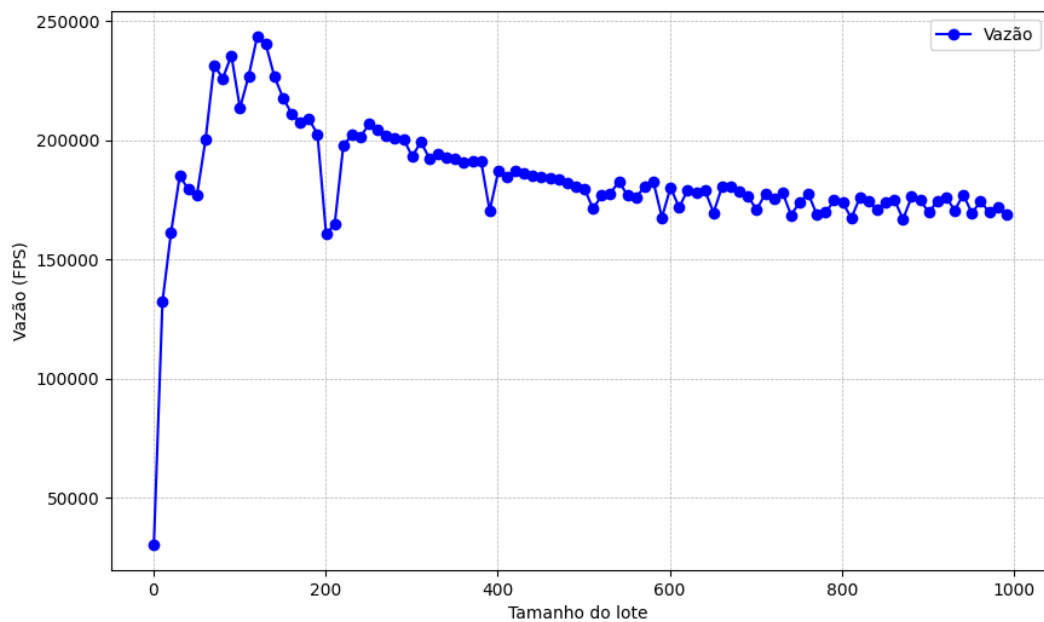
Para mitigar esses efeitos em CPUs, estratégias como o uso de sistemas operacionais de tempo real, isolamento de núcleos para execução exclusiva do processo, ou priorização de processos podem ser adotadas. Essas técnicas ajudam a minimizar as interrupções

externas, reduzindo a variação na latência. No entanto, em aplicações críticas onde a consistência da latência é essencial, a adoção de hardware dedicado, como FPGA, pode ser obrigatória, uma vez que oferece um ambiente de execução mais determinístico.

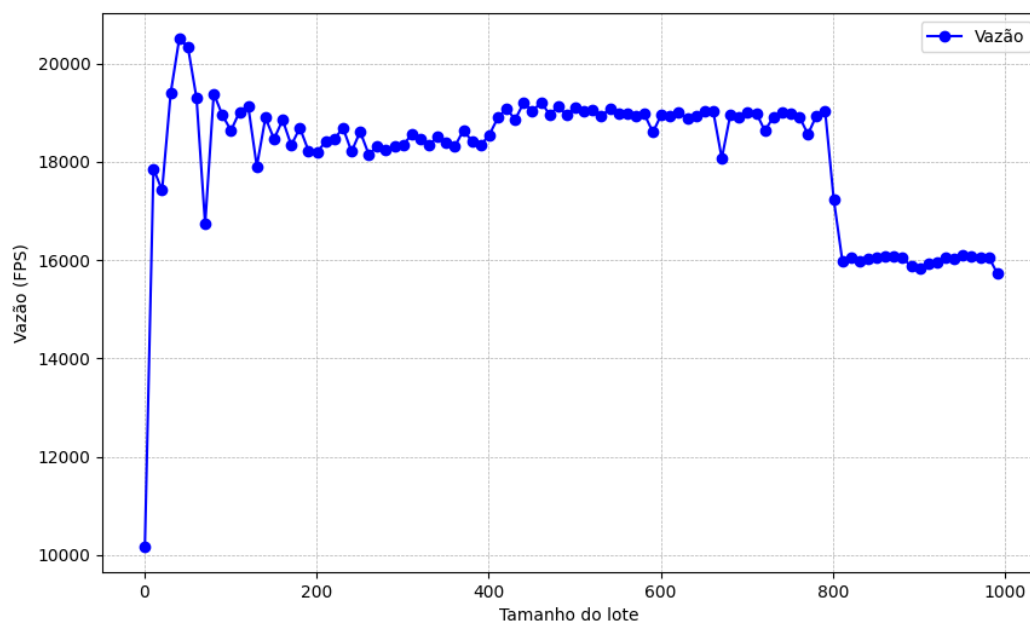
7.1.2 Vazão

Os gráficos de vazão apresentados nas Figuras 22, 23 e 24 mostram o impacto do tamanho do lote na quantidade de amostras processadas por segundo na CPU.

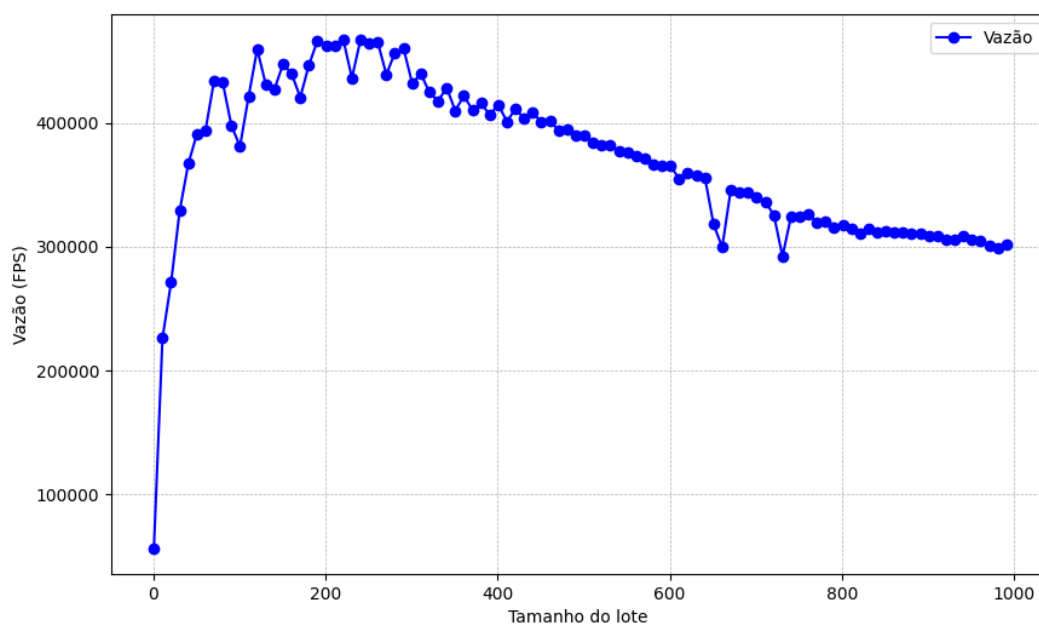
Figura 22 – Vazão por tamanho de lote para o *dataset* MNIST na CPU.



Fonte: o Autor, 2024.

Figura 23 – Vazão por tamanho de lote para o *dataset* CIFAR na CPU.

Fonte: o Autor, 2024.

Figura 24 – Vazão por tamanho de lote para o *dataset* SAT na CPU.

Fonte: o Autor, 2024.

Nos gráficos de vazão obtidos na CPU para os *datasets* MNIST e SAT-6 (Figuras 22 e 24), observa-se que há um ponto ótimo de tamanho de lote onde a vazão atinge seu valor máximo. Para o MNIST, o maior valor de vazão obtido foi de aproximadamente 24.4000 amostras por segundo quando o tamanho do lote foi em 121 entradas. Para o SAT-6, a vazão máxima foi próxima de 475.000 amostras por segundo com um tamanho de lote de 181. Após esse ponto, a vazão tende a diminuir e estabilizar em um valor menor conforme o tamanho do lote continua a aumentar.

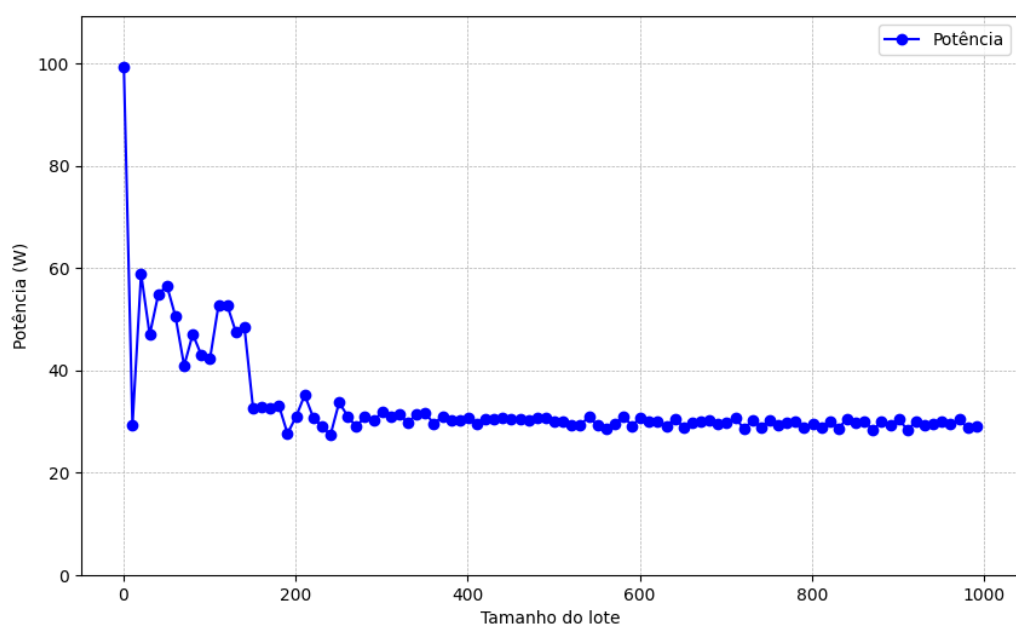
Esse comportamento pode ser explicado pela relação entre o tempo de processamento e a sobrecarga (*overhead*) associada ao gerenciamento dos lotes. Inicialmente, o aumento do tamanho do lote permite que mais amostras sejam processadas em conjunto, reduzindo a sobrecarga relativa por amostra e aumentando a vazão. No entanto, a partir de um certo ponto, o aumento adicional no tamanho do lote resulta em um crescimento proporcional do tempo de processamento sem benefícios significativos na redução da sobrecarga. Além disso, limitações de recursos da CPU, como cache e memória, podem começar a impactar negativamente a eficiência, levando a uma diminuição na vazão.

Para o *dataset* CIFAR-10, também é identificado um ponto ótimo de vazão atingindo 20510 imagens por segundo, seguido por uma diminuição similar. Entretanto, observa-se que em determinado ponto ocorre uma queda acentuada na vazão. Como a execução da inferência dos modelos foi realizada sequencialmente com tamanhos de lote variando de 1 até 1.000, é provável que a CPU tenha enfrentado limitações térmicas devido ao aumento da carga de trabalho.

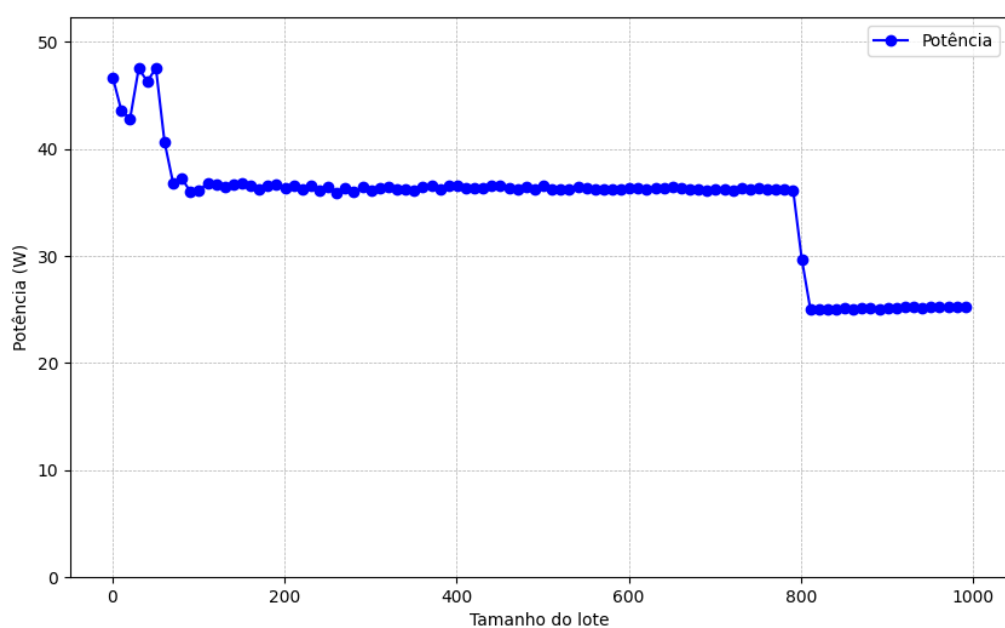
Uma explicação plausível para esse comportamento é o aquecimento da CPU e a ativação do fenômeno conhecido como redução térmica de desempenho (*thermal throttling*). Quando a temperatura do processador atinge níveis elevados, os mecanismos de proteção térmica reduzem automaticamente a frequência de operação da CPU para evitar danos ao hardware. Essa redução de frequência resulta em uma queda no desempenho computacional, aumentando a latência e consequentemente diminuindo a vazão. Como a rede neural utilizada para classificar o CIFAR-10 possui uma maior complexidade, exige maior esforço computacional, o que pode acelerar o aquecimento do processador.

7.1.3 Dissipação de Potência

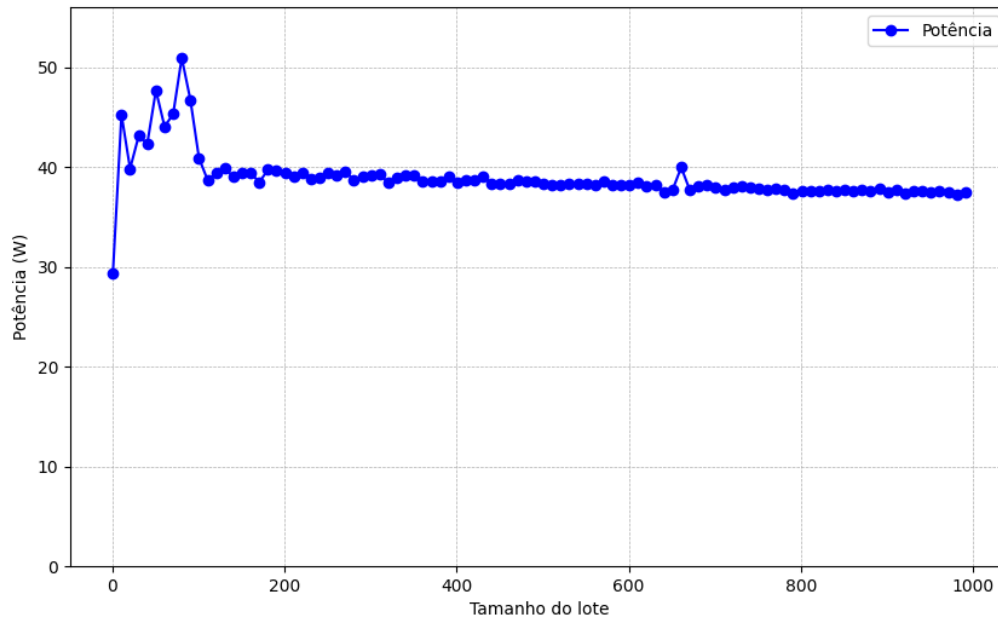
As Figuras 25, 26 e 27 apresentam os valores obtidos para dissipação de potência na CPU.

Figura 25 – Dissipação de potência para o *dataset* MNIST na CPU.

Fonte: o Autor, 2024.

Figura 26 – Dissipação de potência para o *dataset* CIFAR na CPU.

Fonte: o Autor, 2024.

Figura 27 – Dissipação de potência para o *dataset* SAT na CPU.

Fonte: o Autor, 2024.

Nos três *datasets* avaliados, os valores de potência são maiores para tamanhos de lote pequenos, o que pode estar diretamente associada aos pontos de vazão ótima observadas anteriormente. Durante esse estágio, a CPU opera em sua capacidade máxima para processar rapidamente os lotes pequenos, resultando em uma dissipação de potência mais elevada devido à alta demanda computacional e à utilização intensiva de seus núcleos de processamento.

À medida que o tamanho do lote cresce, a dissipação de potência da CPU se estabiliza em um patamar praticamente constante, atingindo os valores de aproximadamente 30W para o MNIST, de 36W para o CIFAR-10 e de 57W para o SAT-6.

No caso do *dataset* CIFAR-10, observa-se uma queda abrupta na potência de 36W para 25W no mesmo ponto em que ocorre o aumento significativo da latência e a queda na vazão, conforme identificado anteriormente. Esse comportamento pode estar associado à ativação de mecanismos de proteção térmica, como a redução térmica de desempenho, que reduz a frequência de operação da CPU para evitar superaquecimento. Com a frequência reduzida, a potência diminui consideravelmente.

7.2 Desempenho na GPU

A Tabela 8 apresenta os resultados de acurácia, latência, vazão e dissipação de potência obtidos durante as inferências na GPU. Assim como na CPU, as topologias entregadas foram executadas em precisão total.

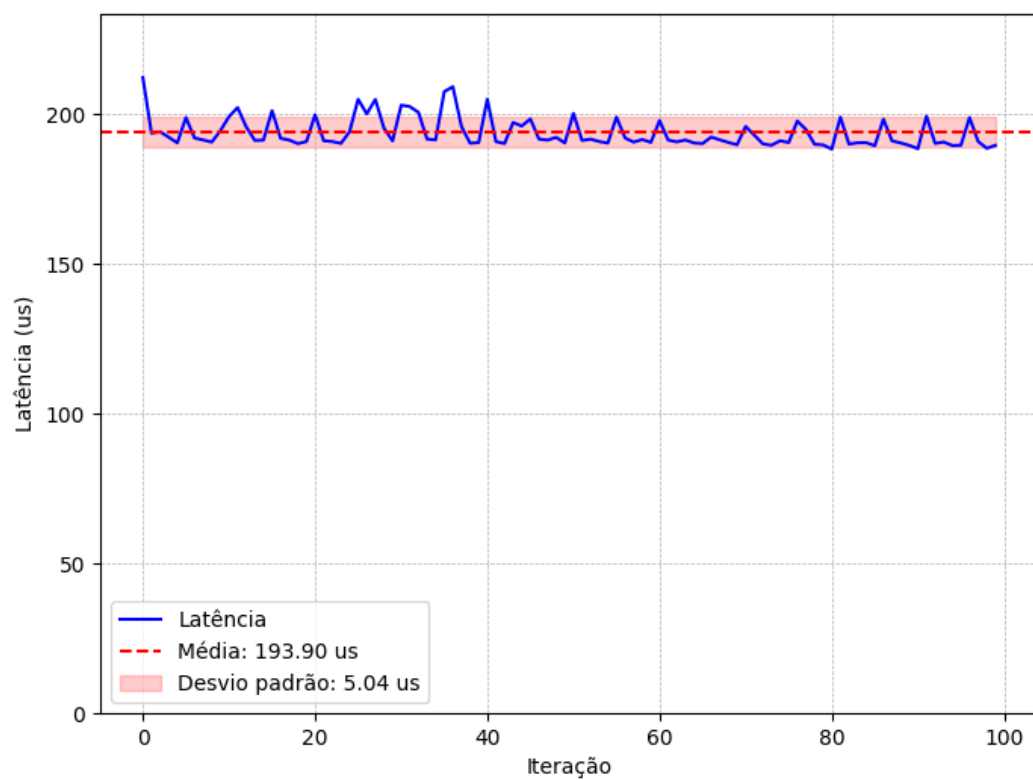
Tabela 8 – Desempenho e Potência para Inferência na GPU.

<i>Dataset</i>	Acurácia (%)	Latência (μs)	Vazão (fps)	Potência (W)
MNIST	98.64	193,90	500.000	107
CIFAR-10	70.32	312,80	100.000	120
SAT-6	97.69	151,37	200.000	60

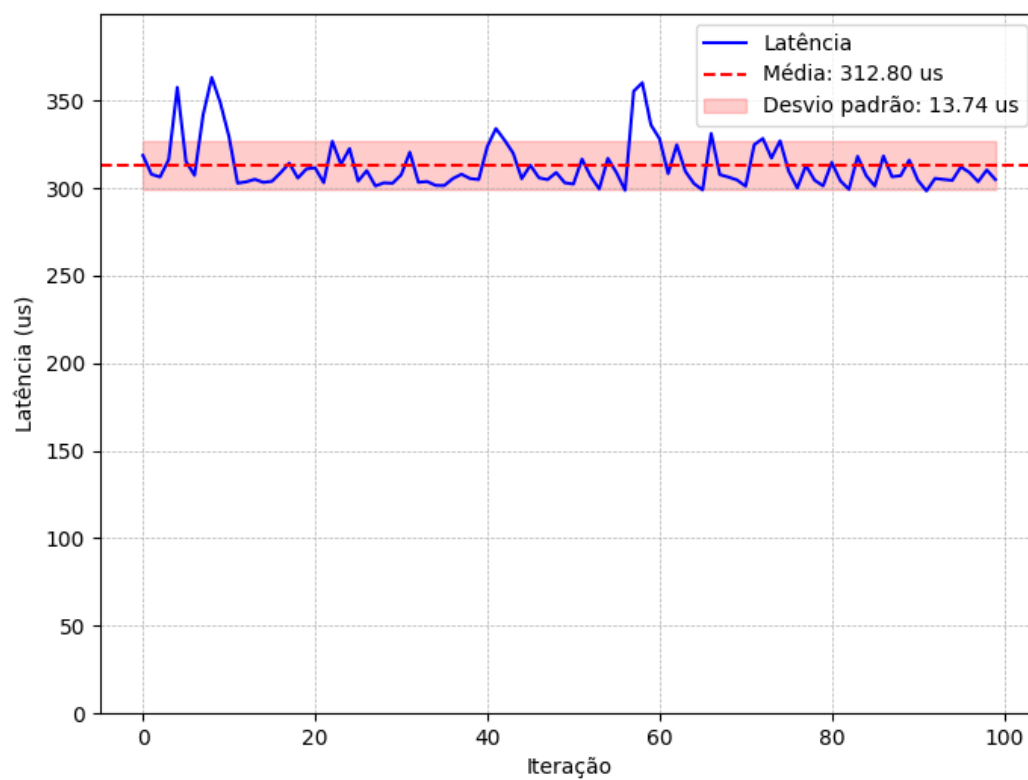
Nas subseções seguintes, os dados obtidos são analisados. Será discutido ainda como o tamanho de lote da inferência influencia o desempenho da rede, além da existência da variação de latência devido a sobrecargas iniciais.

7.2.1 Latência

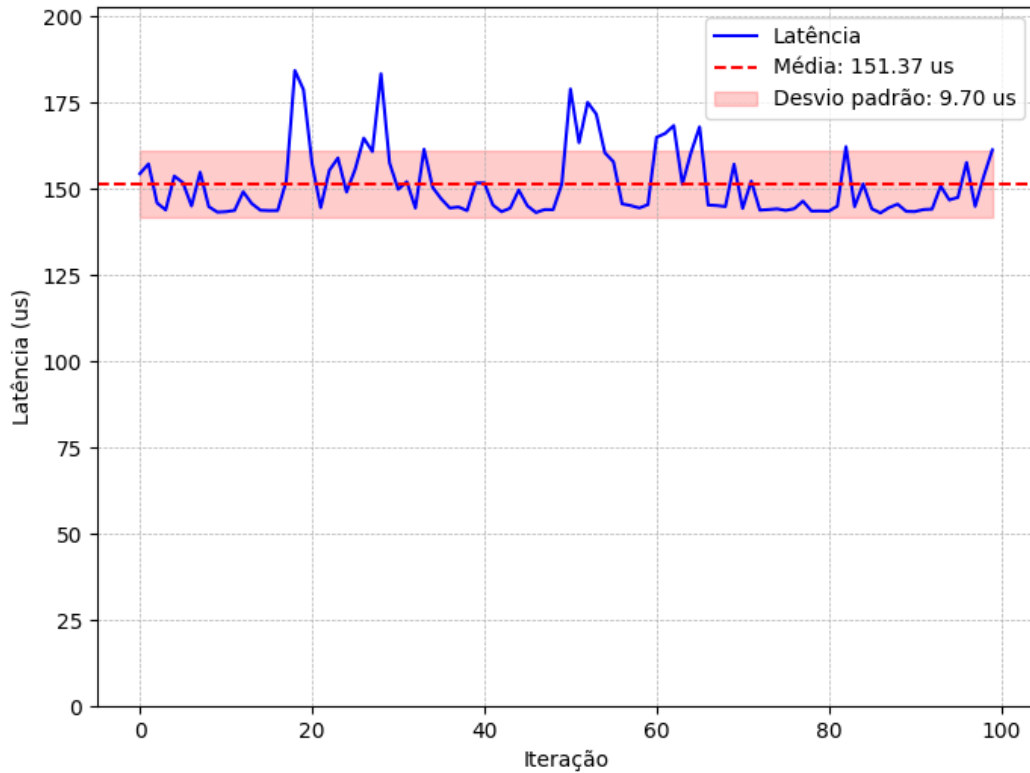
Os resultados para as latências obtidas na GPU são apresentados nas Figuras 28, 29 e 30. As médias de latência obtidas na GPU foram de 193,9 μ s, 312,80 μ s e 151,37 μ s para os respectivos *datasets* MNIST, CIFAR-10 e SAT-6. Esses valores são superiores aos observados na execução na CPU, o que é esperado em cenários com volumes de dados relativamente baixos. A explicação para esse comportamento pode estar associado ao custo computacional causado pela transferência de dados entre a memória da CPU e a memória da GPU. Para lotes pequenos, esse custo frequentemente supera os ganhos obtidos pela execução paralela das operações na GPU, resultando em um desempenho inferior ao da CPU.

Figura 28 – Latência por iterações para o *dataset* MNIST na GPU.

Fonte: o Autor, 2024.

Figura 29 – Latência por iterações para o *dataset* CIFAR na GPU.

Fonte: o Autor, 2024.

Figura 30 – Latência por iterações para o *dataset* SAT-6 na GPU.

Fonte: o Autor, 2024.

Adicionalmente, embora a GPU tenha um tempo de processamento mais determinístico em comparação à CPU, ainda é possível observar variações significativas na latência medida. Essas podem ocorrer porque o tempo de execução registrado pela CPU, inclui a transferência de dados, a qual continua sujeita às interrupções do sistema operacional.

Ainda mais, outro ponto a ser considerado é que o tempo de execução na primeira iteração foi extremamente elevado em comparação com as inferências seguintes. Foram observadas latências quase 100 vezes maiores nas inferências da primeira iteração. Isso se deve à processos iniciais, como a alocação de memória na GPU, a inicialização de bibliotecas e a transferência inicial de dados, que introduzem uma sobrecarga considerável.

Além disso, a biblioteca ONNX Runtime realiza compilações em tempo de execução para otimizar o desempenho do modelo. Esse processo ocorre nas primeiras execuções, pois, inicialmente, a biblioteca não dispõe de informações detalhadas sobre o modelo carregado. Durante essas execuções iniciais, o ONNX Runtime analisa a estrutura do modelo e gera otimizações específicas, como fusões de operadores e alocação eficiente de memória, resultando em um desempenho aprimorado nas inferências subsequentes.

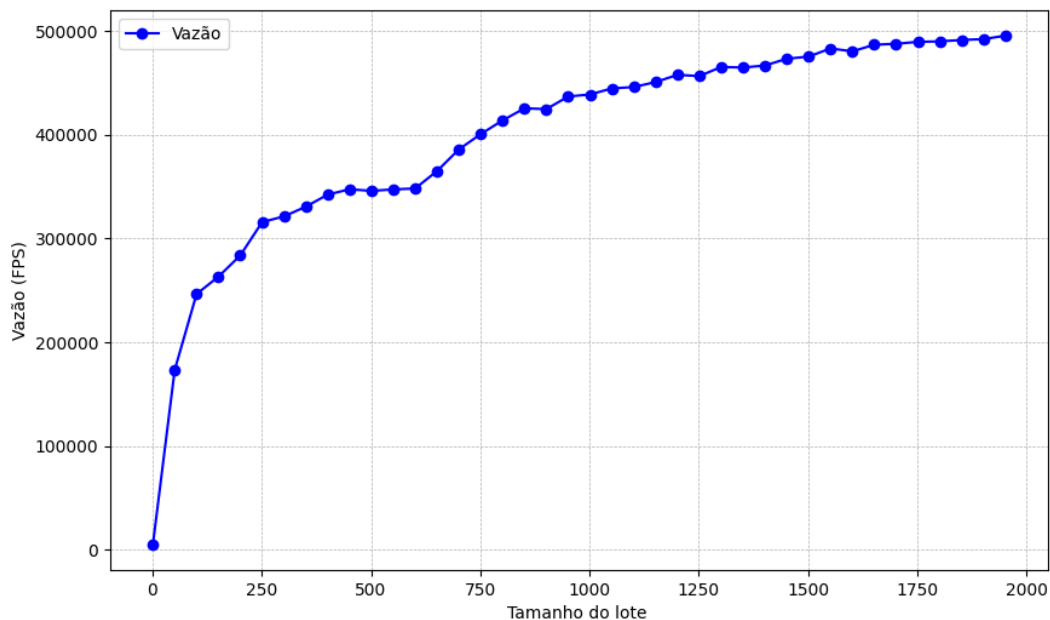
Para mitigar esse efeito e garantir medições mais consistentes, foi realizado um "aquecimento" da rede, no qual várias iterações preliminares foram executadas antes de coletar os dados de latência. Esse procedimento resultou em valores mais homogêneos e representativos do desempenho contínuo.

No entanto, é importante destacar que essa alta latência inicial, embora não evidenciada nos gráficos, pode ser problemática em aplicações onde a primeira inferência é crítica, como em sistemas de resposta em tempo real ou inicializações frequentes. Em tais casos, estratégias como pré-carregamento de modelos, uso de lotes iniciais menores ou otimizações específicas para a primeira execução podem ser necessárias para minimizar o impacto e garantir a eficiência do sistema. Esses resultados reforçam a necessidade de considerar não apenas a latência média, mas também os cenários extremos e suas implicações práticas no design de sistemas.

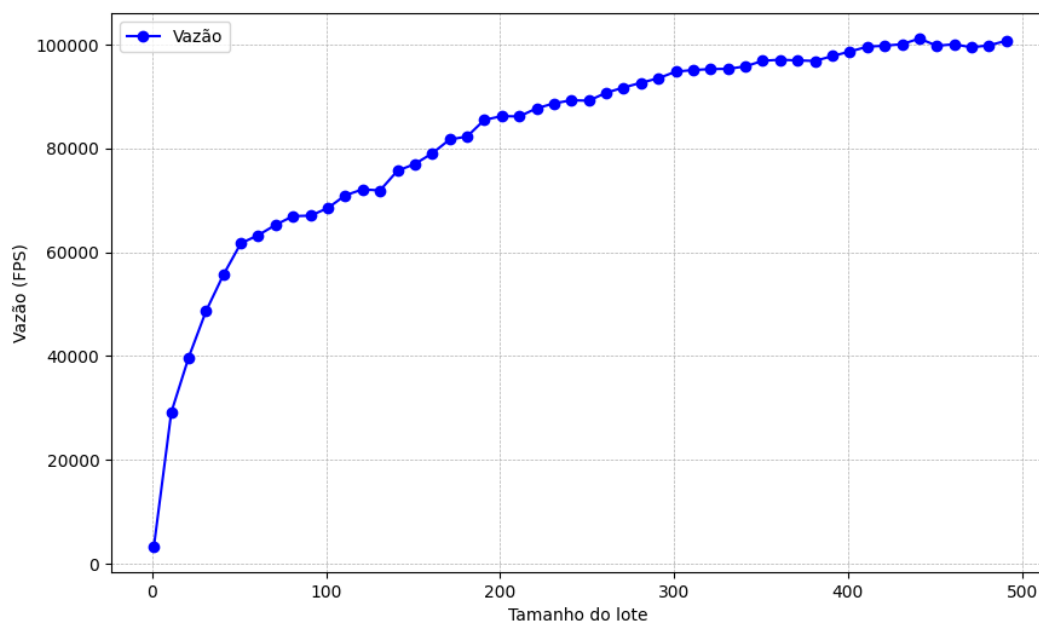
7.2.2 Vazão

As Figuras 31, 32 e 33 apresentam como o tamanho de lote influencia o desempenho em termos de vazão na GPU.

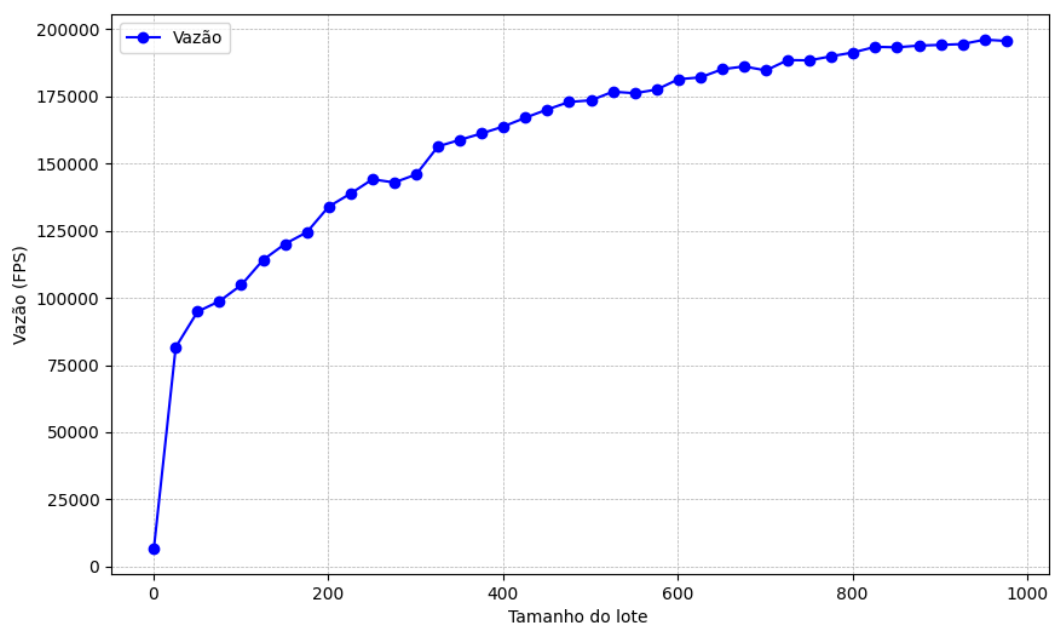
Figura 31 – Vazão por tamanho de lote para o *dataset* MNIST na GPU.



Fonte: o Autor, 2024.

Figura 32 – Vazão por tamanho de lote para o *dataset* CIFAR-10 na GPU.

Fonte: o Autor, 2024.

Figura 33 – Vazão por tamanho de lote para o *dataset* SAT-6 na GPU.

Fonte: o Autor, 2024.

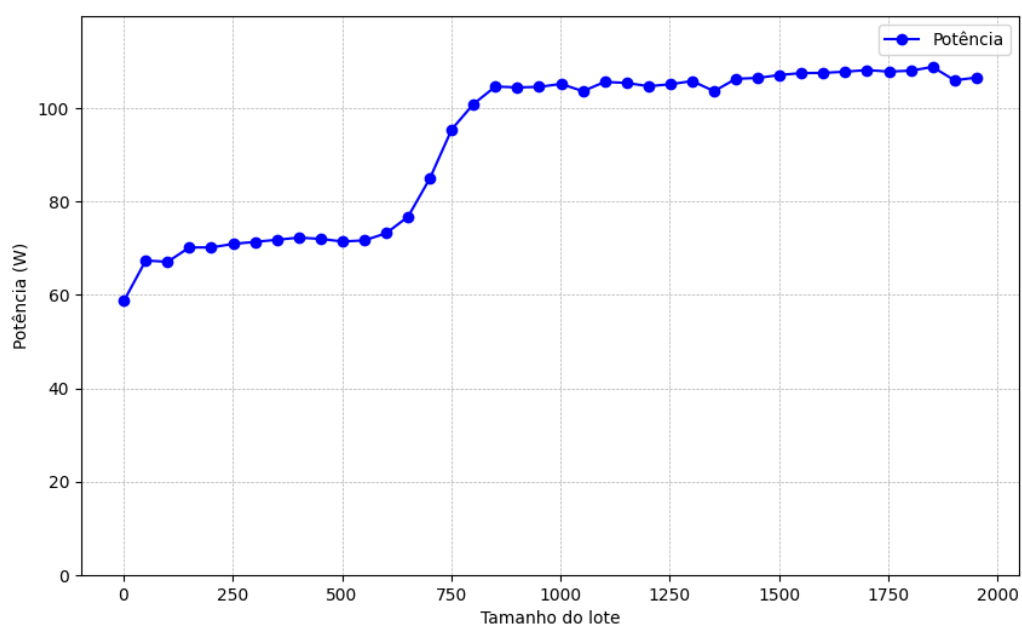
Pelos gráficos das Figuras 31, 32 e 33 observa-se vazões que atingem valores de 500.000 fps para o MNIST, de 100.000 fps para o CIFAR-10 e quase 200.000 fps para o SAT-6. Assim, observa-se um comportamento distinto e valores maiores de vazão em comparação com a CPU

A vazão nos gráficos aumenta conforme o tamanho do lote cresce, e as taxas de incremento tornam-se progressivamente menores à medida que o lote atinge tamanhos maiores. Esse comportamento pode ser explicado pela arquitetura paralela da GPU, que processa múltiplas operações simultaneamente. Inicialmente, o aumento do tamanho do lote permite que mais unidades de processamento da GPU sejam utilizadas de forma eficiente, resultando em um aumento significativo na vazão. No entanto, à medida que o tamanho do lote continua a crescer, a GPU atinge a saturação de seus recursos, como núcleos de processamento e capacidade de memória, fazendo com que o impacto de lotes maiores sobre a vazão diminua.

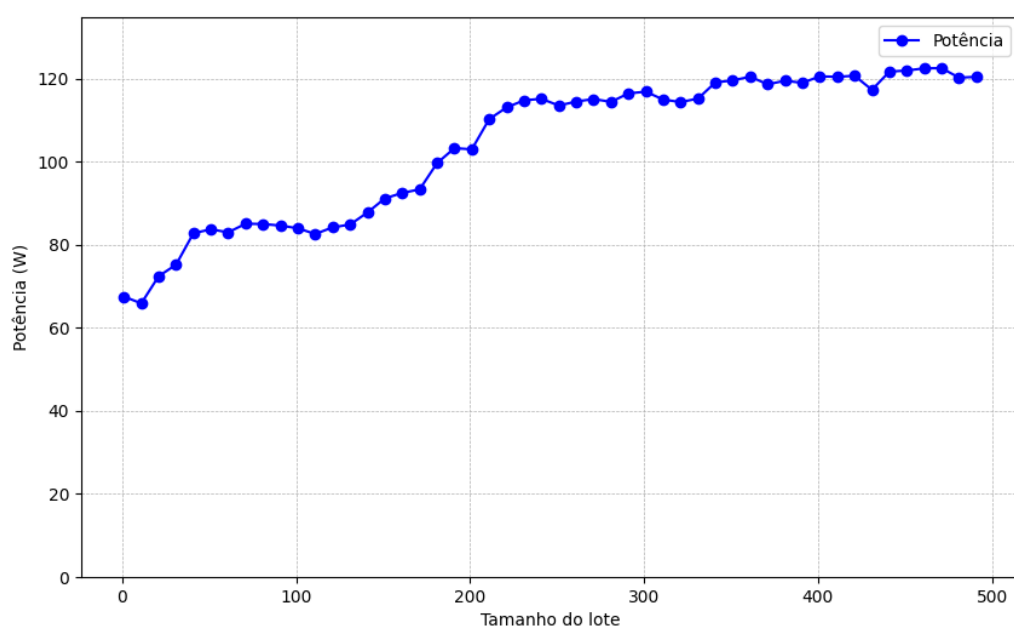
Além disso, nota-se uma sutil quebra desse padrão em determinados pontos, onde a vazão volta a apresentar um crescimento mais acentuado. Esse comportamento pode ser atribuído ao uso otimizado de memória e pipelines internos da GPU. Quando o tamanho do lote atinge um ponto crítico, os mecanismos de gerenciamento de memória da GPU, como cache ou memória compartilhada, podem ajustar a alocação para lidar de forma mais eficiente com os dados. Além disso, é possível que a biblioteca ONNX Runtime aplique otimizações no modelo como a fusões de operações ou ajustes automáticos no pipeline de execução, o que melhora temporariamente a eficiência de processamento.

7.2.3 Dissipação de Potência

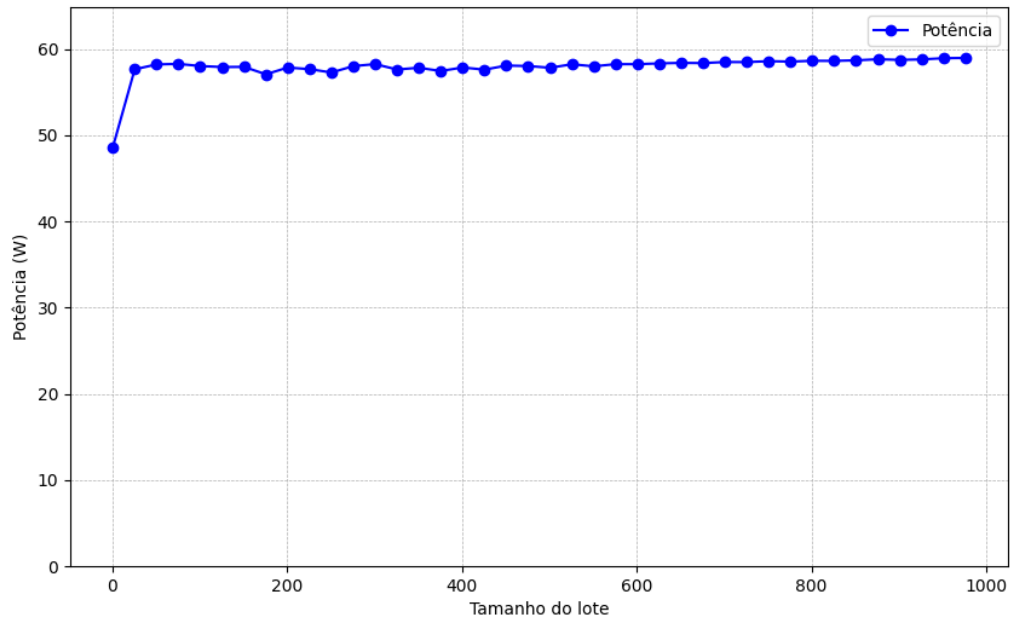
As Figuras 34, 35 e 36 mostram os dados obtidos de dissipação de potência na GPU. A potência observada pela GPU durante as inferências foi consideravelmente maior em comparação à observada na CPU. Os valores de potência máximos registrados para a GPU foram de 107W para o *dataset* MNIST, 120W para o CIFAR-10 e 60W para o SAT-6.

Figura 34 – Dissipação de potência para o *dataset* MNIST na GPU.

Fonte: o Autor, 2024.

Figura 35 – Dissipação de potência para o *dataset* CIFAR-10 na GPU.

Fonte: o Autor, 2024.

Figura 36 – Dissipação de potência para o *dataset* SAT-6 na GPU.

Fonte: o Autor, 2024.

Para os *datasets* MNIST e CIFAR-10, observa-se que, para tamanhos de lote pequenos, a dissipação de potência é ligeiramente menor. Esse comportamento pode ser atribuído à subutilização dos recursos da GPU em lotes pequenos, onde nem todos os núcleos e pipelines paralelos estão sendo explorados de forma eficiente. Além disso, muitas GPUs modernas são otimizadas para trabalhar com tamanhos de dados específicos, o que significa que, em lotes muito pequenos, o hardware ajusta sua potência para economizar energia, resultando em uma operação menos intensiva.

Esse comportamento também pode estar relacionado à sobrecarga inicial associado à alocação de recursos e à inicialização do modelo. Para tamanhos de lote reduzidos, a quantidade de trabalho processada por ciclo de execução é menor, e a GPU pode ajustar sua potência dinamicamente para economizar energia. Entretanto, à medida que o tamanho do lote aumenta, mais recursos são ativados, levando a um aumento proporcional na dissipação de potência até atingir a capacidade máxima.

Vale notar que a topologia utilizada para a classificação do SAT-6 obteve uma dissipação de potência maior na CPU e uma dissipação menor na GPU em comparação aos outros *datasets*. Isso pode ser explicado pelo fato de se tratar de um modelo pequeno em comparação com outros modelos, porém, possui uma quantidade maior de dados para serem transferidos. Dessa forma, como a GPU "perde" tempo na transferência dos dados para realizar poucas operações, acaba por não ser bem aproveitada, ao mesmo tempo que

na CPU essa transferência não é necessária, permanecendo um tempo maior na execução das operações.

7.3 Desempenho no FPGA

A Tabela 9 sintetiza os dados de desempenho, incluindo acurácia, latência, vazão e dissipação de potência.

Tabela 9 – Desempenho e Potência para Inferência em FPGA.

<i>Dataset</i>	Acurácia (%)	Latência (us)	Vazão (fps)	Potência (W)
MNIST	98,67	30,37	98.546	2,113
CIFAR-10	69.73	479,07	4.929	2,770
SAT-6	97,48	24,82	65.761	1,737

As topologias implementadas no FPGA correspondem exclusivamente às versões quantizadas, e, portanto, os dados apresentados nesta seção refletem essas configurações. Nas seções seguintes, serão discutidos os resultados obtidos com a execução no FPGA, destacando os aspectos de desempenho e eficiência.

7.3.1 Latência

As latência obtidas no FPGA foram de 30,37 μ s, 479,07 μ s e 24,82 μ s para os respectivos *datasets* MNIST, CIFAR-10 e SAT-6.

A latência para o MNIST foi menor do que as latências obtidas em todas as outras plataformas, destacando sua eficiência para redes de baixa complexidade. Para o *dataset* SAT-6, a latência foi menor que na GPU, mas ligeiramente maior que na CPU. Já para o *dataset* CIFAR-10, a latência no FPGA foi superior à registrada na CPU e na GPU, devido à maior complexidade da rede neural e às limitações de recursos do FPGA.

7.3.2 Vazão

As vazões obtidas no FPGA foram de 98.546 fps, 4.929 fps e 65.761 fps nos *datasets* MNIST, CIFAR-10 e SAT-6, respectivamente.

Em comparação com as outras plataformas, os resultados mostram que a vazão depende significativamente do tamanho do lote utilizado na CPU e GPU. As plataformas de propósito geral, ao processarem lotes maiores, podem superar facilmente a vazão do FPGA. No entanto, em cenários de inferência com lotes pequenos, como apenas uma entrada por vez, o FPGA supera as outras plataformas, exceto no *dataset* CIFAR-10. Esse comportamento para o CIFAR-10 pode ser atribuído à maior complexidade do modelo e às restrições de recursos do FPGA disponível, além de limitações na alocação de recursos geradas pelo processo de síntese no FINN.

Além disso, as limitações de desempenho observadas, especialmente nas redes para CIFAR-10 e SAT-6, podem estar relacionadas a restrições de recursos do FPGA e à distribuição subótima inferida pelo FINN durante a síntese. Devido a indisponibilidade de um recurso específico no hardware, pode ter sido necessário reduzir a vazão alvo da compilação para viabilizar a implementação. Um ajuste como esse impactaria negativamente o desempenho da rede, especialmente em termos de latência e vazão.

7.3.3 Dissipação de potência

As potências estimadas pelo Vivado para a implementação no FPGA foram de 2,1W, 2,77W e 1,7W, para os respectivos *datasets* MNIST, CIFAR-10 e SAT-6.

Assim, o FPGA demonstrou uma vantagem clara, com valores significativamente menores em comparação às outras plataformas para todos os *datasets* avaliados. Essa eficiência energética torna o FPGA uma ótima solução para aplicações com restrições de energia, como dispositivos IoT e sistemas embarcados.

7.3.4 Utilização de Recursos

São apresentados nas Tabelas 10, 11 e 12, o uso de recursos como LUTs, registradores, blocos de memória e DSPs para as topologias avaliadas.

Tabela 10 – Utilização de Recursos no FPGA para Classificação do MNIST.

Recurso	Quantidade Utilizada	Percentual de Utilização
LUTs	35.001 de 53.200	65,8%
Registradores	34.805 de 106.400	32,7%
Block RAM	26 de 140	18,6%
DSPs	2 de 220	0,9%

Tabela 11 – Utilização de Recursos no FPGA para Classificação do CIFAR-10.

Recurso	Quantidade Utilizada	Percentual de Utilização
LUTs	40.071 de 53.200	75,3%
Registradores	51.358 de 106.400	48,3%
Block RAM	74,5 de 140	53,2%
DSPs	1 de 220	0,4%

Tabela 12 – Utilização de Recursos no FPGA para Classificação do SAT-6.

Recurso	Quantidade Utilizada	Percentual de Utilização
LUTs	40.833 de 53.200	76,7%
Registradores	25.299 de 106.400	23,8%
Block RAM	31 de 140	22,1%
DSPs	128 de 220	58,2%

Os dados apresentados nas Tabelas 10, 11 e 12 demonstram a utilização dos recursos do FPGA, como LUTs, registradores, blocos de memória e DSPs, para cada uma das topologias avaliadas.

No caso da rede destinada à classificação do MNIST, a utilização de LUTs foi de 65,8%, enquanto registradores e blocos de memória apresentaram ocupações de 32,7% e 18,6%, respectivamente. O uso de DSPs foi mínimo, atingindo apenas 0,9%. A topologia para CIFAR-10 apresentou a maior ocupação de LUTs, com 75,3%, e registradores, com 48,3%. Além disso, os blocos de memória atingiram uma utilização de 53,2%, mas o uso de DSPs permaneceu baixo, com apenas 0,4%. Para a classificação do SAT-6, observou-se a maior ocupação de DSPs, com 58,2%, enquanto os demais recursos apresentaram utilizações de 76,7% para LUTs, 23,8% para registradores e 22,1% para blocos de memória.

Embora as redes tenham demonstrado variações no uso de recursos, é evidente que a alocação global ainda está aquém do ideal. Nenhuma das redes utilizou todos os recursos disponíveis de maneira equilibrada, deixando espaço para otimizações. Particularmente, o baixo uso de DSPs nas redes MNIST e CIFAR-10 destaca uma subutilização de recursos críticos para operações intensivas em processamento.

Durante o desenvolvimento, tentou-se forçar o uso de DSPs em determinadas camadas da rede, mas essa abordagem resultou em uma alocação excessiva e inviável, exigindo que a distribuição de recursos fosse delegada ao FINN de forma automática. Apesar de funcional, essa solução não maximizou a eficiência da utilização do hardware.

Esse cenário subótimo de utilização indica que, por meio de estratégias mais avançadas de otimização, é possível melhorar significativamente o desempenho das redes neurais, maximizando o uso de todos os recursos disponíveis na FPGA alvo deste trabalho. Entre as possíveis melhorias está o ajuste dos parâmetros de quantização para torná-los compatíveis com as capacidades específicas dos blocos de hardware, como DSPs e LUTs, de modo a explorar ao máximo o potencial do dispositivo. Outra possibilidade é o refinamento das definições de *folding*, de maneira a equilibrar melhor o uso dos recursos do FPGA, garantindo uma implementação mais eficaz tanto em termos de desempenho quanto de consumo de recursos.

Por outro lado, o baixo consumo de recursos pode ser interpretado como um aspecto positivo, pois permite a alocação de espaço adicional para a integração de outros sistemas no mesmo FPGA.

7.3.5 Modelos ONNX

As Figuras 37, 38, 39, 40, 41 e 42 presentes no Apêndice A.1 ilustram os modelos de precisão total convertidos para o formato ONNX e as versões mapeadas para camadas abstratas de hardware implementadas no FINN, para os *datasets* MNIST, CIFAR-10

e SAT-6. Essas representações visuais fornecem uma exemplificação de como o FINN transforma a rede para sua mapeação para hardware. Essas representações foram obtidas dos modelos convertidos para o formato ONNX e visualizadas através do Netron.

7.4 Comparação Final entre as Plataformas

Com base em todos os resultados apresentados para CPU, GPU e FPGA, pode-se estabelecer um panorama geral acerca do desempenho e da eficiência energética de cada plataforma na execução das redes neurais avaliadas nos *datasets* MNIST, CIFAR-10 e SAT-6. As Tabelas 13, 14, 15 e 16 apresentam as principais métricas de desempenho — acurácia, latência, vazão e dissipação de potência —, organizadas de maneira a facilitar a comparação e destacar as vantagens e limitações de cada plataforma.

Tabela 13 – Acurácia (%) por plataforma e *dataset*

Plataforma	MNIST	CIFAR-10	SAT-6
CPU	98,64	70,32	97,69
GPU	98,64	70,32	97,69
FPGA	98,67	69,73	97,48

Tabela 14 – Latência (μ s) por plataforma e *dataset*

Plataforma	MNIST	CIFAR-10	SAT-6
CPU	28,43	93,14	17,35
GPU	193,90	312,80	151,37
FPGA	30,37	479,07	24,82

Tabela 15 – Vazão máxima (fps) por plataforma e *dataset*

Plataforma	MNIST	CIFAR-10	SAT-6
CPU	244.000	20.510	475.000
GPU	500.000	100.000	200.000
FPGA	98.546	4.929	65.761

Tabela 16 – Potência (W) por plataforma e *dataset*

Plataforma	MNIST	CIFAR-10	SAT-6
CPU	30,00	36,00	57,00
GPU	107,00	120,00	60,00
FPGA	2,113	2,770	1,737

É possível observar que a **acurácia** no FPGA apresentou uma pequena variação em relação ao modelo de precisão total devido ao processo de quantização do modelo. Para o CIFAR-10 e o SAT-6, a queda de acurácia foi menor do que 1%, enquanto, para o

dataset MNIST, a acurácia foi praticamente a mesma, apresentando até mesmo um ligeiro aumento. Isso evidencia que a quantização consciente de treinamento é uma abordagem eficaz redução da complexidade dos modelos, permitindo um desempenho próximo ao original, mesmo em hardware especializado como FPGAs.

Já em termos de **latência**, a CPU apresentou os melhores resultados estando bastante próximos dos resultados obtidos no FPGA. A GPU, no entanto, obteve, de longe, o pior desempenho nesse quesito sendo penalizada pelo custo de transferência de dados.

Quanto à **vazão**, a GPU alcançou os maiores valores em todos os *datasets*, ao utilizar tamanhos de lote maiores, beneficiando-se de seu processamento altamente paralelizado, no entanto, para valores pequenos de lotes a vazão foi extremamente baixa. A CPU manteve vazões consideráveis em lotes de tamanho moderado, porém sensível ao *thermal throttling* em cargas elevadas. Já a vazão obtida no FPGA foi bastante inferior à melhor vazão obtida nas outras plataformas. No entanto, é importante ressaltar que para atingir a vazão apresentada, os tamanhos dos lotes utilizados nas plataformas CPU e GPU tiveram que ser aumentados, aumentando consideravelmente a latência também. Assim, para valores pequenos de lote, a vazão no FPGA, supera facilmente a vazão nas outras plataformas.

Por fim, no quesito **dissipação de potência**, o FPGA evidenciou uma vantagem notável, apresentando o menor consumo de todas as plataformas testadas. Essa característica o torna uma opção particularmente interessante para cenários com restrição energética, como dispositivos embarcados ou aplicações que demandam operação contínua com autonomia de bateria.

8 Conclusões

Este trabalho investigou o desempenho da inferência de três redes neurais voltadas para a classificação de imagens em diferentes arquiteturas de hardware: CPU, GPU e FPGA. Foram utilizadas três bases de dados (MNIST, CIFAR-10 e SAT-6) e redes com diferentes configurações: uma versão em precisão total, empregada na CPU e GPU, e uma versão quantizada, necessária para a implementação no FPGA através do fluxo FINN. As métricas de avaliação adotadas (acurácia, latência, vazão e consumo de energia) possibilitaram uma análise abrangente dos aspectos de desempenho de cada abordagem.

A acurácia obtida nos modelos de precisão total foi satisfatória, atingindo valores de até 98,64% para o *dataset* MNIST e 70% para o CIFAR-10. Embora a acurácia no CIFAR-10 seja inferior à dos outros *datasets*, ela está alinhada com o desempenho esperado da topologia utilizada. Além disso, destaca-se a efetividade do processo de quantização, que resultou em acurácias muito próximas às dos modelos de precisão total, demonstrando a robustez dessa técnica.

Ainda mais, os experimentos realizados evidenciaram que o FPGA é altamente eficiente para inferência de redes neurais em tarefas com lotes pequenos, onde apresenta baixa latência e alta vazão e previsibilidade. Contudo, restrições de recursos e limitações do fluxo de síntese do FINN impactaram o desempenho em redes mais complexas, como a utilizada para o CIFAR-10. Em contrapartida, CPUs e GPUs demonstraram maior flexibilidade e desempenho superior em cenários de lotes grandes, tirando proveito de arquiteturas otimizadas para operações em paralelo e maior disponibilidade de recursos computacionais.

Contudo, alguns desafios surgiram ao longo do fluxo de desenvolvimento e implementação no FPGA. A curva de aprendizado do FINN e suas limitações em termos de alocação de recursos e *folding* podem restringir a adoção de redes mais complexas. Além disso, a disponibilidade e o correto aproveitamento de blocos DSP e BRAM na placa PYNQ Z1 podem ser fatores cruciais para se alcançar um melhor desempenho. Além disso, o uso das bibliotecas ONNX Runtime e PyJoules simplificou a execução e medição de consumo energético nas plataformas convencionais (CPU e GPU), evidenciando a flexibilidade de manter um único modelo para diferentes dispositivos.

8.1 Trabalhos Futuros

Com base nos resultados deste trabalho, diversas direções podem ser exploradas em futuras pesquisas para ampliar as contribuições realizadas. Como propostas de trabalhos futuros, destacam-se:

- **Otimizações de Quantização:** Explorar níveis de quantização diferenciados para pesos e ativações em cada camada da rede, por meio das funcionalidades avançadas do Brevitas. Uma quantização mais granular pode contribuir para atingir maiores níveis de acurácia mantendo uma boa acurácia, permitindo, assim, um maior desempenho ou menor utilização de recursos.
- **Otimização de Implementação no FINN:** Outro ponto de interesse é a otimização do fluxo de síntese e mapeamento no FINN. Estratégias como ajustes refinados no *folding* e a exploração mais eficiente dos recursos disponíveis, como blocos DSP e memória BRAM, podem levar a uma melhor utilização do hardware. Além disso, também pode ser relevante avaliar o desempenho de outras ferramentas, para comparar abordagens e identificar a mais adequada para redes específicas.
- **Topologias mais Complexas:** A exploração de arquiteturas de redes neurais mais modernas em conjunto com *datasets* mais desafiadores também pode ser tomada como direção. Redes de última geração, poderiam ser quantizadas e adaptadas para inferência em hardware especializado. Paralelamente, a utilização de *datasets* mais complexos ou voltados para aplicações específicas, como imagens médicas ou dados tridimensionais, permitiria avaliar a escalabilidade das soluções propostas e do FINN.
- **Integração em Ambientes Reais:** Implementar as redes neurais em sistemas embarcados com restrições de energia e recursos de hardware, como dispositivos IoT, para validação prática do desempenho obtido. Aplicações em visão computacional, automação industrial e monitoramento ambiental podem se beneficiar diretamente dessas soluções, ao mesmo tempo em que novos desafios relacionados à integração e operação em ambientes reais podem ser identificados e solucionados.

8.2 Considerações Finais

Este trabalho demonstrou que a aceleração de redes neurais em FPGA é uma solução promissora, especialmente em cenários onde eficiência energética e baixa latência são prioritárias. Ainda que a CPU e a GPU se destaquem pela alta vazão em maiores quantidades de dados e pela praticidade de desenvolvimento, o *trade-off* entre eficiência energética e desempenho sugere fortemente o uso de FPGA em cenários embarcados e de tempo real.

Entretanto, a adoção de fluxos de síntese como o FINN exige uma curva de aprendizado inicial e, muitas vezes, adaptação das redes às limitações de recursos do dispositivo. Do ponto de vista prático, este trabalho consolidou um roteiro para indo desde a aquisição de dados, treinamento, quantização de redes por meio do Brevitas e posterior implementação no FPGA, destacando as principais dificuldades e estratégias adotadas, além de sua análise de desempenho e comparação com plataformas tradicionais como CPU e GPU.

Em suma, espera-se que a experiência e os resultados discutidos contribuam para futuras pesquisas na aceleração de redes neurais em hardware reconfigurável, incentivando o desenvolvimento de metodologias e ferramentas mais maduras e robustas. Dessa forma, será possível ampliar ainda mais a adoção de soluções eficientes em aplicações reais, desde sistemas embarcados de pequeno porte até cenários de computação em larga escala, mantendo a busca contínua pelo melhor equilíbrio entre desempenho e consumo energético.

Referências Bibliográficas

AMD. *Vivado High-Level Design*. 2025. Accessed: 2025-01-05. Disponível em: <<https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado/high-level-design.html>>.

BAEK, E.; KWON, D.; KIM, J. A multi-neural network acceleration architecture. In: *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. [S.l.: s.n.], 2020. p. 940–953.

BASU, S.; GANGULY, S.; MUKHOPADHYAY, S.; DIBIANO, R.; KARKI, M.; NEMANI, R. Deepsat: a learning framework for satellite imagery. In: *Proceedings of the 23rd SIGSPATIAL International Conference on Advances in Geographic Information Systems*. New York, NY, USA: Association for Computing Machinery, 2015. (SIGSPATIAL '15). ISBN 9781450339674. Disponível em: <<https://doi.org/10.1145/2820783.2820816>>.

BISHOP, C. *Pattern Recognition and Machine Learning*. [S.l.]: Journal of Electronic Imaging, 2006. v. 16. 140-155 p.

BLOTT, M.; PREUSSER, T. B.; FRASER, N. J.; GAMBARDILLA, G.; O'BRIEN, K.; UMUROGLU, Y.; LEESER, M.; VISSERS, K. Finn-r: An end-to-end deep-learning framework for fast exploration of quantized neural networks. *ACM Trans. Reconfigurable Technol. Syst.*, Association for Computing Machinery, New York, NY, USA, v. 11, n. 3, dez. 2018. ISSN 1936-7406. Disponível em: <<https://doi.org/10.1145/3242897>>.

BORDIGNON, L. M. *Análise de protocolos de comunicação para a eficiência energética em sistemas IoT*. 2023. Trabalho de Conclusão de Curso (Graduação em Engenharia de Computação) – Universidade Federal de Santa Maria, Santa Maria. Disponível em: <<https://repositorio.ufsm.br/handle/1/31391>>.

CHEN, Y.; XIE, Y.; SONG, L.; CHEN, F.; TANG, T. A survey of accelerator architectures for deep neural networks. *Engineering*, v. 6, n. 3, p. 264–274, 2020. ISSN 2095-8099. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S2095809919306356>>.

DIGILENT. *PYNQ-Z1: Getting Started*. 2025. Acesso em: 5 jan. 2025. Disponível em: <<https://digilent.com/reference/programmable-logic/pynq-z1/start>>.

ELY, A. F.; BENEVENUTI, F.; KOROL, G.; BECK, A. C. S.; AZAMBUJA, J. R.; KASTENSMIDT, F. L. Image classification cnns using the finn engine for sram-based apsoc in satellite applications. In: *2024 37th SBC/SBMicro/IEEE Symposium on Integrated Circuits and Systems Design (SBCCI)*. [S.l.: s.n.], 2024. p. 1–5.

FAROOQ, U.; MARRAKCHI, Z.; MEHREZ, H. *Tree-based Heterogeneous FPGA Architectures: Application Specific Exploration and Optimization*. [S.l.]: Springer Publishing Company, Incorporated, 2012. ISBN 1461435935.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. [S.l.]: MIT Press, 2016. <<http://www.deeplearningbook.org>>.

GUO, K.; ZENG, S.; YU, J.; WANG, Y.; YANG, H. *A Survey of FPGA-Based Neural Network Accelerator*. 2018. Disponível em: <<https://arxiv.org/abs/1712.08934>>.

HAMANAKA, F.; ODAN, T.; KISE, K.; CHU, T. V. An exploration of state-of-the-art automation frameworks for fpga-based dnn acceleration. *IEEE Access*, v. 11, p. 5701–5713, 2023.

Intel Corporation. *Intel Core i5-1335U Processor*. 2025. Acesso em: 5 jan. 2025. Disponível em: <<https://www.intel.com.br/content/www/br/pt/products/sku/232153/intel-core-i51335u-processor-12m-cache-up-to-4-60-ghz/specifications.html>>.

JAJAL, P.; JIANG, W.; TEWARI, A.; KOCINARE, E.; WOO, J.; SARRAF, A.; LU, Y.-H.; THIRUVATHUKAL, G. K.; DAVIS, J. C. Interoperability in deep learning: A user survey and failure analysis of onnx model converters. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. New York, NY, USA: Association for Computing Machinery, 2024. (ISSTA 2024), p. 1466–1478. ISBN 9798400706127. Disponível em: <<https://doi.org/10.1145/3650212.3680374>>.

JOUPPI, N. P.; YOUNG, C.; PATIL, N.; PATTERSON, D.; AGRAWAL, G.; BAJWA, R.; BATES, S.; BHATIA, S.; BODEN, N.; BORCHERS, A. *et al.* In-datacenter performance analysis of a tensor processing unit. In: *Proceedings of the 44th annual international symposium on computer architecture*. [S.l.: s.n.], 2017. p. 1–12.

KAGGLE. *Kaggle: Your Machine Learning and Data Science Community*. 2025. <<https://www.kaggle.com>>. Acessado em: 10 jan. 2025.

KRIZHEVSKY, A. Learning multiple layers of features from tiny images. In: . [s.n.], 2009. Disponível em: <<https://api.semanticscholar.org/CorpusID:18268744>>.

LECUN, Y.; BOTTOU, L.; BENGIO, Y.; HAFFNER, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, v. 86, n. 11, p. 2278–2324, 1998.

LI, Z.; LIU, F.; YANG, W.; PENG, S.; ZHOU, J. A survey of convolutional neural networks: Analysis, applications, and prospects. *IEEE Transactions on Neural Networks and Learning Systems*, v. 33, n. 12, p. 6999–7019, 2022.

LIU, Y.; WANG, Y.; YU, R.; LI, M.; SHARMA, V.; WANG, Y. *Optimizing CNN Model Inference on CPUs*. 2019. Disponível em: <<https://arxiv.org/abs/1809.02697>>.

MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, v. 5, n. 4, p. 115–133, 1943. ISSN 1522-9602. Disponível em: <<https://doi.org/10.1007/BF02478259>>.

MITTAL, S. A survey of fpga-based accelerators for convolutional neural networks. *Neural Computing and Applications*, v. 32, n. 4, p. 1109–1139, February 2020. ISSN 1433-3058. Disponível em: <<https://doi.org/10.1007/s00521-018-3761-1>>.

MOHAIDAT, T.; KHALIL, K. A survey on neural network hardware accelerators. *IEEE Transactions on Artificial Intelligence*, v. 5, n. 8, p. 3801–3822, 2024.

NAGATA, F.; SAKATA, S.; ABE, R.; WATANABE, K.; HABIB, M. K. Evaluation of interoperability of cnn models between matlab and python environments using onnx runtime model. *AI, Computer Science and Robotics Technology*, IntechOpen, Dec 2024. Disponível em: <<https://doi.org/10.5772/acrt.20240043>>.

NAGEL, M.; BAALEN, M. van; BLANKEVOORT, T.; WELLING, M. Data-free quantization through weight equalization and bias correction. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, IEEE, p. 1325–1334, 2021. Disponível em: <<https://arxiv.org/abs/2006.10518>>.

NAVEEN, S.; KOUNTE, M. R. Optimized convolutional neural network at the iot edge for image detection using pruning and quantization. *Multimedia Tools and Applications*, December 2024. ISSN 1573-7721. Disponível em: <<https://doi.org/10.1007/s11042-024-20523-1>>.

NECHI, A.; GROTH, L.; MULHEM, S.; MERCHANT, F.; BUCHTY, R.; BEREKOVIC, M. Fpga-based deep learning inference accelerators: Where are we standing? *ACM Trans. Reconfigurable Technol. Syst.*, Association for Computing Machinery, New York, NY, USA, v. 16, n. 4, out. 2023. ISSN 1936-7406. Disponível em: <<https://doi.org/10.1145/3613963>>.

NOUMI, T.; CONTRIBUTORS. *tiny-dnn: header only, dependency-free deep learning framework in C++14*. 2016. <<https://github.com/tiny-dnn/tiny-dnn>>. Accessed: 2024-10-11.

NVIDIA CORPORATION. *Deep Learning Performance Guide: GPU Architecture Background*. [S.l.], 2023. Acesso em: 4 jan. 2025. Disponível em: <<https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html#gpu-arch>>.

NVIDIA Corporation. *Pascal GPU Architecture*. 2025. Acesso em: 5 jan. 2025. Disponível em: <<https://www.nvidia.com/en-us/data-center/pascal-gpu-architecture/>>.

PAPPALARDO, A. *Xilinx/brevitas*. Zenodo, 2024. Disponível em: <<https://doi.org/10.5281/zenodo.3333552>>.

PETER, P. *How Fast is the new NVIDIA Titan X (Pascal) Graphics Card?* ThePCEnthusiast, 2016. Accessed: 2024-12-26. Disponível em: <<https://thepecenthusiast.com/how-fast-nvidia-titan-x-pascal-graphics-card/>>.

PLAGWITZ, P.; HANNIG, F.; STRÖBEL, M.; STROHMEYER, C.; TEICH, J. A safari through fpga-based neural network compilation and design automation flows. In: *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. [S.l.: s.n.], 2021. p. 10–19.

RIOS, R. M. *Técnicas energeticamente eficientes para o posicionamento de aplicações em computação na borda*. Dissertação (Dissertação de Mestrado) — Pontifícia Universidade Católica do Rio Grande do Sul, Porto Alegre, 2022. Disponível em: <<https://repositorio.pucrs.br/dspace/handle/10923/25784>>.

ROEDER, L. *Netron: Visualizer for neural network, deep learning and machine learning models*. [S.l.], 2025. Acesso em: 4 jan. 2025. Disponível em: <<https://netron.app>>.

ROSENBLATT, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, American Psychological Association, v. 65, p. 386–408, 1958.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, n. 6088, p. 533–536, 1986. ISSN 1476-4687. Disponível em: <<https://doi.org/10.1038/323533a0>>.

SAKR, C.; DAI, S.; VENKATESAN, R.; ZIMMER, B.; DALLY, W. J.; KHAILANY, B. *Optimal Clipping and Magnitude-aware Differentiation for Improved Quantization-aware Training*. 2022. Disponível em: <<https://arxiv.org/abs/2206.06501>>.

STALLINGS, W. *Computer Organization and Architecture: Designing for Performance*. 10th. ed. [S.l.]: Pearson Education, 2016. Available in both print and digital formats. ISBN 9780134101613.

SZE, V.; CHEN, Y.-H.; YANG, T.-J.; EMER, J. S. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, Ieee, v. 105, n. 12, p. 2295–2329, 2017.

UMUROGLU, Y.; FRASER, N. J.; GAMBARDELLA, G.; BLOTT, M.; LEONG, P.; JAHRE, M.; VISSERS, K. Finn: A framework for fast, scalable binarized neural network inference. In: *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. New York, NY, USA: Association for Computing Machinery, 2017. (FPGA '17), p. 65–74. ISBN 9781450343541. Disponível em: <<https://doi.org/10.1145/3020078.3021744>>.

UMUROGLU, Y.; JAHRE, M. *Streamlined Deployment for Quantized Neural Networks*. 2018. Disponível em: <<https://arxiv.org/abs/1709.04060>>.

WANG, C.; LUO, Z. A review of the optimal design of neural networks based on fpga. *Applied Sciences*, v. 12, n. 21, 2022. ISSN 2076-3417. Disponível em: <<https://www.mdpi.com/2076-3417/12/21/10771>>.

WEI, X.; GONG, R.; LI, Y.; LIU, X.; YU, F. *QDrop: Randomly Dropping Quantization for Extremely Low-bit Post-Training Quantization*. 2023. Disponível em: <<https://arxiv.org/abs/2203.05740>>.

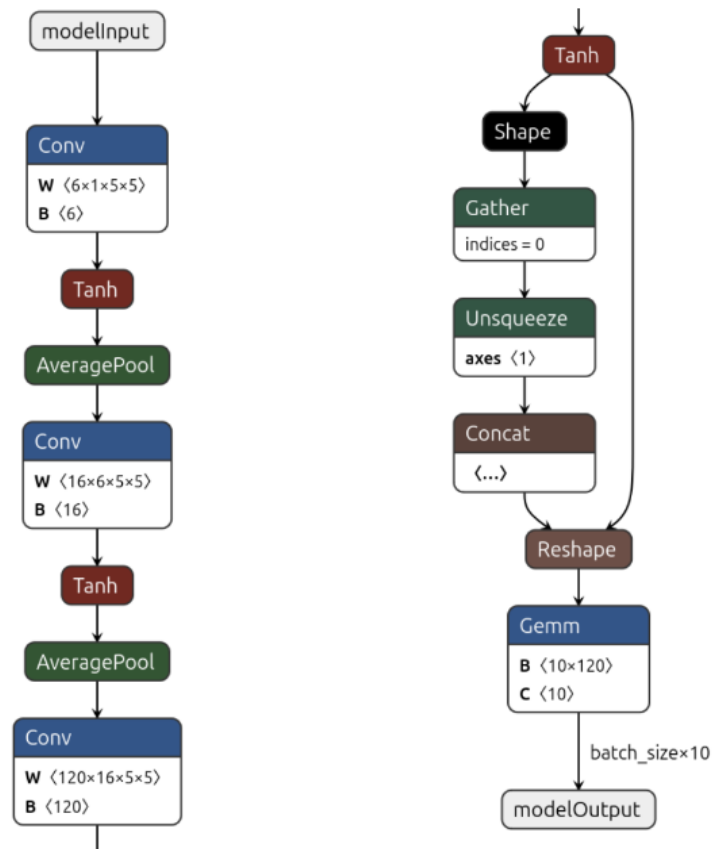
XILINX. *FINN End-to-End Flow*. [S.l.], 2025. Acesso em: 7 out. 2024. Disponível em: <https://finn.readthedocs.io/en/latest/end_to_end_flow.html>.

Apêndices

APÊNDICE A – Modelos ONNX

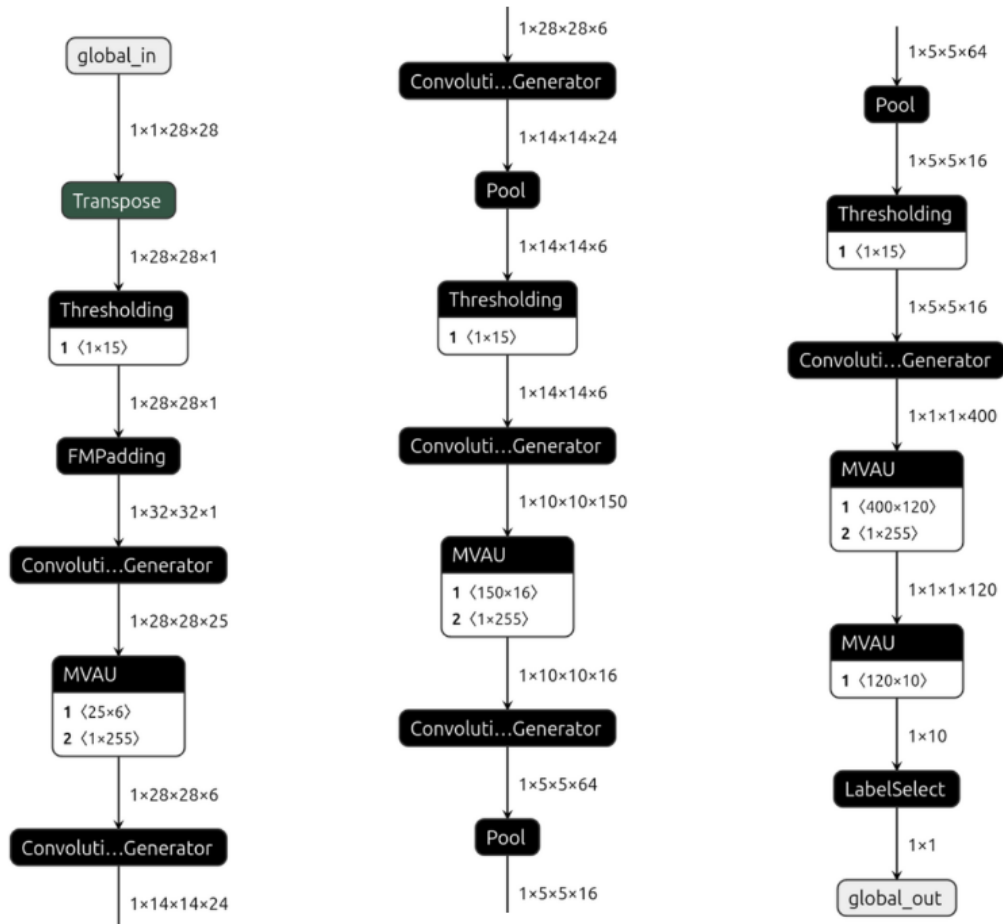
A.1 Modelos para o *dataset* MNIST

Figura 37 – Modelo de precisão total ONNX para o *dataset* MNIST.



Fonte: o Autor, 2024.

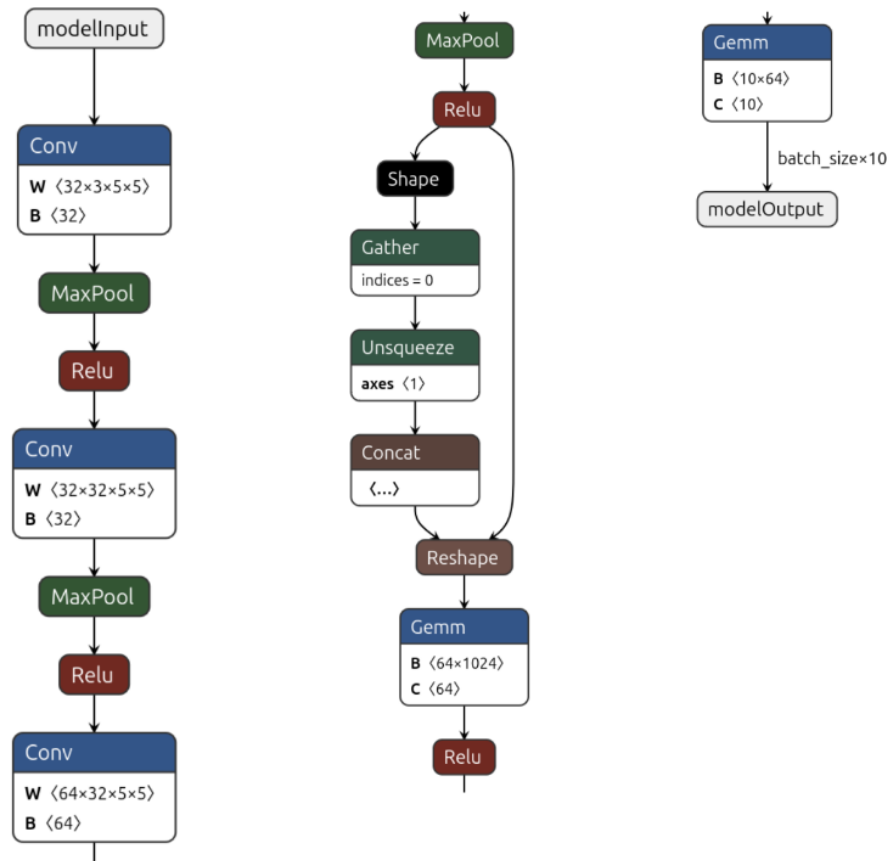
Figura 38 – Modelo FINN para o dataset MNIST.



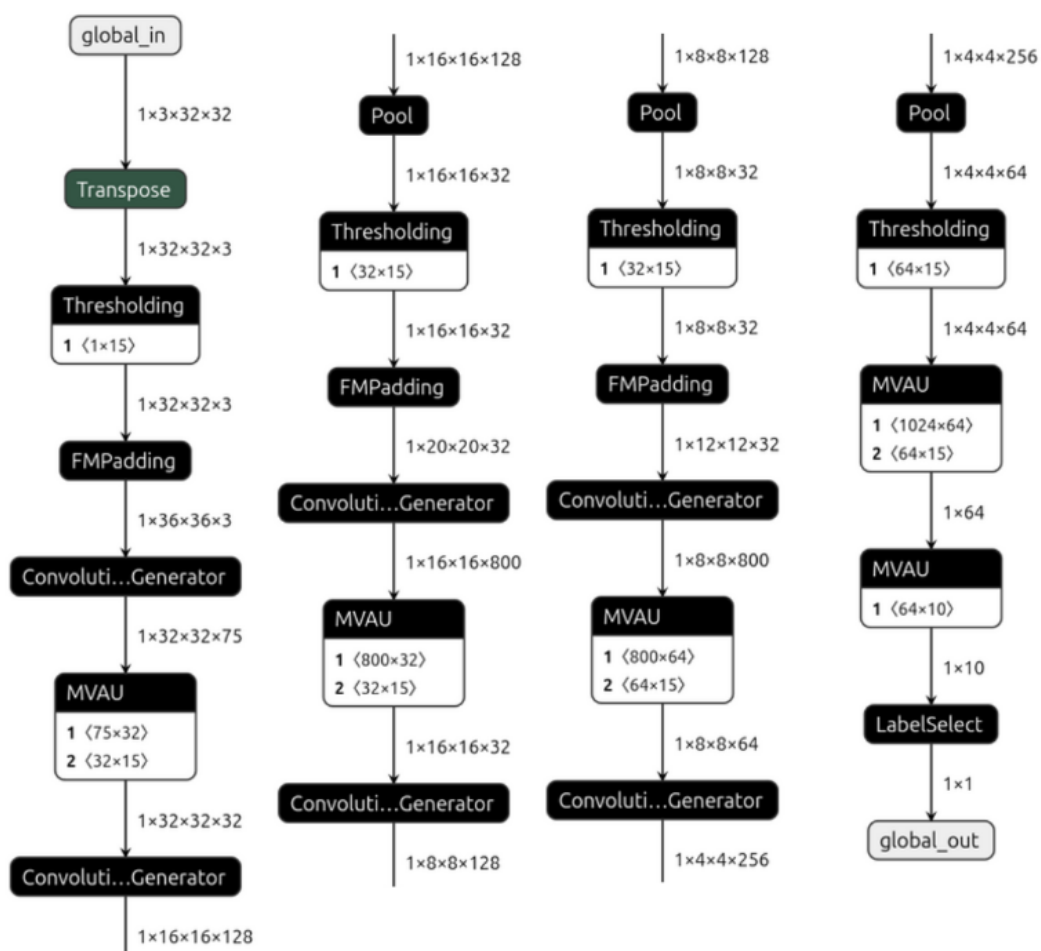
Fonte: o Autor, 2024.

A.2 Modelos para o *dataset* CIFAR-10

Figura 39 – Modelo de precisão total ONNX para o *dataset* CIFAR-10.



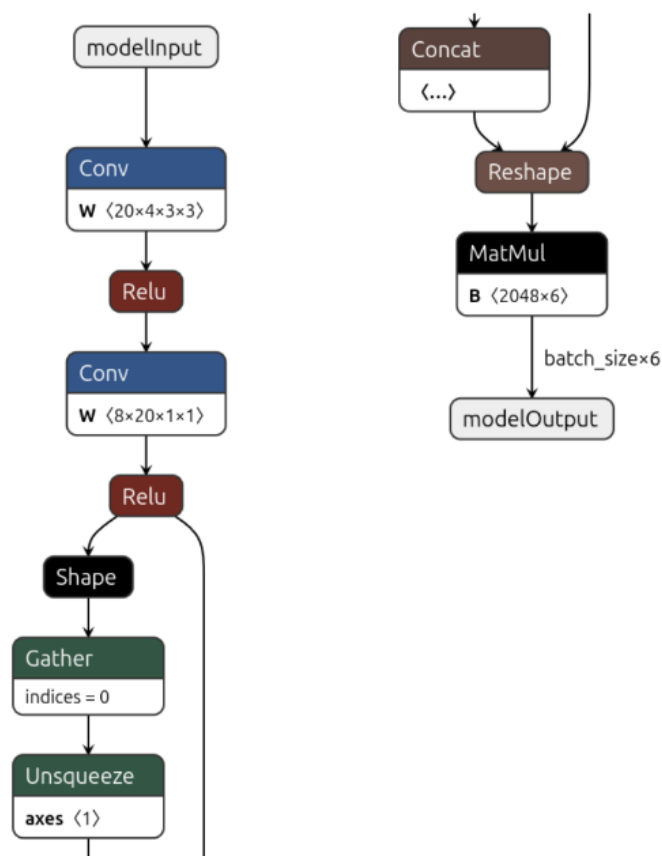
Fonte: o Autor, 2024.

Figura 40 – Modelo FINN para o *dataset* CIFAR-10.

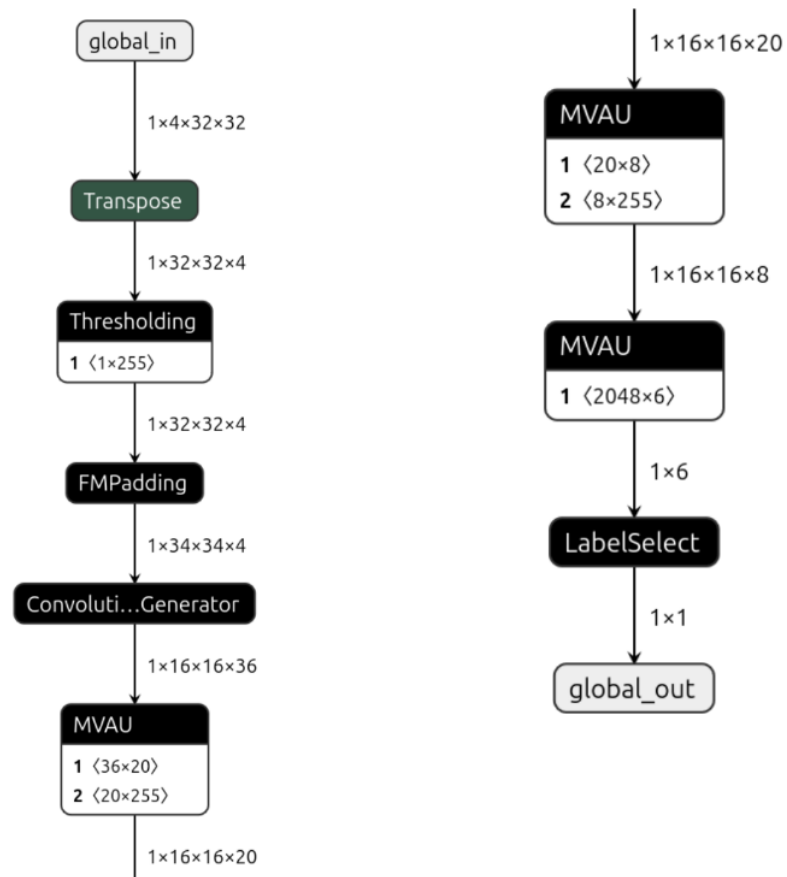
Fonte: o Autor, 2024.

A.3 Modelos para o *dataset* SAT-6

Figura 41 – Modelo de precisão total ONNX para o *dataset* SAT-6.



Fonte: o Autor, 2024.

Figura 42 – Modelo FINN para o *dataset* SAT-6.

Fonte: o Autor, 2024.